

Adaptively Secure Threshold Blind BLS Signatures and Threshold Oblivious PRF

Stanislaw Jarecki[✉] and Phillip Nazarian[✉]

University of California Irvine, USA {sjarecki, pnazaria}@uci.edu

Abstract. We show the first threshold blind signature scheme and threshold Oblivious PRF (OPRF) scheme which remain secure in the presence of an *adaptive adversary*, who can adaptively decide which parties to corrupt throughout the lifetime of the scheme. Moreover, our adaptively secure schemes preserve the minimal round complexity and add only a small computational overhead over prior solutions that offered security only for a much less realistic *static adversary*, who must choose the subset of corrupted parties before initializing the protocol. Our threshold blind signature scheme computes standard BLS signatures while our threshold OPRF computes the 2HashDH OPRF [58], and we prove adaptive security of both schemes in the Algebraic Group Model (AGM). Our adaptively secure threshold schemes are as practical as the underlying standard (i.e. single-server) BLS blind signature [16] and 2HashDH OPRF, and they can be used to add cryptographic fault-tolerance and decentralize trust in any system that relies on blind signatures, like anonymous credentials and e-cash, or on OPRF, like the *OPAQUE* password authentication and the *Privacy Pass* anonymous authentication scheme, among many others.

1 Introduction

Threshold and Blind Signatures. In threshold signatures, introduced by Desmedt and Frankel [41, 42], n parties can compute signatures but corruption of any t parties leaks no information on the signature key. Threshold signatures enhance security and robustness of signatures, and have been a topic of a long line of research, with the recent interest driven by blockchain and other decentralized systems, motivating efficiency and security improvements, e.g. [50, 73, 74, 49, 23, 26, 69, 36, 4, 14, 34, 32], and a NIST call for standardization [18].

Blind signatures, introduced by Chaum [30], enable a server who holds the key, to issue a signature on a message held by a user, in a way that keeps the message hidden to the signer, and the user obtains only a signature on the single message which it specified in the protocol. Blind signatures are a powerful tool that enables many privacy-protecting applications, including ecash [30, 31, 77], anonymous credentials [20, 19], electronic voting [52], and blockchain transactions [57]. They are undergoing standardization efforts [40, 27, 6] and are used in products by Apple [7] and Google [51]. In all these applications the party who holds the signature key is the root of trust. One can

make these schemes fault-tolerant and reduce the risk of key compromise by secret-sharing this key and replacing the blind signature scheme with a *threshold blind signature scheme* (tBSig) [86, 70, 35, 71].

BLS signatures [17], based on a bilinear map group, have the simplest (and round-minimal) threshold signature protocol and blind signature protocol, both proposed by Boldyreva¹ [16]. Vo et al. [86] observed that these two protocols can be naturally composed into a *threshold blind BLS* scheme, which is still round-minimal and essentially as efficient as the BLS signature itself. There are only four further works on provably secure threshold blind signatures. Kuchta and Manulis [70] construct threshold blind signatures secure under the Computational Diffie-Hellman (CDH) assumption, but it is much more computationally intensive than threshold blind BLS. Doerner et al. [43] provide a threshold blind scheme that generates BBS+ signatures and is secure assuming the security of oblivious transfer. Crites et al. [35] show a construction that is secure under the Discrete Logarithm (DL) assumption in the Algebraic Group Model (AGM). Lehmann et al. [71] modify existing schemes to achieve stronger security properties, e.g. issuer set coordination. Besides threshold blind BLS, only the scheme of [70] is round-optimal, and all these works only consider a *static adversary*.²

Threshold Oblivious PRF. An oblivious pseudorandom function (OPRF) is a protocol by which a user and server can jointly evaluate a PRF. Similar to a blind signature scheme, the user does not learn the PRF key and the server does not learn the PRF input. Applications include private set intersection [56, 62, 63, 68], password-protected secret sharing [13, 58, 59, 37], private information retrieval [45], password-authenticated key exchange (PAKE) [61], threshold PAKE [58, 53], and key management for outsourced file systems [60].

Among many OPRF realizations, see the survey on OPRF in [25], a popular low-cost approach is called “(Two-)Hashed Diffie-Hellman” (2HashDH) OPRF [58]. The 2HashDH OPRF of [58] is almost identical to Boldyreva’s blind BLS, except that there is no pairing that allows for public verification of function output. Statically secure threshold OPRF (tOPRF) protocols, which compute 2HashDH OPRF in a threshold setting [59, 53], are likewise very similar to the (statically secure) threshold blind BLS [86].

Static vs. Adaptive security. In the context of threshold cryptosystems an adaptive adversary model considers attackers who can corrupt up to a prescribed threshold of protocol participants, but can decide when and whom

¹ There are many other blind signature schemes, e.g. [78, 81, 2, 83, 35, 29, 65], but these are more complex and are not round-optimal. There are also alternative schemes using bilinear maps that are proven secure under weaker assumptions, e.g. [28, 55], but these schemes are significantly more computationally intensive than the blind BLS of Boldyreva and do not produce standard BLS signatures.

² Some other threshold blind signature schemes have been proposed under informal security models and proofs, e.g. [66, 79]. There have also been recent works targeting closely related notions such as threshold anonymous credentials, e.g. [80]. None of these works consider an adaptive adversary.

to corrupt at any time, possibly on the basis of protocol executions observed so far. By contrast, the static adversary model, where the attacker must decide which subset of participants to corrupt before the protocol starts, is an artificial restriction, without much practical justification beyond the fact that it makes security proofs easier. Indeed, constructing threshold cryptosystems, e.g. threshold signatures, with provable security against adaptive adversary has been a long-standing challenge, e.g. [24, 44, 75, 3, 5, 72], which has recently seen a renewed interest spiked by emerging practical applications of threshold cryptosystems, e.g. [10, 34, 38, 48, 11, 39]. We note that adaptively secure threshold protocols usually tend to be either more expensive or rely on stronger assumptions than schemes known to be statically secure. Our results follow this pattern, as we explain further below.

The applications of threshold *blind* signatures and threshold Oblivious PRF would benefit from adaptive security because static security is an artificial constraint. If a protocol achieves adaptive security with only very mild impact on efficiency (as is the case for the protocols we show), such protocol will be preferable in practice because stronger confidence in application security will outweigh a mild efficiency impact. To pick just one example from the applications of blind signatures and threshold OPRF listed above, an adaptive tOPRF would imply adaptively secure proactive threshold PAKE, applying a black-box tOPRF-to-tPAKE compiler of [53].

We stress that, until our work, there were *no threshold blind signature or threshold OPRF solutions which were shown adaptively secure*. Moreover, with regards specifically to BLS signatures, even for threshold *non-blind* BLS, adaptive security was not known until very recently. First, Bacho and Loss [10] proved adaptive security for Boldyreva’s threshold BLS scheme, relying on the AGM and the One-More Discrete Log (OMDL) assumption. Even more recently, Das and Ren [39] showed a modified threshold BLS scheme, with a very mild impact on efficiency, which is adaptively secure under the Decisional Diffie-Hellman (DDH) and (co-)Computational Diffie-Hellman (CDH) assumptions, but not the AGM.

Our contributions. We provide the first tBSig and tOPRF schemes with a proof of adaptive security. Our proofs rely on the AGM and the OMDL, DDH, and Strong Discrete Log (SDL) assumptions in the Random Oracle Model (ROM). These dependencies are strong but justified in comparison to previous work. The One-More Diffie-Hellman (OMDH) assumption is necessary for standard, i.e. *single-server*, blind BLS [16] and 2HashDH OPRF [58], and OMDH implies SDL. The AGM and OMDL have been used by proofs of adaptively secure *non-blind* threshold signatures, including threshold BLS [10] and threshold Schnorr [34], and the only proof of *statically secure* tOPRF [53] uses DDH and OMDH assumptions³, and all these works assume ROM. We note that our adaptively-secure tOPRF and BLS-based tBSig schemes impose

³ A secure tOPRF based on just OMDH was claimed by [59], but [53] showed an error in the proof of [59], and used the DDH assumption to fix it.

only a small computational overhead over resp. the statically-secure tOPRF of [53] and statically-secure threshold blind BLS [16, 86].

We note that we define adaptive tOPRF using game-based definitions comparable to the game-based definitions of threshold blind signatures [70, 35]. However, in Appendix D we show a simple extension of our game-based tOPRF which realizes an adaptively secure *Universally Composable (UC) tOPRF* formalized in [53], and computes the same PRF as the 2HashDH OPRF of [58]. Since our proofs are in the AGM, we show the above in the UC framework adapted to AGM by Abdalla et al. [1]. This framework is limited in that security does not necessarily compose with other protocols using the same group setup. This limitation is mitigated by our use of a “global/multi-session” UC functionality, which means that it is at least safe to self-compose many tOPRF instances using the same group setup.

From a technical standpoint, we build on a novel blinding technique introduced in two recent works, the adaptive threshold (but not blind) BLS signature of [39] and the (static) threshold OPRF of [53].⁴ The technique adds a blinding factor to threshold exponentiation, by masking it with an exponentiation using an independent base, sampled via an RO hash, and exponents which form a zero-sharing (which causes the blinding factor to vanish after interpolation). The two works used this technique for very different purposes: [39] used it to prove adaptive security for *non-blind* threshold BLS, while [53] used it to eliminate attacks on *blind* threshold exponentiation/BLS, present already in the static setting. We combine and expand on both usages of these blinding factors (and we use two such factors instead of one) to realize adaptive threshold blind BLS and oblivious PRF, and our techniques (see below) may have further applications to threshold cryptosystems.

Lastly, in most of the paper our main focus is on tOPRF, but throughout we use `shadowed text` to mark the changes required to shift from tOPRF to tBSig. Indeed, one can see (e.g. in our main protocol, Figure 3) that these shifts are small. Intuitively, the protocols differ only in the presence of the bilinear map which can be used for verification of the tOPRF output, which in the tBSig context is interpreted as a signature. Only in our technical overview we favor discussion of tBSig over tOPRF in order to more easily compare our work to the threshold signature scheme of [39].

1.1 Technical Overview

In this section, we provide a high-level explanation of the main ideas in our work. As a jumping-off point, we begin with a simplified overview of the adaptively secure threshold *non-blind* BLS scheme of Das and Ren [39], and we overview the security challenges encountered by its threshold *blind* counterpart. Overcoming these challenges requires several novel techniques, which we also outline here.

⁴ Works concurrent to ours [9, 8] have also used this blinding technique to construct adaptively secure (non-blind) threshold Schnorr signature schemes.

Das-Ren argument for adaptive threshold *non-blind* BLS. Recall that the BLS signature is $y = H_0(x)^k$ where $\text{pk} = g^k$ is the public key, k is the secret key, and H_0 is a hash onto group $\langle g' \rangle$ s.t. there is a bilinear map on $\langle g \rangle \times \langle g' \rangle$. In Das and Ren’s threshold BLS, servers hold a Shamir secret-sharing $\{k_i\}$ of k and a sharing $\{z_i\}$ of $z = 0$. To sign message x , server i computes partial signature $q_i := H_0(x)^{k_i} \cdot H_1(x)^{z_i}$, where H_1 is another hash onto group $\langle g' \rangle$. Using interpolation in the exponent the user can aggregate q_i ’s to recover

$$y := \prod_{i \in \mathbf{S}} (q_i)^{\lambda_i} = H_0(x)^{\sum_{i \in \mathbf{S}} \lambda_i k_i} \cdot H_1(x)^{\sum_{i \in \mathbf{S}} \lambda_i z_i} = H_0(x)^k \cdot H_1(x)^0 = H_0(x)^k$$

for a large enough subset $\mathbf{S}$ and Lagrange interpolation coefficients $\{\lambda_i\}_{i \in \mathbf{S}}$. Note that the “blinding factors” $H_1(x)^{z_i}$ in the partial signatures disappear from the final signature, but their presence is integral to the proof of security.

In proving adaptive security, the reduction must be able to, upon corruption of any server, produce secret keys (k_i, z_i) that explain that server’s previous partial signatures. This thwarts a simple Computational Diffie-Hellman (CDH) reduction that sets $\text{pk} := g^\alpha$ where g^α is one of the two CDH challenge group elements, and uses trapdoors embedded in H_0 responses to compute partial signatures in spite of not knowing α , and therefore not knowing all k_i . Das and Ren’s idea is that the reduction generates all k_i ’s honestly, but generates z_i ’s as a sharing of random z rather than $z = 0$, and embeds the CDH challenge in H_1 responses. Specifically, the reduction “rigs” $\text{pk} := g^k \cdot g^\alpha = g^{k+\alpha}$, and for all x queried to H_0 except one of them chosen at random, it sets $H_1(x)$ and $H_0(x)$ s.t. $H_1(x)^z = H_0(x)^\alpha$, and this way partial signatures $q_i = H_0(x)^{k_i} \cdot H_1(x)^{z_i}$ interpolate to $H_0(x)^k \cdot H_1(x)^z = H_0(x)^{k+\alpha}$. This correlated programming of H_0, H_1 is possible using only g^α and z (and if the reduction trapdoors H_0 outputs as $H_0(x) := g^{\tau_x}$ for random τ_x ’s), and it is undetectable under the DDH assumption. If the adversary corrupts only the allowed threshold of servers then the fact that $z \neq 0$ is hidden, and all appears as normal if the key was $k' = k + \alpha$. The other group element of the CDH challenge is programmed to $H_0(x^*) := g^\beta$, where x^* is randomly chosen from the queries to H_0 , and if the adversary outputs forgery $y^* = H_0(x^*)^{k+\alpha}$ for the guessed argument x^* , the reduction can extract $g^{\alpha \cdot \beta}$ and thus win CDH.

Moving to threshold *blind* BLS. Using the “blind exponentiation” technique due to Chaum [30], a threshold BLS signature scheme can be made blind [16, 86]: The user computes $p := H_0(x)^r$ for randomly chosen r and sends it to the servers. The servers use p in place of $H_0(x)$ in their partial signatures, and the user “de-blinds” the aggregated result by exponentiating to $1/r$ to arrive at the final signature. To make Das and Ren’s threshold BLS scheme blind, the servers can use p instead of x in their blinding factors, i.e. set $q_i := p^{k_i} \cdot H_1(p)^{z_i}$.

At first glance, it might seem that Das and Ren’s proof techniques are still immediately applicable to this *blind* threshold BLS scheme, but this is not the case. In the newly blinded scheme an adversarial user can query the servers using arbitrary p values, and this introduces major complications in the proof of security. There are four types of adversarial query behaviors that our proof

must handle. Below we summarize these patterns and sketch the techniques we use to achieve and prove security in spite of them. Our final protocol uses two blinding factors (with two zero-sharings) and sets $q_i := p^{k_i} \cdot H_1(p)^{z_i} \cdot H_2(p)^{\hat{z}_i}$.

#1: Blinded Queries. Suppose the reduction programs $H_0(x) := g^{\tau_x}$ for a random trapdoor τ_x . An honest user will then pick random r and send $p := H_0(x)^r = g^{\tau_x \cdot r}$ to the servers. If our reduction were to follow the above sketch, it would have to program $H_1(p) := p^{\alpha/z} = (g^\alpha)^{(\tau_x \cdot r)/z}$ given the CDH challenge g^α , so that the de-blinded partial signatures interpolate to $H_0(x)^{k+\alpha}$. This seems hard, though, because p is a random group element, and it is not clear how the reduction could link it to exponents τ_x and r .

We resort to the Algebraic Group Model (AGM) to solve this: If the adversary must accompany $p = H_0(x)^r$ with its algebraic representation, i.e. base $H_0(x)$ and exponent r , then the reduction learns x, r and can set $H_1(p) := (g^\alpha)^{(\tau_x \cdot r)/z}$.

#2: Re-Randomized Queries. Suppose the adversary is an honest user who re-uses some input x , i.e. it sets $p_1 := H_0(x)^{r_1}$ and $p_2 := H_0(x)^{r_2}$ for random r_1, r_2 . If $H_1(p_1) = (p_1)^{\alpha/z}$ and $H_1(p_2) = (p_2)^{\alpha/z}$ as above, the adversary can detect that these responses are not random because $H_1(p_1)^{1/r_1} = H_1(p_2)^{1/r_2}$.

We solve this problem using a second zero-sharing $\{\hat{z}_i\}$ and making the partial signatures include two blinding factors, i.e. $q_i := p^{k_i} \cdot H_1(p)^{z_i} \cdot H_2(p)^{\hat{z}_i}$. In the reduction, shares z_i and $\hat{z}_i$ interpolate to random z and $\hat{z}$, and the reduction programs H_1, H_2 in a correlated way that achieves the same result as before, setting $H_1(p) := p^{\delta_p \cdot \alpha/z}$ and $H_2(p) := p^{(1-\delta_p) \cdot \alpha/\hat{z}}$, where δ_p is randomly chosen for each p . This way q_i 's still interpolate to $p^{k+\alpha}$, and under the DDH assumption this correlated programming remains undetectable to the adversary.

#3: Mixed Queries. Honest users use only a single $H_0(x)$ base to form p . However, an adversary can instead combine many H_0 outputs to form $p := H_0(x_1)^{r_1} \cdot \dots \cdot H_0(x_L)^{r_L}$. This strategy is not necessarily self-defeating: If the adversary forms L queries $p_1, \dots, p_L$, each p_i using a different randomizer vector $r_{i,1}, \dots, r_{i,L}$, and sends each such p_i to $t+1$ servers and interpolates these answers into p_i^k , then it can extract $H_0(x_1)^k, \dots, H_0(x_L)^k$ as long as these randomizer vectors are linearly independent. This is not a wasteful strategy because it extracts signatures on L values using $L \cdot (t+1)$ queries. Das and Ren's CDH reduction relies upon guessing a single forgery target x^* and embedding the CDH challenge (rather than a trapdoor) into $H_0(x^*)$, but in our case, the above adversary can make $H_0(x^*)$ a component in every query p , and if it contains a challenge, and not a trapdoor, the reduction wouldn't know how to respond (in particular, it could not set $H_1(p)$ and $H_2(p)$ as above).

We circumvent this problem by replacing the above CDH reduction with two reductions, to resp. One-More Discrete Logarithm (OMDL) and Discrete Logarithm (DL). In the first reduction, to OMDL, we embed the t OMDL challenges into the blinding factor shares $z_i, \hat{z}_i$, while the k_i shares are picked honestly. The reduction only knows g^{z_i} and $g^{\hat{z}_i}$, but this is sufficient to compute partial signatures due to the embedding of trapdoors in H_1 and H_2 responses. (This OMDL reduction programs H_1 and H_2 outputs differently

than described above, because at this point in the argument the main key is not yet “rigged”, and pk is simply set as g^k for k known to the reduction.) Upon server corruption, the DL oracle is used to find the correct $z_i, \hat{z}_i$. We use this OMDL reduction to argue that the adversary must form its final signatures via correct interpolation of partial signatures on the same query p ; otherwise, the reduction learns a system of equations that is sufficient to solve for some z_i or $\hat{z}_i$. Consequently, any p that is not queried to at least $t+1$ servers (minus the number of eventually corrupted servers) is useless to the adversary. The above step conceptually simplifies the multiple servers setting into a single signing entity. Moreover, the same argument ensures that the adversary’s final signatures are accompanied by algebraic representations which involve $t+1$ partial signatures for any meaningfully used server query p .

In the second step, the reduction solves DL if the adversary computes the signatures on L arguments but makes less than $L \cdot (t+1)$ partial signature queries (it is $t+1$ minus the number of eventually corrupted parties, but we omit that for simplicity). This reduction proceeds similarly as the reduction described in part #2 above, but the only challenge is g^α : The reduction picks all $k_i, z_i, \hat{z}_i$ shares and correlates H_1, H_2 responses as the reduction in part #2. However, unlike in Das and Ren’s proof, this reduction trapdoors all H_0 outputs (in particular there is no x^* for which $H_0(x^*)$ is programmed differently), and we show that if the adversary’s algebraic representations explain L correct signatures using less than L (times $t+1$) different p queries, then these representations provide a system of equations by which the reduction can solve for α .

#4: Recursive Queries. Lastly, the adversary might perform recursive queries, i.e. the adversary can use a server’s response as part of the input to a new server query. Similarly, the adversary can use H_1 and H_2 outputs to form server queries (and/or new H_1 and H_2 inputs). Recursive and mixed querying behaviors might even occur in tandem, creating bizarre queries like $p = H_0(x_1) \cdot H_1(H_0(x_1) \cdot H_1(H_0(x_2)))$. Unlike the other three querying behaviors, we have no concrete example of an adversary that would act in these ways and succeed; nonetheless, our reductions must be able to handle this behavior.

To solve this problem, we upgrade the aforementioned DL reduction into a q -Strong DL (q -SDL) reduction. In the q -SDL problem, the reduction is given not just g^α , but also $g^{\alpha^2}, g^{\alpha^3}, \dots, g^{\alpha^q}$. These higher powers can be used by the reduction to compute proper H_1 and H_2 responses to recursive random oracle queries. The q -SDL reduction knows all secret keys $\{k_i, z_i, \hat{z}_i\}$, so there is no further problem handling recursive queries to the servers.

2 Preliminaries

2.1 Group Setting and Hardness Assumptions

We defer some preliminaries to Appendix A. In particular: the definitions of bilinear pairings and the Discrete Logarithm (DL), N -Relational Discrete Logarithm (N -RDL), q -Strong (co-)Discrete Logarithm (q -(co-)SDL),

t -One-More Discrete Logarithm (t -OMDL), and Decisional Diffie-Hellman (DDH) assumptions.

2.2 Threshold Oblivious Pseudorandom Function

Threshold OPRF was defined in the UC setting in [59, 53] but here we provide simpler game-based security definitions for tOPRF. We do so for three reasons: First, to break down the monolithic UC definition to several easier to understand properties, e.g. to see which assumption is used to achieve which property. Second, to relate the tOPRF properties to the corresponding properties of threshold Blind Signatures in Section 2.3. Third, in Appendix D we show that a scheme that satisfies the game-based tOPRF properties can be generically modified, by hashing the tOPRF input and output via a (Random Oracle) hash, to realize the UC tOPRF functionality of [53].

Throughout this work we refer to the two types of parties in our schemes as *users* and *servers*. Our definitions are simplified for the case of non-interactive, i.e. single-round, protocols, but they can be easily extended to the general case where UEval and SEval consist of multiple sequential rounds.

Definition 1 (tOPRF). A threshold oblivious pseudorandom function *with domain* $\mathcal{X}$ *and codomain* $\mathcal{Y}$ *is a tuple of PPT algorithms* $\text{tOPRF} = (\text{Setup}, \text{KeyGen}, \text{UEval}, \text{SEval}, \text{UAgg}, \text{Offline})$ *with the following properties:*

1. $\text{Setup}(1^\kappa) \rightarrow \text{pp}$. Given an input of security parameter length, the setup algorithm selects public parameters pp for the protocol. The output of Setup is an implicit input to all other algorithms.
2. $\text{KeyGen}(n, t) \rightarrow (\{\text{sk}_i\}_{i \in [n]}, \text{aux})$. Given the number of servers n and threshold t , the key generation algorithm produces a secret key sk_i for each server and auxiliary public data aux .
3. $\text{UEval}(x, \text{aux}) \rightarrow (\text{st}^U, \text{pm}^U)$. Given function input $x \in \mathcal{X}$ and public data aux , the user evaluation algorithm produces a message pm^U sent to all servers, and an internal state st^U to be used later when aggregating the server responses.
4. $\text{SEval}(i, \text{sk}_i, \text{ctx}, \text{pm}^U) \rightarrow \text{pm}^i$. Given server i 's secret key sk_i , server context data ctx , and user message pm^U , the server evaluation algorithm produces a response message pm^i .
5. $\text{UAgg}(\text{st}^U, \mathbf{S}, \{\text{pm}^i\}_{i \in \mathbf{S}}) \rightarrow y/\perp$. Given a state from UEval and server responses $\{\text{pm}^i\}_{i \in \mathbf{S}}$ for a set of $t + 1$ servers $\mathbf{S}$, the user aggregation algorithm produces either a function output $y \in \mathcal{Y}$ or a failure symbol $\perp$.
6. $\text{Offline}(\{\text{sk}_i\}_{i \in [n]}, \text{aux}, x) \rightarrow y$. Given an output of KeyGen and input $x \in \mathcal{X}$, the offline evaluation algorithm produces a function output $y \in \mathcal{Y}$.

tOPRF must further satisfy Deterministic Correctness, Pseudorandomness, Blindness, and One-More Unpredictability, as defined below.

Correctness. Deterministic Correctness ensures that a properly performed function evaluation on any given input always produces the same output, i.e. that the protocol computes a function defined by KeyGen outputs.

Definition 2 (Deterministic Correctness). Consider the following experiment (*), which takes as input security parameter κ , integers n and t s.t. $0 \leq t < n$, $x \in \mathcal{X}$, set $\mathbf{S} \subseteq [n]$ s.t. $|\mathbf{S}| = t + 1$, and $\text{ctx} \in \{0, 1\}^*$:

$$\begin{aligned} \text{pp} &\leftarrow \text{tOPRF.Setup}(1^\kappa) \\ (\{\text{sk}_i\}_{i \in [n]}, \text{aux}) &\leftarrow \text{tOPRF.KeyGen}(n, t) \\ (\text{st}^U, \text{pm}^U) &\leftarrow \text{tOPRF.UEval}(x, \text{aux}) \\ \forall_{i \in \mathbf{S}} \quad \text{pm}^i &\leftarrow \text{tOPRF.SEval}(i, \text{sk}_i, \text{ctx}, \text{pm}^U) \\ y &\leftarrow \text{tOPRF.UAagg}(\text{st}^U, \mathbf{S}, \{\text{pm}^i\}_{i \in \mathbf{S}}) \end{aligned}$$

Threshold oblivious pseudorandom function **tOPRF** is deterministically correct iff algorithm **tOPRF.Offline** is deterministic and, on every possible input, experiment (*) always produces $(\{\text{sk}_i\}_{i \in [n]}, \text{aux}, y)$ such that $y = \text{tOPRF.Offline}(\{\text{sk}_i\}_{i \in [n]}, \text{aux}, x)$.

Pseudorandomness. We define **tOPRF** pseudorandomness using the standard definition for PRF Pseudorandomness applied to the function computed in the honest protocol execution.

Definition 3 (Pseudorandomness). The advantage of adversary $\mathcal{A}$ against protocol **tOPRF** in the pseudorandomness game is defined as:

$$\text{Adv}_{\mathcal{A}, \text{tOPRF}}^{\text{pseudorandom}}(\kappa) = |\Pr[\mathcal{A}^{\text{tOPRF.Offline}(\{\text{sk}_i\}_{i \in [n]}, \text{aux}, \cdot)}(1^\kappa) = 1] - \Pr[\mathcal{A}^{f(\cdot)}(1^\kappa) = 1]|$$

where in the first probability $\text{pp} \leftarrow \text{tOPRF.Setup}(1^\kappa)$ and $(\{\text{sk}_i\}_{i \in [n]}, \text{aux}) \leftarrow \text{tOPRF.KeyGen}(n, t)$ and in the second probability f is a uniformly chosen function $\mathcal{X} \rightarrow \mathcal{Y}$. Before beginning, $\mathcal{A}$ outputs integers n and t s.t. $0 \leq t < n$.

Threshold oblivious pseudorandom function **tOPRF** is pseudorandom iff, for all PPT adversaries $\mathcal{A}$, $\text{Adv}_{\mathcal{A}, \text{tOPRF}}^{\text{pseudorandom}}(\kappa)$ is negligible.

Verifiability. We define a **tOPRF** verifiability property, which ensures that dishonest servers cannot trick a user into performing an incorrect evaluation. Note that a verifiable **tOPRF** is not necessarily a **tBSig**. Verifiability only requires that the user be assured of correct behavior; they may not be able to prove it to others (e.g. because they were convinced of it by an interactive proof or because doing so would compromise the blindness of their sessions).

Definition 4 (Verifiability). The verifiability game $\text{Game}_{\mathcal{A}, \text{tOPRF}}^{\text{verifiable}}(\kappa)$ is defined as:

$$\begin{aligned} \text{pp} &\leftarrow \text{tOPRF.Setup}(1^\kappa) \\ (n, t, \text{st}^A) &\leftarrow \mathcal{A}(1^\kappa) \\ (\{\text{sk}_i\}_{i \in [n]}, \text{aux}) &\leftarrow \text{tOPRF.KeyGen}(n, t) \\ (x, \text{st}^A) &\leftarrow \mathcal{A}(\text{st}^A, \{\text{sk}_i\}_{i \in [n]}, \text{aux}) \\ (\text{st}^U, \text{pm}^U) &\leftarrow \text{tOPRF.UEval}(x, \text{aux}) \\ (\mathbf{S}, \{\text{pm}^i\}_{i \in \mathbf{S}}) &\leftarrow \mathcal{A}(\text{st}^A, \text{pm}^U) \\ y &\leftarrow \text{tOPRF.UAagg}(\text{st}^U, \mathbf{S}, \{\text{pm}^i\}_{i \in \mathbf{S}}) \end{aligned}$$

The advantage of adversary $\mathcal{A}$ in the verifiability game is defined as:

$$\mathbf{Adv}_{\mathcal{A}, \text{tOPRF}}^{\text{verifiable}} = \Pr[y \neq \perp \wedge y \neq \text{tOPRF.Offline}(\{\text{sk}_i\}_{i \in [n]}, \text{aux}, x)]$$

where $(\{\text{sk}_i\}_{i \in [n]}, \text{aux}, x, y) \leftarrow \mathbf{Game}_{\mathcal{A}, \text{tOPRF}}^{\text{verifiable}}(\kappa)$.

Threshold oblivious pseudorandom function tOPRF is verifiable iff, for all PPT adversaries $\mathcal{A}$, $\mathbf{Adv}_{\mathcal{A}, \text{tOPRF}}^{\text{verifiable}}(\kappa)$ is negligible.

Blindness and Obliviousness. We next define the closely related properties of Blindness and Obliviousness. The Blindness property ensures that servers do not learn the function inputs that users are evaluating. Even if function inputs and outputs are later revealed, the servers still cannot link those evaluations to the particular sessions that produced them. This property is required to hold even if keygen and all servers are malicious (i.e. $\mathcal{A}$ has full control over aux and $\{\text{pm}^i\}_{i \in \mathcal{S}}$). This definition has appeared before in [70, 35, 71].⁵

$\mathbf{Game}_{\mathcal{A}, \Pi}^{\text{blind/oblivious}}(\kappa)$ $\text{pp} \leftarrow \Pi.\text{Setup}(1^\kappa)$ $S_{\text{started}}, S_{\text{completed}} := \emptyset$ $\theta \leftarrow_{\$} \{0, 1\}$ $\theta' \leftarrow \mathcal{A}^{\mathcal{O}^{\text{UEval}}[\mathcal{O}^{\text{UAgg}}]}(1^\kappa)$ if $\theta = \theta'$ return 1 else return 0	$\mathcal{O}^{\text{UEval}}(sid, \text{aux}_{sid}, x_{0,sid}, x_{1,sid})$ if $sid \in S_{\text{started}}$ return $\perp$ $S_{\text{started}} := S_{\text{started}} \cup \{sid\}$ $(\text{st}_{0,sid}^U, \text{pm}_{0,sid}^U) \leftarrow \Pi.\text{UEval}(x_{\theta,sid}, \text{aux}_{sid})$ $(\text{st}_{1,sid}^U, \text{pm}_{1,sid}^U) \leftarrow \Pi.\text{UEval}(x_{1-\theta,sid}, \text{aux}_{sid})$ return $(\text{pm}_{0,sid}^U, \text{pm}_{1,sid}^U)$
$\mathcal{O}^{\text{UAgg}}(sid, \mathbf{S}_{0,sid}, \mathbf{S}_{1,sid}, \{\text{pm}_{0,sid}^i\}_{i \in \mathbf{S}_{0,sid}}, \{\text{pm}_{1,sid}^i\}_{i \in \mathbf{S}_{1,sid}})$ if $sid \notin S_{\text{started}} \vee sid \in S_{\text{completed}}$ return $\perp$ $S_{\text{completed}} := S_{\text{completed}} \cup \{sid\}$ $y_{\theta,sid} \leftarrow \Pi.\text{UAgg}(\text{st}_{\theta,sid}^U, \mathbf{S}_{\theta,sid}, \{\text{pm}_{\theta,sid}^i\}_{i \in \mathbf{S}_{\theta,sid}})$ $y_{1-\theta,sid} \leftarrow \Pi.\text{UAgg}(\text{st}_{1-\theta,sid}^U, \mathbf{S}_{1-\theta,sid}, \{\text{pm}_{1-\theta,sid}^i\}_{i \in \mathbf{S}_{1-\theta,sid}})$ if $y_{\theta,sid} = \perp \vee y_{1-\theta,sid} = \perp$ return $(\perp, \perp)$ else return $(y_{\theta,sid}, y_{1-\theta,sid})$	

Fig. 1. The blindness game $\mathbf{Game}_{\mathcal{A}, \Pi}^{\text{blind}}(\kappa)$ (include boxed text) and the obliviousness game $\mathbf{Game}_{\mathcal{A}, \Pi}^{\text{oblivious}}(\kappa)$ (omit boxed text) for a threshold oblivious pseudorandom function or threshold blind signature scheme.

Definition 5 (Blindness). The advantage of adversary $\mathcal{A}$ against protocol tOPRF in the blindness game $\mathbf{Game}_{\mathcal{A}, \text{tOPRF}}^{\text{blind}}(\kappa)$ (see Figure 1), is defined as:

$$\mathbf{Adv}_{\mathcal{A}, \text{tOPRF}}^{\text{blind}}(\kappa) = |\Pr[\mathbf{Game}_{\mathcal{A}, \text{tOPRF}}^{\text{blind}}(\kappa) = 1] - 1/2|$$

Threshold oblivious pseudorandom function tOPRF is blind iff, for all PPT adversaries $\mathcal{A}$, $\mathbf{Adv}_{\mathcal{A}, \text{tOPRF}}^{\text{blind}}(\kappa)$ is negligible.

⁵ Our Obliviousness and Blindness definitions also closely resemble the POPRIV1/2 definitions of [85], which concern (non-threshold) *partially*-oblivious PRFs.

The Obliviousness property, an extension of the corresponding property of single-server OPRFs, is a weaker variant of blindness, where $\mathcal{A}$ has no access to $\mathcal{O}^{\text{UAgg}}$. In other words, the adversary does not get to see the user’s eventual function outputs. Consequentially, a scheme which is oblivious but not blind could allow the adversary to rig some evaluation sessions to produce incorrect outputs: Since those outputs are not revealed to the adversary, this rigging doesn’t help the adversary discern which inputs correspond to which sessions.

Definition 6 (Obliviousness). *The advantage of adversary $\mathcal{A}$ against protocol tOPRF in the obliviousness game $\text{Game}_{\mathcal{A}, \text{tOPRF}}^{\text{oblivious}}(\kappa)$ (see Figure 1), is defined as:*

$$\text{Adv}_{\mathcal{A}, \text{tOPRF}}^{\text{oblivious}}(\kappa) = |\Pr[\text{Game}_{\mathcal{A}, \text{tOPRF}}^{\text{oblivious}}(\kappa) = 1] - 1/2|$$

Threshold oblivious pseudorandom function tOPRF is oblivious iff, for all PPT adversaries $\mathcal{A}$, $\text{Adv}_{\mathcal{A}, \text{tOPRF}}^{\text{oblivious}}(\kappa)$ is negligible.

Blindness implies Verifiability. Blindness is achievable only if the user holds the public data aux . Otherwise the user could not detect the session-rigging attack described above. This provides a hint that blindness is closely related to the user’s ability to verify the correctness of the messages $\{\text{pm}^i\}_{i \in \mathbf{S}}$ received from the servers, i.e. that blindness implies verifiability. To see this, suppose there existed an adversary $\mathcal{A}$ that broke verifiability. An adversary in the blindness game could run $\mathcal{A}$ as a subroutine and use the resulting $(\text{aux}, x, \mathbf{S}, \{\text{pm}_i\}_{i \in \mathbf{S}})$ as input 0 to $\mathcal{O}^{\text{UEval}}$ and $\mathcal{O}^{\text{UAgg}}$. For input 1, the adversary could use the same x , but coupled with honest SEval responses. With non-negligible probability, the UEval and UAgg executions faced with input 0 will produce an incorrect function output. The executions faced with input 1 will (by deterministic correctness) always produce a correct function output. The adversary, who sees these outputs, clearly has non-negligible advantage in discerning which one corresponds to the session that was rigged.

Although verifiability is a valued property for OPRFs, many applications require users to evaluate OPRFs without having reliable access to trusted public data aux . We explain in Appendix B how our tOPRF scheme gracefully degrades from blind to merely oblivious when users run without input aux (in which case verifiability is lost as well).

One-More Unpredictability. Finally we define One-More Unpredictability, which ensures that users cannot evaluate the function on more inputs than allowed by the servers. Specifying precisely how many evaluations should be allowed after a given series of server actions is not trivial. This definitional question was recently studied by Lehmann et al. [71], who proposed a hierarchy of One-More Unforgeability (OMUF) definitions for threshold blind signatures, inspired by the similar Unforgeability hierarchy proposed by Bellare et al. [15] for non-blind threshold signatures. Our One-More Unpredictability definition is adapted from the “OMUF-2” notion of [71], which allows a single evaluation for each input $(\text{ctx}, \text{pm}^U)$ that is processed by (at least) $t + 1$ servers. [71] show this notion to be stronger than those previously considered for threshold blind

signatures. Previous works on tOPRFs [59, 53] have instead used simulation-based security definitions. The functionality of [53] includes an OMUF-2-like ctx-binding property, which is useful to applications that wish to enforce server agreement on arbitrary side data, e.g. a timestamp or user ID. In Appendix D we show that a scheme satisfying our game-based tOPRF properties can be generically modified to realize the UC tOPRF functionality of [53], which suggests that our One-More Unpredictability game provides security guarantees comparable to those of the previous tOPRF works.

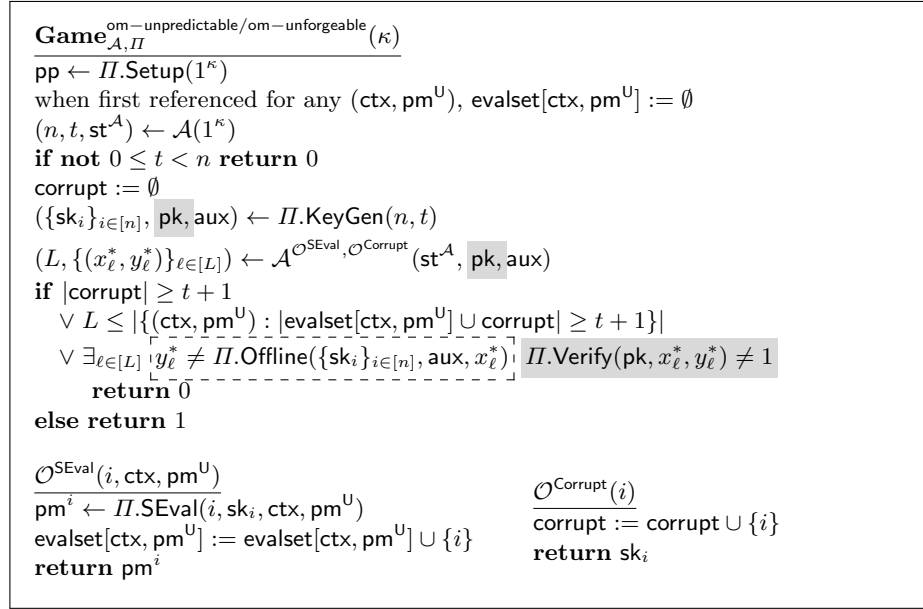


Fig. 2. The one-more unpredictability game **Game**_{A,tOPRF}^{om-unpredictable}(κ) for a threshold oblivious pseudorandom function (dashed text) and the one-more unforgeability game **Game**_{A,tBSig}^{om-unforgeable}(κ) for a threshold blind signature scheme (shadowed text).

Definition 7 (One-More Unpredictability). *The advantage of adversary A against protocol tOPRF in the one-more unpredictability game **Game**_{A,tOPRF}^{om-unpredictable}(κ) (as defined in Figure 2), is defined as:*

$$\text{Adv}_{\mathcal{A}, \text{tOPRF}}^{\text{om-unpredictable}}(\kappa) = \Pr[\text{Game}_{\mathcal{A}, \text{tOPRF}}^{\text{om-unpredictable}}(\kappa) = 1]$$

Threshold oblivious pseudorandom function tOPRF is one-more unpredictable iff, for all PPT adversaries A, $\text{Adv}_{\mathcal{A}, \text{tOPRF}}^{\text{om-unpredictable}}(\kappa)$ is negligible.

2.3 Threshold Blind Signature

In this section we define Threshold Blind Signature (tBSig), a functionality very similar to tOPRF. Unlike a tOPRF, a tBSig need not be deterministic nor

pseudorandom, but it must be *publicly* verifiable (contrast with the non-public tOPRF verifiability property above (Definition 4)).

Definition 8 (tBSig). A threshold blind signature scheme with message space $\mathcal{X}$ and signature space $\mathcal{Y}$ is a tuple of PPT algorithms $\text{tBSig} = (\text{Setup}, \text{KeyGen}, \text{UEval}, \text{SEval}, \text{UAgg}, \text{Verify})$ with the following properties:

1. $\text{Setup}(1^\kappa) \rightarrow \text{pp}$.
2. $\text{KeyGen}(n, t) \rightarrow (\{\text{sk}_i\}_{i \in [n]}, \text{pk}, \text{aux})$. The key generation algorithm outputs a public verification key pk .
3. $\text{UEval}(x, \text{pk}, \text{aux}) \rightarrow (\text{st}^U, \text{pm}^U)$.
4. $\text{SEval}(i, \text{sk}_i, \text{ctx}, \text{pm}^U) \rightarrow \text{pm}^i$.
5. $\text{UAgg}(\text{st}^U, \mathbf{S}, \{\text{pm}^i\}_{i \in \mathbf{S}}) \rightarrow y/\perp$.
6. $\text{Verify}(\text{pk}, x, y) \rightarrow 0/1$. Given a public key pk , a message $x \in \mathcal{X}$, and a purported signature $y \in \mathcal{Y}$, the verification algorithm confirms whether the signature is valid.

A *tBSig* is further required to satisfy Verifiable Correctness, Blindness, and One-More Unforgeability, as defined below.

Verifiable Correctness ensures that a properly performed signing process on any given message really does produce a valid signature.

Definition 9 (Verifiable Correctness). Consider the following experiment $(*)$, which takes as input security parameter κ , integers n and t s.t. $0 \leq t < n$, $x \in \mathcal{X}$, set $\mathbf{S} \subseteq [n]$ s.t. $|\mathbf{S}| = t + 1$, and $\text{ctx} \in \{0, 1\}^*$:

$$\begin{aligned}
&\text{pp} \leftarrow \text{tBSig.Setup}(1^\kappa) \\
&(\{\text{sk}_i\}_{i \in [n]}, \text{pk}, \text{aux}) \leftarrow \text{tBSig.KeyGen}(n, t) \\
&(\text{st}^U, \text{pm}^U) \leftarrow \text{tBSig.UEval}(x, \text{aux}) \\
&\forall_{i \in \mathbf{S}} \quad \text{pm}^i \leftarrow \text{tBSig.SEval}(i, \text{sk}_i, \text{ctx}, \text{pm}^U) \\
&y \leftarrow \text{tBSig.UAgg}(\text{st}^U, \mathbf{S}, \{\text{pm}^i\}_{i \in \mathbf{S}})
\end{aligned}$$

Threshold blind signature scheme tBSig is verifiably correct iff, on every possible input, experiment $(*)$ always produces (pk, y) such that $\text{tBSig.Verify}(\text{pk}, x, y) = 1$.

The Blindness property is identical to the tOPRF Blindness property.

Definition 10 (Blindness). Threshold blind signature scheme tBSig is blind iff $(\text{tBSig.Setup}, \text{tBSig.KeyGen}, \text{tBSig.UEval}, \text{tBSig.SEval}, \text{tBSig.UAgg})$ satisfies threshold oblivious pseudorandom function blindness (Definition 5) (where aux is parsed as (pk, aux) in the input to UEval).

Finally we consider One-More Unforgeability, which we define almost identically to tOPRF One-More Unpredictability. We merely alter the adversary's win condition in the game to match tBSig 's different notion of

correctness. Note that this definition enforces “strong” unforgeability, under which an adversary must be unable to re-randomize a signature on some message into a different signature on the same message. For “weak” unforgeability, $\mathbf{Game}^{\text{om-unforgeable}}_{\mathcal{A}, \text{tBSig}}$ could be amended so that the win condition requires every x_k^* to be unique.

Definition 11 (One-More Unforgeability). *The advantage of adversary $\mathcal{A}$ against protocol tBSig in the one-more unforgeability game $\mathbf{Game}^{\text{om-unforgeable}}_{\mathcal{A}, \text{tBSig}}(\kappa)$ (as defined in Figure 2), is defined as:*

$$\mathbf{Adv}^{\text{om-unforgeable}}_{\mathcal{A}, \text{tBSig}}(\kappa) = \Pr[\mathbf{Game}^{\text{om-unforgeable}}_{\mathcal{A}, \text{tBSig}}(\kappa) = 1]$$

Threshold blind signature scheme tBSig is one-more unforgeable iff, for all PPT adversaries $\mathcal{A}$, $\mathbf{Adv}^{\text{om-unforgeable}}_{\mathcal{A}, \text{tBSig}}(\kappa)$ is negligible.

3 The 2B-HashTDH and 2B-tBlindBLS Protocols

Figure 3 presents our main protocols, the threshold oblivious pseudorandom function scheme 2B-HashTDH (“twice-blind threshold hashed Diffie-Hellman”) and the threshold blind signature scheme 2B-tBlindBLS (“twice-blind threshold blind BLS”). The term “twice-blind” refers to the *two* blinding factors applied to the server responses.

Note that scheme 2B-HashTDH is a threshold oblivious evaluation of function $F_k(x) = H_0(x)^k$. In Appendix D we show that a slight variant of this scheme, where the final output is re-defined as $F'_k(x) = \tilde{H}(x, F_k(x))$ for a random oracle hash $\tilde{H}$, realizes the universally composable tOPRF notion of [53].

The protocols in Figure 3 utilize two non-interactive zero knowledge proof of knowledge systems (NIZKs), denoted $\text{NIZK}_{\text{KeyGen}}$ and $\text{NIZK}_{\text{SEval}}$. Intuitively, the former ensures that public keys are well-formed, and the latter ensures that servers behave honestly. Formal definitions of these NIZKs appear in Appendix B; both have standard DL-based realizations. Note that they are only relevant to the proof of blindness and play no role in one-more unpredictability/unforgeability.

Our schemes differ from the simplest tOPRF and tBSig protocols [59, 86] only in the addition of the two server response blinding factors and the NIZKs. The former come at no cost to the users and a low cost to servers: just the storage of key shares z_i and $\hat{z}_i$, computing two hash onto a group operations, and replacing an exponentiation with a multi-exponentiation on three bases. The blinding factors’ importance is explained in detail in the technical overview (Section 1.1). Our NIZKs impose a more significant (though still reasonable) overhead, but in section B.1 we explain that the cost of NIZKs can be avoided or mitigated in many important cases.

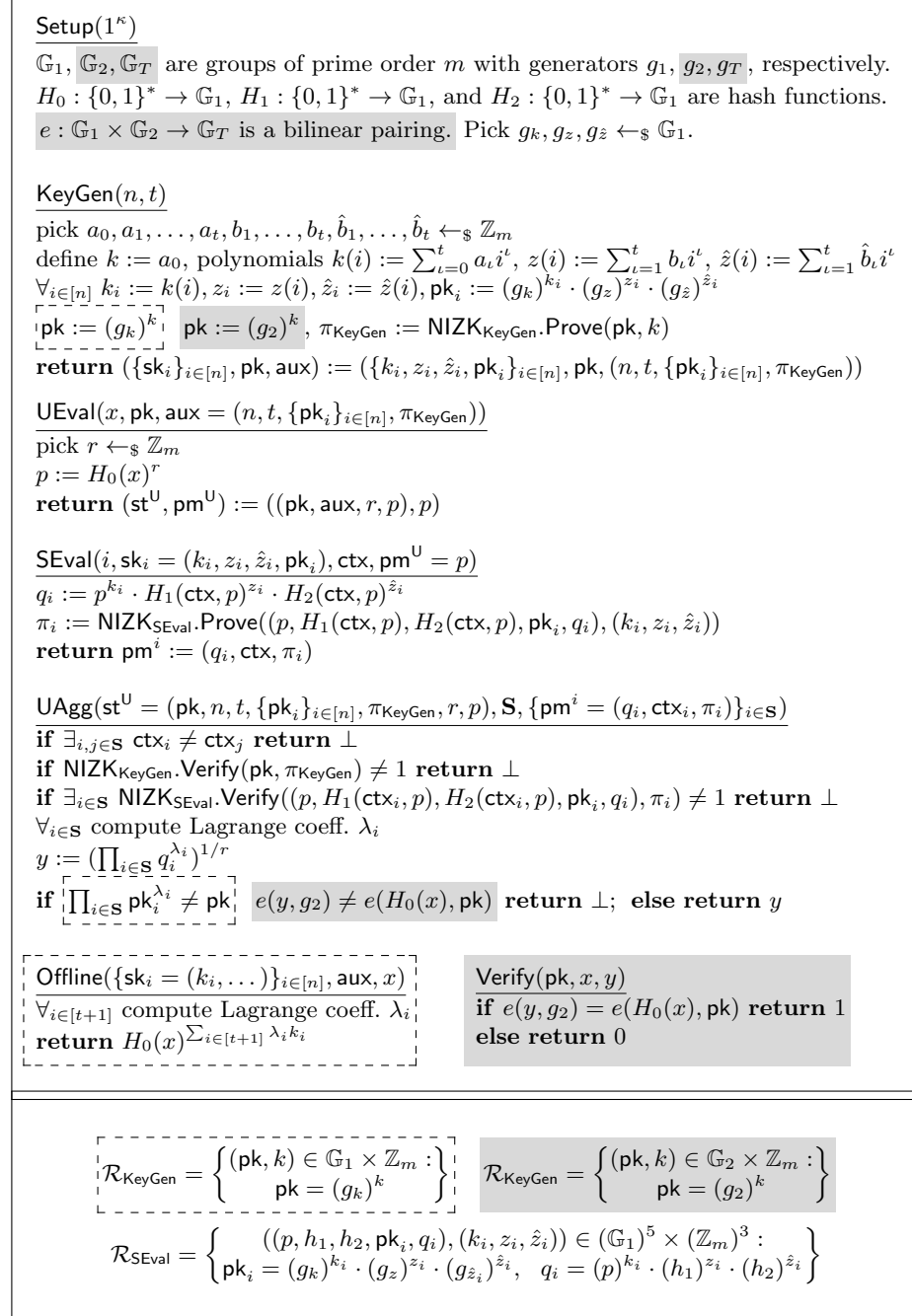


Fig. 3. The threshold oblivious pseudorandom function protocol 2B-HashTDH (dashed text) and the threshold blind BLS signature protocol 2B-tBlindBLS (shadowed text). $\mathcal{R}_{\text{KeyGen}}$ and $\mathcal{R}_{\text{SEval}}$ are the relations for $\text{NIZK}_{\text{KeyGen}}$ and $\text{NIZK}_{\text{SEval}}$, respectively.

4 Proof of Adaptive Security

To prove the security of our schemes, we begin by showing that 2B-HashTDH and 2B-tBlindBLS are secure tOPRFs. A simple lemma then implies that 2B-tBlindBLS is also a secure tBSig.

Correctness, Pseudorandomness, and Blindness. It is easy to see that the schemes satisfy deterministic correctness (Definition 2); the function they compute is $\text{Offline}(\{\text{sk}_i\}_{i \in [n]}, \text{aux}, x) = H_0(x)^k$ where k is the secret key that interpolates from $\{k_i\}_{i \in [n]}$. It is also apparent that, under the DDH assumption, this function is pseudorandom (Definition 3); we omit the proof of this fact, which is a well-known and straightforward DDH reduction.

We also wish to prove that 2B-HashTDH and 2B-tBlindBLS are blind. Doing so does not require any novel techniques and is not related to the challenge of adaptive corruptions, so we defer this proof to Appendix C.

One-More Unpredictability. Here we tackle the main challenge of our work: proving the one-more unpredictability of 2B-HashTDH against an adaptive adversary (which will imply the one-more unforgeability of 2B-tBlindBLS). This proof requires very slight differences (mostly in Lemma 1) for the case of 2B-tBlindBLS, due to its different definition of pk . We mark the differences with shadowed text.

Preliminaries. We begin by classifying all the $\mathbb{G}_1$ elements received and sent by an adversary $\mathcal{A}$ in the one-more unpredictability game $\text{Game}_{\mathcal{A}, 2\text{B-HashTDH}}^{\text{om-unpredictable}}$. At the game's start $\mathcal{A}$ sees $g_1, g_k, g_z, g_{\hat{z}}, \text{pk}, \text{pk}_1, \dots, \text{pk}_n \in \mathbb{G}_1$. Throughout the game, $\mathcal{A}$ sends queries to random oracles H_0, H_1, H_2 . Define that $\mathcal{A}$ queries H_0 at most q times with inputs $x_1, x_2, \dots, x_q \in \{0, 1\}^*$ and receives responses $h_1, h_2, \dots, h_q \in \mathbb{G}_1$. Assume without loss of generality that $\mathcal{A}$ always queries H_1 and H_2 in tandem on the same inputs. Define that $\mathcal{A}$ queries H_1 and H_2 at most $\bar{q}$ times with inputs $(\text{ctx}_1, p_1), (\text{ctx}_2, p_2), \dots, (\text{ctx}_{\bar{q}}, p_{\bar{q}}) \in \{0, 1\}^* \times \mathbb{G}_1$ and receives responses $\bar{h}_1, \bar{h}_2, \dots, \bar{h}_{\bar{q}} \in \mathbb{G}_1$ from H_1 and $\hat{h}_1, \hat{h}_2, \dots, \hat{h}_{\bar{q}} \in \mathbb{G}_1$ from H_2 . Also, $\mathcal{A}$ sends queries to SEval . Assume without loss of generality that $\mathcal{A}$ queries H_1 and H_2 on every (ctx, p) sent to SEval . Define that query $\text{SEval}(i, \text{ctx}_{\bar{j}}, p_{\bar{j}})$ produces response $q_{i, \bar{j}} \in \mathbb{G}_1$ (note that SEval is deterministic). Finally, $\mathcal{A}$ outputs purported function evaluations $(x_1^*, y_1^*), (x_2^*, y_2^*), \dots, (x_L^*, y_L^*) \in \{0, 1\}^* \times \mathbb{G}_1$.

Since we assume that $\mathcal{A}$ is algebraic, every $\mathbb{G}_1$ element sent by $\mathcal{A}$ must be accompanied by an algebraic representation in terms of $\mathcal{A}$'s previously seen $\mathbb{G}_1$ elements. Label the y_ℓ^* representations (using notation $[X]^+ \equiv \{0\} \cup [X]$):

$$y_\ell^* = \prod_{j \in [q]^+} (h_j)^{r_j^\ell} \cdot \prod_{J \in [\bar{q}]^+} (\bar{h}_J)^{s_J^\ell} \cdot (\hat{h}_J)^{u_J^\ell} \cdot \prod_{i, J \in [n] \times [\bar{q}]^+} (q_{i, J})^{v_{i, J}^\ell}$$

Group elements $g_k, g_z, g_{\hat{z}}$ are not explicit in this representation. Looking ahead, all our reductions and simulations will generate these values in the same way as H_0, H_1, H_2 responses, respectively. Therefore, $(g_k, g_z, g_{\hat{z}})$ can be understood as $(h_0, \bar{h}_0, \hat{h}_0)$ for $p_0 := h_0$. For the same reason, $(\text{pk}_1, \dots, \text{pk}_n)$ can

be understood as $(q_{1,0}, \dots, q_{n,0})$. Since $\mathbf{pk}$ can be expressed in terms of $\mathbf{pk}_1, \dots, \mathbf{pk}_n$, assume without loss of generality that $\mathcal{A}$ does not use directly $\mathbf{pk}$ in any representations. Finally, observe that g_1 is never used by our protocols. We therefore omit it here under the simplifying assumption that $g_1 = g_k$.

The H_1 , H_2 , and $\mathbf{SEval}$ inputs $p_{\bar{j}}$ are also $\mathbb{G}_1$ elements sent by $\mathcal{A}$ and so must be accompanied by algebraic representations. Because these values are indexed chronologically, the representation of $p_{\bar{j}}$ cannot include any $\bar{h}_J$, $\hat{\bar{h}}_J$, or $q_{i,J}$ for $J \geq \bar{j}$. Therefore, label them as:

$$p_{\bar{j}} = \prod_{j \in [q]^+} (h_j)^{\bar{r}_{\bar{j}}^j} \cdot \prod_{J=0}^{\bar{j}-1} (\bar{h}_J)^{\bar{s}_{\bar{j}}^J} \cdot (\hat{\bar{h}}_J)^{\bar{u}_{\bar{j}}^J} \cdot \prod_{i \in [n]} \prod_{J=0}^{\bar{j}-1} (q_{i,J})^{\bar{v}_{i,J}^{\bar{j}}}$$

For all $J \in [\bar{q}]^+$, define $\mathbf{S}(J) \subseteq [n]$ to be the set of servers i for which $\exists \ell \in [L] v_{i,J}^\ell \neq 0$ or $\exists \bar{j} \in [\bar{q}] \bar{v}_{i,J}^{\bar{j}} \neq 0$. (Except for $J = 0$) $\mathcal{A}$ must have queried $\mathbf{SEval}$ with (ctx_J, p_J) for every server in $\mathbf{S}(J)$. Assume without loss of generality that $\mathbf{S}(J)$ and corrupt are disjoint; for $i \in \text{corrupt}$, $\mathcal{A}$ knows how to represent $q_{i,J}$ in terms of other elements. Similarly, assume without loss of generality that, for all J , $|\mathbf{S}(J)| \leq t + 1 - |\text{corrupt}|$; $\mathcal{A}$ can represent any $q_{i,J}$ values beyond that threshold in terms of the others using interpolation.

Per the code of $\mathbf{SEval}$, we know that $q_{i,J} = (p_J)^{k_i} \cdot (\bar{h}_J)^{z_i} \cdot (\hat{\bar{h}}_J)^{\hat{z}_i}$. We can rewrite the representation of y_ℓ^* accordingly:

$$y_\ell^* = \prod_{j \in [q]^+} (h_j)^{r_j^\ell} \cdot \prod_{J \in [\bar{q}]^+} (p_J)^{\sum_{i \in \mathbf{S}(J)} v_{i,J}^\ell \cdot k_i} \cdot (\bar{h}_J)^{s_J^\ell + \sum_{i \in \mathbf{S}(J)} v_{i,J}^\ell \cdot z_i} \cdot (\hat{\bar{h}}_J)^{u_J^\ell + \sum_{i \in \mathbf{S}(J)} v_{i,J}^\ell \cdot \hat{z}_i} \quad (1)$$

We can perform the same expansion on the $p_{\bar{j}}$ representations:

$$p_{\bar{j}} = \prod_{j \in [q]^+} (h_j)^{\bar{r}_{\bar{j}}^j} \cdot \prod_{J=0}^{\bar{j}-1} (p_J)^{\sum_{i \in \mathbf{S}(J)} \bar{v}_{i,J}^{\bar{j}} \cdot k_i} \cdot (\bar{h}_J)^{\bar{s}_{\bar{j}}^J + \sum_{i \in \mathbf{S}(J)} \bar{v}_{i,J}^{\bar{j}} \cdot z_i} \cdot (\hat{\bar{h}}_J)^{\bar{u}_{\bar{j}}^J + \sum_{i \in \mathbf{S}(J)} \bar{v}_{i,J}^{\bar{j}} \cdot \hat{z}_i} \quad (2)$$

Without loss of generality, assume $\mathcal{A}$ queries H_0 on every x_ℓ^* and define $j(\ell) \in [q]$ to be the index such that $x_\ell^* = x_{j(\ell)}$.

The Wrapper Algorithm. To bound the probability that $\mathcal{A}$ wins the one-more unpredictability game, we define a wrapper algorithm $\mathcal{B}(\mathcal{A})$ that runs $\mathcal{A}$ as a subroutine, simulating its input and oracle responses (step I). When $\mathcal{A}$ finishes, $\mathcal{B}$ verifies the win conditions and aborts if they fail (step II). $\mathcal{B}$ then performs several additional steps that act as a blueprint for subsequent reductions. In step III, $\mathcal{B}$ iteratively “unrolls” any recursive parts of the algebraic representations provided by $\mathcal{A}$ for its PRF evaluation outputs. Along the way, this step checks that every $\mathbf{SEval}$ response used in those representations appears as part of a set of $t + 1 - |\text{corrupt}|$ properly interpolated responses from different servers. If this check fails, $\mathcal{A}$ must have found an unexpected way of making some blinding factor cancel out. In this case, $\mathcal{B}$ aborts, and an OMDL reduction is implied because a

non-trivial relationship can be extracted between the $\{z_i, \hat{z}_i\}$ values. If step III checks pass, then $\mathcal{B}$ in step IV checks that the PRF evaluation outputs provided by $\mathcal{A}$ are not represented in other unexpected way, i.e. that the secret key does not appear directly in the representation and that recursive query responses are not meaningfully used. If this check fails, $\mathcal{B}$ aborts, and an SDL reduction is implied because a non-trivial equation can be extracted involving the secret element α , a component of the “rigged” effective key k' . Finally, in step V, we use a matrix rank argument to show that, if all previous checks pass, then it is impossible for $\mathcal{A}$ to have produced more than the allowed number of evaluation outputs, which is a contradiction with step II. Since our reductions show that step III and IV aborts occur with negligible probability, it follows that the step II abort must occur with high probability, i.e. that $\mathcal{A}$ does not win the game.

Our SDL reduction requires careful rigging of the public keys and random oracle responses, but our OMDL reduction does not. Also, we must ultimately show that $\mathcal{A}$ cannot win the game in the “real world”, i.e. without rigging. Below we first present the reduction blueprint $\mathcal{B}$ in the “rigged” form. Then, we use two hybrids $\mathcal{B}'$ and $\mathcal{B}''$ to argue that the removal of the rigging behavior is not detectable by $\mathcal{A}$, under the DDH assumption. Our SDL reduction is based on $\mathcal{B}$, and our OMDL reduction is based on $\mathcal{B}''$.

$\mathcal{B}$ step I: run $\mathcal{A}$

- First generate the initial input to $\mathcal{A}$:
 - Pick “rigging values” $\alpha, z, \hat{z} \leftarrow_{\$} \mathbb{Z}_m$.
 - Pick $\tau_k, \delta_z \leftarrow_{\$} \mathbb{Z}_m$ and define public parameters $g_k := (g_1)^{\tau_k}$, $g_z := (g_k)^{(\delta_z) \cdot \alpha / z}$, $g_{\hat{z}} := (g_k)^{(1-\delta_z) \cdot \alpha / \hat{z}}$.
 - Receive (n, t) from $\mathcal{A}$.
 - Pick $a_0 (= k), a_1, \dots, a_t, b_1, \dots, b_t, \hat{b}_1, \dots, \hat{b}_t \leftarrow_{\$} \mathbb{Z}_m$.
 - Define $k(i) := \sum_{\ell=0}^t a_{\ell} i^{\ell}$, $z(i) := \sum_{\ell=1}^t b_{\ell} i^{\ell}$, $\hat{z}(i) := \sum_{\ell=1}^t \hat{b}_{\ell} i^{\ell}$, for all i .
 - Compute honest keys $\forall_{i \in [n]} k_i := k(i), z_i := z(i), \hat{z}_i := \hat{z}(i)$.
 - Define the “effective” PRF key $k' := k + \alpha$ and corresponding public key $\text{pk}' := (g_k)^{k+\alpha}$ (or $\text{pk}' := (g_2)^{k+\alpha}$ for 2B-tBlindBLS).
 - Compute rigged keys $\forall_{i \in [n]} z'_i := z_i + z, \hat{z}'_i := \hat{z}_i + \hat{z}, \text{pk}'_i := (g_k)^{k_i+\alpha} \cdot (g_z)^{z_i} \cdot (g_{\hat{z}})^{\hat{z}_i}$. Set $\{z'_i\}_{i \in [n]}$ and $\{\hat{z}'_i\}_{i \in [n]}$ as sharings of $z' := z + \alpha$ and $\hat{z}' := \hat{z} + \alpha$.
 - Simulate proof π_{KeyGen} (see Appendix B for details).
 - Send $(\text{pk}', (n, t, \{\text{pk}'_i\}_{i \in [n]}, \pi_{\text{KeyGen}}))$ to $\mathcal{A}$.
- Respond to **SEval** and **Corrupt** game oracle queries earnestly using the rigged keys $\{\text{sk}'_i = (k_i, z'_i, \hat{z}'_i, \text{pk}'_i)\}_{i \in [n]}$, except simulate the $\text{NIZK}_{\text{SEval}}$ proofs in **SEval** responses (see Appendix B for details). Maintain **evalset** as in $\text{Game}^{\text{om-unpredictable}}$, indexed by (ctx, p) .
- Respond to H_0 queries using random $\mathbb{G}_1$ elements with known discrete logarithms (i.e. “trapdoors”), i.e. set $h_j = H_0(x_j) := (g_1)^{\tau_j}$ for $\tau_j \leftarrow_{\$} \mathbb{Z}_m$.
- Respond to H_1 and H_2 queries in a manner that is correlated and dependent on the input element. Specifically, pick $\delta_{\bar{j}} \leftarrow_{\$} \mathbb{Z}_m$ and set $\bar{h}_{\bar{j}} = H_1(\text{ctx}_{\bar{j}}, p_{\bar{j}}) := (p_{\bar{j}})^{(\delta_{\bar{j}}) \cdot \alpha / z}$ and $\hat{\bar{h}}_{\bar{j}} = H_2(\text{ctx}_{\bar{j}}, p_{\bar{j}}) := (p_{\bar{j}})^{(1-\delta_{\bar{j}}) \cdot \alpha / \hat{z}}$.

(One can verify at this point that honest evaluation by $\mathcal{A}$ will behave as expected, i.e. $\mathcal{A}$ will correctly compute PRF outputs for the effective key k' . We will later show by DDH reduction that $\mathcal{A}$'s view is indeed indistinguishable from the real world in spite of $\mathcal{B}$'s rigging of keys and random oracle responses.)

$\mathcal{B}$ step II: verify $\mathcal{A}$'s victory

Receive claimed function evaluations $\{(x_\ell^*, y_\ell^*)\}_{\ell \in [L]}$ from $\mathcal{A}$, and verify that (i) $|\text{corrupt}| < t + 1$, (ii) $L > |\{\bar{j} : |\text{evalset}[\text{ctx}_{\bar{j}}, p_{\bar{j}}] \cup \text{corrupt}| \geq t + 1\}|$, and (iii) $\forall \ell \in [L] \ y_\ell^* = H_0(x_\ell^*)^{k'}$. If any condition fails output (II.ADVLOSES) and abort.

$\mathcal{B}$ step III: verify proper interpolation

For all $J \in [\bar{q}]^+$ such that $|\mathbf{S}(J)| = t + 1 - |\text{corrupt}|$, define $\{\lambda_i^J\}$ to be the Lagrange interpolation coefficients for interpolation set $\mathbf{S}(J) \cup \text{corrupt}$. Also, as a shorthand, define $\kappa_J := k' - \sum_{i \in \text{corrupt}} \lambda_i^J k_i$. For all other J (i.e. those for which $|\mathbf{S}(J)| < t + 1 - |\text{corrupt}|$), define $\kappa_J := 0$.

For each $\ell \in [L]$, we use the following loop to eliminate all $p_{\bar{j}}$ elements from the representation of y_ℓ^* . The loop begins with $\bar{j} := \bar{q}$ and ends with $\bar{j} = 0$:

1. At the start of each iteration, we have the following representation for y_ℓ^* , where $\{\mu_J^\ell\}_{J > \bar{j}}$ are defined by the previous iterations of the loop. Note that in the first iteration, i.e. $\bar{j} = \bar{q}$, this representation matches equation (1).

$$\begin{aligned}
y_\ell^* = & \prod_{j \in [q]^+} (h_j)^{r_j^\ell + \sum_{J=\bar{j}+1}^{\bar{q}} \mu_J^\ell \kappa_J \bar{r}_j^J} \\
& \cdot \prod_{J=0}^{\bar{j}} (p_J)^{\sum_{i \in \mathbf{S}(J)} (v_{i,J}^\ell + \sum_{G=\bar{j}+1}^{\bar{q}} \mu_G^\ell \kappa_G \bar{v}_{i,J}^G) \cdot k_i} \\
& \cdot \prod_{J=0}^{\bar{j}} (\bar{h}_J)^{(s_J^\ell + \sum_{G=\bar{j}+1}^{\bar{q}} \mu_G^\ell \kappa_G \bar{s}_J^G) + \sum_{i \in \mathbf{S}(J)} (v_{i,J}^\ell + \sum_{G=\bar{j}+1}^{\bar{q}} \mu_G^\ell \kappa_G \bar{v}_{i,J}^G) \cdot z'_i} \\
& \cdot \prod_{J=0}^{\bar{j}} (\hat{\bar{h}}_J)^{(u_J^\ell + \sum_{G=\bar{j}+1}^{\bar{q}} \mu_G^\ell \kappa_G \bar{u}_J^G) + \sum_{i \in \mathbf{S}(J)} (v_{i,J}^\ell + \sum_{G=\bar{j}+1}^{\bar{q}} \mu_G^\ell \kappa_G \bar{v}_{i,J}^G) \cdot \hat{z}'_i}
\end{aligned} \tag{3}$$

2. (Case 1) Set $\mu_{\bar{j}}^\ell := 0$ if the following three conditions are all satisfied:

$$\begin{aligned}
(s_{\bar{j}}^\ell + \sum_{G=\bar{j}+1}^{\bar{q}} \mu_G^\ell \kappa_G \bar{s}_{\bar{j}}^G) &= 0 \\
(u_{\bar{j}}^\ell + \sum_{G=\bar{j}+1}^{\bar{q}} \mu_G^\ell \kappa_G \bar{u}_{\bar{j}}^G) &= 0 \\
(v_{i,\bar{j}}^\ell + \sum_{G=\bar{j}+1}^{\bar{q}} \mu_G^\ell \kappa_G \bar{v}_{i,\bar{j}}^G) &= 0 \quad \text{for all } i \in \mathbf{S}(\bar{j})
\end{aligned}$$

(This case happens when $\bar{h}_{\bar{j}}$, $\hat{\bar{h}}_{\bar{j}}$ and $\{q_{i,\bar{j}}\}_{i \in \mathbf{S}(\bar{j})}$ bases do not meaningfully participate in the representation of y_ℓ^* .)

3. (Case 2) Otherwise do the following steps:

- Verify that $|\mathbf{S}(\bar{j})| = t + 1 - |\text{corrupt}|$, otherwise output (III.BADINTERP) and abort. (*This abort indicates that $\mathcal{A}$ did not make enough SEval queries involving $p_{\bar{j}}$ to perform proper interpolation.*)
- Set $\mu_{\bar{j}}^\ell$ to a value that satisfies all the equations below:

$$\begin{aligned} (s_{\bar{j}}^\ell + \sum_{G=\bar{j}+1}^{\bar{q}} \mu_G^\ell \kappa_G \bar{s}_{\bar{j}}^G) &= \mu_{\bar{j}}^\ell \cdot \sum_{i \in \text{corrupt}} \lambda_i^{\bar{j}} z'_i \\ (u_{\bar{j}}^\ell + \sum_{G=\bar{j}+1}^{\bar{q}} \mu_G^\ell \kappa_G \bar{u}_{\bar{j}}^G) &= \mu_{\bar{j}}^\ell \cdot \sum_{i \in \text{corrupt}} \lambda_i^{\bar{j}} \hat{z}'_i \\ (v_{i,\bar{j}}^\ell + \sum_{G=\bar{j}+1}^{\bar{q}} \mu_G^\ell \kappa_G \bar{v}_{i,\bar{j}}^G) &= \mu_{\bar{j}}^\ell \cdot \lambda_i^{\bar{j}} \quad \text{for all } i \in \mathbf{S}(\bar{j}) \end{aligned}$$

If there is no such value $\mu_{\bar{j}}^\ell$ then output (III.BADINTERP) and abort. (*This abort indicates that $\mathcal{A}$, though having queried enough $q_{i,\bar{j}}$ values, did not perform proper interpolation with them, e.g. the blinding factors $\bar{h}_{\bar{j}}, \hat{\bar{h}}_{\bar{j}}$ do not cancel out in the algebraic expression for y_ℓ^* .)*

- Now it is verified that the exponents of the $p_{\bar{j}}$, $\bar{h}_{\bar{j}}$, and $\hat{\bar{h}}_{\bar{j}}$ factors in equation (3) are just some constant multiple $\mu_{\bar{j}}^\ell$ of interpolation coefficients. Therefore, these factors simplify to:

$$\begin{aligned} (p_{\bar{j}})^{\sum_{i \in \mathbf{S}(\bar{j})} \mu_{\bar{j}}^\ell \lambda_i^{\bar{j}} k_i} \cdot (\bar{h}_{\bar{j}})^{\mu_{\bar{j}}^\ell \sum_{i \in \text{corrupt}} \lambda_i^{\bar{j}} z'_i + \sum_{i \in \mathbf{S}(\bar{j})} \mu_{\bar{j}}^\ell \lambda_i^{\bar{j}} z'_i} \cdot (\hat{\bar{h}}_{\bar{j}})^{\mu_{\bar{j}}^\ell \sum_{i \in \text{corrupt}} \lambda_i^{\bar{j}} \hat{z}'_i + \sum_{i \in \mathbf{S}(\bar{j})} \mu_{\bar{j}}^\ell \lambda_i^{\bar{j}} \hat{z}'_i} \\ = \left((p_{\bar{j}})^{k - \sum_{i \in \text{corrupt}} \lambda_i^{\bar{j}} k_i} \cdot (\bar{h}_{\bar{j}})^{z'} \cdot (\hat{\bar{h}}_{\bar{j}})^{\hat{z}'} \right)^{\mu_{\bar{j}}^\ell} \end{aligned} \quad (4)$$

Due to the rigged programming of H_1 and H_2 in step I, we know that $\bar{h}_{\bar{j}} = (p_{\bar{j}})^{(\delta_{\bar{j}}) \cdot \alpha / z}$ and $\hat{\bar{h}}_{\bar{j}} = (p_{\bar{j}})^{(1-\delta_{\bar{j}}) \cdot \alpha / \hat{z}}$ where $z = z'$ and $\hat{z} = \hat{z}'$. Therefore, $(\bar{h}_{\bar{j}})^{z'} \cdot (\hat{\bar{h}}_{\bar{j}})^{\hat{z}'} = (p_{\bar{j}})^\alpha$ and we can simplify expression (4) further:

$$\left((p_{\bar{j}})^{k - \sum_{i \in \text{corrupt}} \lambda_i^{\bar{j}} k_i} \cdot (\bar{h}_{\bar{j}})^{z'} \cdot (\hat{\bar{h}}_{\bar{j}})^{\hat{z}'} \right)^{\mu_{\bar{j}}^\ell} = \left((p_{\bar{j}})^{\kappa_{\bar{j}}} \right)^{\mu_{\bar{j}}^\ell} \quad (5)$$

4. In either case, we have now rewritten the $p_{\bar{j}}$, $\bar{h}_{\bar{j}}$, and $\hat{\bar{h}}_{\bar{j}}$ factors of equation (3) as $(p_{\bar{j}})^{\mu_{\bar{j}}^\ell \kappa_{\bar{j}}}$ (in Case 1, this is simply because $\mu_{\bar{j}}^\ell = 0$). Expand $p_{\bar{j}}$ using its provided representation, equation (2), and then substitute these factors back into equation (3). The result is precisely the representation of y_ℓ^* needed for the next loop iteration.
5. Decrement $\bar{j}$ and iterate.

At the end of this loop, if no abort occurs, we extract values $\{\mu_J^\ell\}_{J \in [\bar{q}]^+}$ and obtain a greatly simplified representation of y_ℓ^* , for all $\ell \in [L]$:

$$y_\ell^* = \prod_{j \in [q]^+} (h_j)^{r_j^\ell + \sum_{J \in [\bar{q}]^+} \mu_J^\ell \kappa_J \bar{r}_j^J}$$

$\mathcal{B}$ step IV: verify that $\mathcal{A}$ does not know the secret key k'

Rather than compute the μ_J^ℓ values numerically, we can represent each one as a polynomial function of α while treating α as a variable of unknown value. Such a representation can be constructed during the loop in step III. There, μ_J^ℓ is defined as the value satisfying three equations that depend upon the values $\{\kappa_G\}$ and $\{\mu_G^\ell\}$ for $G > J$. The $\{\kappa_G\}$ values are linear functions of α (in the simplest case where no servers are corrupted, all $\kappa_G = k + \alpha$). The $\{\mu_G^\ell\}$ values have their own representations as polynomial functions of α , found by previous loop iterations. Each iteration can create a polynomial of degree at most 1 greater than those already found, so these polynomials are all of degree at most $\bar{q}$.

Define coefficients $\{\beta_{J,e}^\ell\}$ found via the procedure described above:

$$\forall \ell, J \in [L] \times [\bar{q}]^+ \quad \mu_J^\ell = \sum_{e=0}^{\bar{q}} \beta_{J,e}^\ell \alpha^e \quad (6)$$

Next, verify that:

$$\forall \ell, j \in [L] \times [q] \quad \gamma_j^\ell := \sum_{J \in [\bar{q}]^+} \beta_{J,0}^\ell \bar{r}_j^J = \begin{cases} 1 & j = j(\ell) \\ 0 & j \neq j(\ell) \end{cases} \quad (7)$$

If not, output (IV.KEYKNOWN) and abort. This abort indicates that $\mathcal{A}$ has found an unexpected representation for the secret key k' , which is using either the scalar r_j^ℓ (implying direct knowledge of k') or the higher-degree coefficients $\{\beta_{J,e}^\ell\}_{e \geq 1}$ (implying meaningful participation of recursive queries).

Note that the $J = 0$ terms can be omitted from the summation in equation (7) because $\bar{r}_j^0 = 0$ for all $j \neq 0$.

B step V: expose contradiction

Express the matrix of all $\{\gamma_j^\ell\}$ values in terms of the $\{\beta_{J,0}^\ell\}$ and $\{\bar{r}_j^J\}$ values:

$$\begin{bmatrix} \gamma_1^1 & \gamma_2^1 & \dots & \gamma_q^1 \\ \gamma_1^2 & \gamma_2^2 & & \gamma_q^2 \\ \vdots & & \ddots & \vdots \\ \gamma_1^L & \gamma_2^L & \dots & \gamma_q^L \end{bmatrix} = \begin{bmatrix} \beta_{1,0}^1 & \beta_{2,0}^1 & \dots & \beta_{\bar{q},0}^1 \\ \beta_{1,0}^2 & \beta_{2,0}^2 & & \beta_{\bar{q},0}^2 \\ \vdots & & \ddots & \vdots \\ \beta_{1,0}^L & \beta_{2,0}^L & \dots & \beta_{\bar{q},0}^L \end{bmatrix} \begin{bmatrix} \bar{r}_1^1 & \bar{r}_2^1 & \dots & \bar{r}_q^1 \\ \bar{r}_1^2 & \bar{r}_2^2 & \dots & \bar{r}_q^2 \\ \vdots & \ddots & \ddots & \vdots \\ \bar{r}_1^{\bar{q}} & \bar{r}_2^{\bar{q}} & \dots & \bar{r}_q^{\bar{q}} \end{bmatrix}$$

$$\Gamma = BR$$

Matrix Γ is $L \times q$. As verified in step IV, each row of Γ consists of all 0s except for a single 1; each column contains at most a single 1. Therefore, $\text{rank}(\Gamma) = L$.

Matrix B is $L \times \bar{q}$, and matrix R is $\bar{q} \times q$. Define $\bar{q}^* \leq \bar{q}$ to be the number of columns in B that are not entirely 0. $\text{rank}(B) \leq \bar{q}^*$, and therefore $\text{rank}(BR) \leq \bar{q}^*$. Since $\Gamma = BR$, it follows that $L \leq \bar{q}^*$.

The existence of a nonzero $\beta_{J,0}^\ell$ implies that (Case 2) occurred in the $\bar{j} = J$ iteration of step III. As verified there, then, $|\mathbf{S}(J)| = t + 1 - |\text{corrupt}|$, which implies that $\mathcal{A}$ must have queried SEval with (ctx_J, p_J) for at least $t + 1 - |\text{corrupt}|$ uncorrupted servers, i.e. $|\text{evalset}[\text{ctx}_J, p_J] \cup \text{corrupt}| \geq t + 1$. Therefore, as verified in step II, $L > \bar{q}^*$. This is a contradiction!

Reductions. It remains to show that the probability of (II.ADVLOSES) is equal (up to a negligible difference) to the probability that $\mathcal{A}$ loses the real one-more unpredictability game and that events (III.BADINTERP) and (IV.KEYKNOWN) both occur with negligible probability. We approach these events in the reverse of the order they appear in $\mathcal{B}$.

All our reductions have some steps in common, since they are all based on the same “reduction blueprint” $\mathcal{B}$. First, they all run $\mathcal{A}$, whose runtime we denote $T_{\mathcal{A}}$. Second, they all run the loop in $\mathcal{B}$ step III; since $L \leq q$, this loop is $\mathcal{O}(q\bar{q})$. The $\bar{q}$ -co-SDL reduction in the proof of Lemma 1 additionally maintains polynomial representations of the μ_j^ℓ values as explained in $\mathcal{B}$ step IV; this increases the runtime of the step III loop to $\mathcal{O}(q\bar{q}^2)$. That reduction also runs a $\mathcal{O}(\bar{q}^{1.815} \log m)$ runtime polynomial root-finding algorithm as a subroutine. Finally, two of the reductions employ as a substep the reduction from DL to N -RDL (Lemma 6), with $N \in \mathcal{O}(q + \bar{q})$. Therefore, all reductions run in time $\mathcal{O}(T_{\mathcal{A}} + q\bar{q}^2 + q \log m + \bar{q}^{1.815} \log m)$. To avoid redundancy, we omit repeated mention of runtimes from our reductions’ explanations until we combine our results in Theorem 1.

Lemma 1. *If $\bar{q}$ -co-SDL is $(\epsilon_{SDL}, T_{SDL})$ -hard in $(\mathbb{G}_1, \mathbb{G}_2)$ and $\mathcal{A}$ makes at most q queries to H_0 and at most $\bar{q}$ queries to H_1 and H_2 , then $\Pr[\mathcal{B}(\mathcal{A}) \rightarrow (\text{IV.KEYKNOWN})] \leq 4 \cdot \epsilon_{SDL}$.*

Proof. At the time of a (IV.KEYKNOWN) event, we have found some $\ell, j \in [L] \times [q]$ that violates equation (7). Define F as the event that the following equation holds:

$$r_j^\ell + \sum_{j \in [q]^+} \mu_{j\kappa_j}^\ell \bar{r}_j^j = \begin{cases} k + \alpha & j = j(\ell) \\ 0 & j \neq j(\ell) \end{cases} \quad (8)$$

Intuitively, event F states that no H_0 responses other than $H_0(x_\ell^*)$ are meaningfully used in $\mathcal{A}$ ’s algebraic representation of $y_\ell^* = H_0(x_\ell^*)^{k+\alpha}$.

Given a $\bar{q}$ -co-SDL input challenge $= (g_1, (g_1)^\alpha, (g_1)^{\alpha^2}, \dots, (g_1)^{\alpha^{\bar{q}}}, g_2, (g_2)^\alpha)$, we use $\mathcal{A}$ to solve for α . We follow one of two reduction strategies at random. Strategy #1 succeeds if event F occurs, and strategy #2 succeeds if it doesn’t.

Strategy #1

Run $\mathcal{B}(\mathcal{A})$, but using the group elements of **challenge** to compute everything involving α . To confirm that this is possible, observe that α is never directly needed by $\mathcal{B}$:

- In step I, the τ_k trapdoor embedded in g_k allows us to compute the public parameters g_z and $g_{\hat{z}}$ and the public keys $\text{pk}'_i = (g_k)^{k_i+\alpha} \cdot (g_z)^{z'_i} \cdot (g_{\hat{z}})^{\hat{z}'_i}$ and $\text{pk}' = (g_k)^{k+\alpha}$. (For 2B-tBlindBLS, $\text{pk}' = (g_2)^{k+\alpha}$ can be computed easily.)
- In step I, every p_j^- input to H_1 and H_2 is accompanied by its algebraic representation. In these representations, only bases involving α have discrete logarithms that are unknown to us. Therefore, the elements of **challenge** are sufficient to compute the correct responses. (In the extreme, $\mathcal{A}$ might

recursively call the random oracles on their own outputs $\bar{q}$ times, in which case all elements of **challenge** are needed.)

- In step II, the $\tau_{j(\ell)}$ trapdoor embedded in $h_{j(\ell)}$ allows us to compute the correct function output $(h_{j(\ell)})^{k+\alpha}$ for each x_ℓ^* .
- In step III, we cannot compute the $\{\mu_j^\ell\}$ values numerically. However, it is sufficient to represent them as polynomial functions of α , as explained at the start of step IV. The condition for (Case 1) can be checked in the exponent using the elements of **challenge**. The check for (III.BADINTERP) can be skipped, since this reduction only succeeds in the case of (IV.KEYKNOWN), which requires that the earlier (III.BADINTERP) does not occur. Instead, we can simply assume in (Case 2) that a satisfactory μ_j^ℓ exists and use any one of the three equations to find its representation as a polynomial function of α , see equation (6).

After applying the definition of κ_J and the aforementioned polynomial representation of μ_j^ℓ , equation (8) is a polynomial function of α , with degree at most $\bar{q} + 1$. The degree 1 coefficient of this polynomial is:

$$\left(\sum_{J \in [\bar{q}]^+} \beta_{J,0}^\ell \bar{r}_J^J \right) - \begin{cases} 1 & j = j(\ell) \\ 0 & j \neq j(\ell) \end{cases}$$

The violation of equation (7) guarantees that this coefficient is nonzero, and therefore that the polynomial as a whole is nonzero. The polynomial has at most $\bar{q} + 1$ roots, one of which must be α . We can use a polynomial root-finding algorithm, and then test all possibilities until finding the correct α and thereby solving $\bar{q}$ -co-SDL.

Strategy #2

There is an obvious reduction from $\bar{q}$ -co-SDL to DL, and (per (Lemma 6)) there is a reduction from DL to RDL. Therefore, we present strategy #2 as an RDL reduction on input $((g_1)^{x_k}, (g_1)^{x_1}, \dots, (g_1)^{x_q})$.

Run $\mathcal{B}(\mathcal{A})$, but use $g_k := (g_1)^{x_k}$ and for all $j \in [q]$ use $h_j := (g_1)^{x_j}$ to respond to random oracle query $H_0(x_j)$. To confirm that this is possible, observe that the $\tau_k, \{\tau_j\}_{j \in [q]}$ trapdoors are never used by $\mathcal{B}$.

Since (II.ADVLOSES) did not occur, y_ℓ^* must correctly equal $(h_{j(\ell)})^{k+\alpha}$:

$$(h_{j(\ell)})^{k+\alpha} = \prod_{j \in [q]^+} (h_j)^{r_j^\ell + \sum_{J \in [\bar{q}]^+} \mu_J^\ell \kappa_J \bar{r}_J^J}$$

If F does not occur, the exponent of h_j is nonzero, and we have an RDL solution.

The probability of correctly guessing whether or not F will occur is $1/2$. The asymptotically fastest algorithms for polynomial root-finding over a finite field are probabilistic and can find the roots in time $\mathcal{O}(\bar{q}^{1.815} \log m)$ with failure probability $\leq 1/2$ [64, 87]. Thus, our reduction wins $\bar{q}$ -co-SDL with probability $\geq 1/2 \cdot 1/2 \cdot \Pr[\mathcal{B}(\mathcal{A}) \rightarrow (\text{IV.KEYKNOWN})]$. Our hardness assumption therefore implies that $\Pr[\mathcal{B}(\mathcal{A}) \rightarrow (\text{IV.KEYKNOWN})] \leq 4 \cdot \epsilon_{SDL}$. $\square$

To help us proceed, we define a hybrid $\mathcal{B}'$ of the wrapper algorithm. $\mathcal{B}'$ only runs steps I through III and does *not* perform any key rigging in step I. In other words, the $\{z'_i\}$ and $\{\hat{z}'_i\}$ keys really are sharings of zero in $\mathcal{B}'$. In particular, $\mathcal{B}'$ sets all $z'_i := z_i$ and $\hat{z}'_i := \hat{z}_i$; consequently, $z' := 0$ and $\hat{z}' := 0$. Also, $k' := k$, $\text{pk}' := (g_k)^k$ (for 2B-HashTDH) or $\text{pk}' := (g_2)^k$ (for 2B-tBlindBLS), and all $\text{pk}'_i := (g_k)^{k_i} \cdot (g_z)^{z_i} \cdot (g_{\hat{z}})^{\hat{z}_i}$.

Lemma 2. *Define event (EARLYABORT) to be the union of (II.ADVLOSES) and (III.BADINTERP). This event is equally probable for $\mathcal{B}(\mathcal{A})$ and $\mathcal{B}'(\mathcal{A})$, i.e. $\Pr[\mathcal{B}(\mathcal{A}) \rightarrow (\text{EARLYABORT})] = \Pr[\mathcal{B}'(\mathcal{A}) \rightarrow (\text{EARLYABORT})]$.*

Proof. If $\mathcal{A}$ corrupts $t + 1$ or more servers, then (EARLYABORT) occurs with probability 1 in both $\mathcal{B}$ and $\mathcal{B}'$. Therefore, it suffices to show that the view of $\mathcal{A}$ is identical in $\mathcal{B}$ and $\mathcal{B}'$ as long as $\mathcal{A}$ does not corrupt $t + 1$ servers.

In both $\mathcal{B}$ and $\mathcal{B}'$, $k(i)$ is a uniformly random t -degree polynomial. In $\mathcal{B}$, α is a uniformly random field element. For any $k(i)$ and α that may be chosen in $\mathcal{B}$, an identical view can be produced in $\mathcal{B}'$ if polynomial $k'(i) = k(i) + \alpha$ is chosen:

- In $\mathcal{B}$ we set $\text{pk} := (g_k)^{k(0)+\alpha}$ while in $\mathcal{B}'$ we set $\text{pk} := (g_k)^{k'(0)}$. (For 2B-tBlindBLS we replace g_k with g_2 .) Clearly these are equal.
- In $\mathcal{B}$, $\text{pk}_i := (g_k)^{k(i)+\alpha} \cdot (g_z)^{z_i} \cdot (g_{\hat{z}})^{\hat{z}_i}$. In $\mathcal{B}'$, $\text{pk}_i := (g_k)^{k'(i)} \cdot (g_z)^{z_i} \cdot (g_{\hat{z}})^{\hat{z}_i}$. Clearly these are equal.
- In $\mathcal{B}$, SEval response $q_{i,\bar{j}} = (p_{\bar{j}})^{k(i)} \cdot H_1(\text{ctx}_{\bar{j}}, p_{\bar{j}})^{z_i+z} \cdot H_2(\text{ctx}_{\bar{j}}, p_{\bar{j}})^{\hat{z}_i+\hat{z}}$. In $\mathcal{B}'$, $q_{i,\bar{j}} = (p_{\bar{j}})^{k'(i)} \cdot H_1(\text{ctx}_{\bar{j}}, p_{\bar{j}})^{z_i} \cdot H_2(\text{ctx}_{\bar{j}}, p_{\bar{j}})^{\hat{z}_i}$. These are equal due to the rigged programming of H_1 and H_2 :

$$\begin{aligned}
& (p_{\bar{j}})^{k(i)} \cdot H_1(\text{ctx}_{\bar{j}}, p_{\bar{j}})^{z_i+z} \cdot H_2(\text{ctx}_{\bar{j}}, p_{\bar{j}})^{\hat{z}_i+\hat{z}} \\
&= (p_{\bar{j}})^{k(i)} \cdot ((p_{\bar{j}})^{(\delta_{\bar{j}}) \cdot \alpha / z})^{z_i+z} \cdot ((p_{\bar{j}})^{(1-\delta_{\bar{j}}) \cdot \alpha / \hat{z}})^{\hat{z}_i+\hat{z}} \\
&= (p_{\bar{j}})^{k(i)} \cdot (p_{\bar{j}})^{(\delta_{\bar{j}}) \cdot \alpha} \cdot (p_{\bar{j}})^{(1-\delta_{\bar{j}}) \cdot \alpha} \cdot ((p_{\bar{j}})^{(\delta_{\bar{j}}) \cdot \alpha / z})^{z_i} \cdot ((p_{\bar{j}})^{(1-\delta_{\bar{j}}) \cdot \alpha / \hat{z}})^{\hat{z}_i} \\
&= (p_{\bar{j}})^{k(i)+\alpha} \cdot ((p_{\bar{j}})^{(\delta_{\bar{j}}) \cdot \alpha / z})^{z_i} \cdot ((p_{\bar{j}})^{(1-\delta_{\bar{j}}) \cdot \alpha / \hat{z}})^{\hat{z}_i} \\
&= (p_{\bar{j}})^{k(i)+\alpha} \cdot H_1(\text{ctx}_{\bar{j}}, p_{\bar{j}})^{z_i} \cdot H_2(\text{ctx}_{\bar{j}}, p_{\bar{j}})^{\hat{z}_i}
\end{aligned}$$

- In $\mathcal{B}$, Corrupt response $\text{sk}_i = (k(i), z_i + z, \hat{z}_i + \hat{z})$. In $\mathcal{B}'$, $\text{sk}_i = (k'(i), z_i, \hat{z}_i)$. Since at most t servers are corrupted, the perfect security of Shamir secret sharing ensures that these key shares are uniformly distributed. In particular, $\mathcal{A}$ does not see that z'_i and $\hat{z}'_i$ are not really sharings of zero in $\mathcal{B}$. It remains only to show that sk_i agrees with the previously revealed public key pk_i and SEval responses $\{q_{i,\bar{j}}\}_{\bar{j} \in [\bar{q}]}$. The latter is immediate, because the revealed secret keys are the same as those used in SEval. The former holds due to the rigged choice of g_z and $g_{\hat{z}}$. We omit step-by-step analysis of this fact because it is essentially the same as the analysis given for $q_{i,\bar{j}}$ above.
- The only remaining element of $\mathcal{A}$'s view is the NIZKs sent during KeyGen and SEval. In both $\mathcal{B}$ and $\mathcal{B}'$, all NIZKs are simulated, and the only inputs to the simulators are the values that are shown to be equal above. Therefore, the NIZKs are also equal across $\mathcal{B}$ and $\mathcal{B}'$.

- Also, steps II and III of $\mathcal{B}$ and $\mathcal{B}'$ only use these same values that are shown to be equal above. $\square$

We move now to a second hybrid $\mathcal{B}''$ that additionally does not rig the choice of g_z and $g_{\hat{z}}$ and the programming of the random oracles in step I. In other words, $g_z, g_{\hat{z}}$, and the outputs of H_1 and H_2 really are all random (and uncorrelated) $\mathbb{G}_1$ elements in $\mathcal{B}''$. In particular, $g_z := (g_1)^{\tau_z}$ and $g_{\hat{z}} := (g_1)^{\tau_{\hat{z}}}$ where $\tau_z, \tau_{\hat{z}} \leftarrow_{\$} \mathbb{Z}_m$; for all $\bar{j} \in [\bar{q}]$, $H_1(\text{ctx}_{\bar{j}}, p_{\bar{j}}) := (g_1)^{\bar{\tau}_{\bar{j}}}$ and $H_2(\text{ctx}_{\bar{j}}, p_{\bar{j}}) := (g_1)^{\hat{\tau}_{\bar{j}}}$ where $\bar{\tau}_{\bar{j}}, \hat{\tau}_{\bar{j}} \leftarrow_{\$} \mathbb{Z}_m$. In $\mathcal{B}''$, the view of $\mathcal{A}$ is identical to the real world one-more unpredictability game except for the use of simulated NIZKs.

Lemma 3. *If DDH is $(\epsilon_{DDH}, T_{DDH})$ -hard in $\mathbb{G}_1$ then $\Pr[\mathcal{B}'(\mathcal{A}) \rightarrow (\text{EARLYABORT})] \leq \Pr[\mathcal{B}''(\mathcal{A}) \rightarrow (\text{EARLYABORT})] + \epsilon_{DDH}$.*

Proof. Given a DDH input challenge $= (d_1, d'_1, d_2, d'_2)$, we can use $\mathcal{A}$ to distinguish whether or not it is a Diffie-Hellman tuple. Run $\mathcal{B}'(\mathcal{A})$, but using the group elements of challenge to program H_1 and H_2 . In particular:

$$\begin{aligned} H_1(\text{ctx}_{\bar{j}}, p_{\bar{j}}) &= \bar{h}_{\bar{j}} := \left((d_1)^{\delta_{1,\bar{j}}} \cdot (d_2)^{\delta_{2,\bar{j}}} \right)^{\alpha} \\ H_2(\text{ctx}_{\bar{j}}, p_{\bar{j}}) &= \hat{h}_{\bar{j}} := \left(\frac{p_{\bar{j}}}{(d'_1)^{\delta_{1,\bar{j}}} \cdot (d'_2)^{\delta_{2,\bar{j}}}} \right)^{\alpha/\hat{z}} \end{aligned}$$

where $\delta_{1,\bar{j}}, \delta_{2,\bar{j}} \leftarrow_{\$} \mathbb{Z}_m$ for all $\bar{j} \in [\bar{q}]$. Set g_z and $g_{\hat{z}}$ in the same way for random $\delta_{1,z}, \delta_{2,z} \leftarrow_{\$} \mathbb{Z}_m$. Notice that we do not explicitly choose z . If challenge is a Diffie-Hellman tuple, then z is effectively the secret exponent hidden in that tuple (called b in Definition 18). We are able to generate many pairs of correlated group elements from challenge by using the well-known “DDH randomized self-reduction” technique [82, 76].

In the case that challenge is a Diffie-Hellman tuple, $(d_1)^{\delta_{1,\bar{j}}} \cdot (d_2)^{\delta_{2,\bar{j}}}$ is a uniformly random group element and $(d'_1)^{\delta_{1,\bar{j}}} \cdot (d'_2)^{\delta_{2,\bar{j}}} = ((d_1)^{\delta_{1,\bar{j}}} \cdot (d_2)^{\delta_{2,\bar{j}}})^z$. Thus, the execution is identical to $\mathcal{B}'(\mathcal{A})$.

In the case that challenge is a random tuple, $\bar{h}_{\bar{j}}$ and $\hat{h}_{\bar{j}}$ are independently and uniformly random group elements. Thus, the execution is identical to $\mathcal{B}''(\mathcal{A})$. Note that step III of $\mathcal{B}$ makes use of the correlated programming of H_1 and H_2 in equation (5). However, that equation still holds in $\mathcal{B}''$ for a different reason: $z' = \hat{z}' = 0$ and $k = k'$.

If event (EARLYABORT) occurs, then our distinguisher outputs 1. Otherwise, our distinguisher outputs 0. This distinguisher’s advantage is exactly equal to the difference between $\Pr[\mathcal{B}'(\mathcal{A}) \rightarrow (\text{EARLYABORT})]$ and $\Pr[\mathcal{B}''(\mathcal{A}) \rightarrow (\text{EARLYABORT})]$. Our hardness assumption therefore implies that $\Pr[\mathcal{B}'(\mathcal{A}) \rightarrow (\text{EARLYABORT})] \leq \Pr[\mathcal{B}''(\mathcal{A}) \rightarrow (\text{EARLYABORT})] + \epsilon_{DDH}$. $\square$

We now consider the remaining abort conditions in $\mathcal{B}''$.

Lemma 4. *If t -OMDL is $(\epsilon_{OMDL}, T_{OMDL})$ -hard in $\mathbb{G}_1$ and $\mathcal{A}$ makes at most q queries to H_0 and $\bar{q}$ queries to H_1 and H_2 , then $\Pr[\mathcal{B}''(\mathcal{A}) \rightarrow (\text{III.BADINTERP})] \leq 2 \cdot \epsilon_{OMDL}$.*

Proof. At the time of a (III.BADINTERP) abort, we have found some $\ell, \bar{j} \in [L] \times [\bar{q}]$ such that, in the representation of y_ℓ^* , the exponent of either $\bar{h}_{\bar{j}}$ or $\hat{\bar{h}}_{\bar{j}}$ is not a properly-formed interpolation of some $t+1$ $\{z_i\}$ or $\{\hat{z}_i\}$ keys, respectively. Without loss of generality, assume that it's the $\bar{h}_{\bar{j}}$ component that triggered the abort (the $\hat{\bar{h}}_{\bar{j}}$ case is entirely symmetric). Define F as the event that the following equation holds:

$$(s_{\bar{j}}^\ell + \sum_{G=\bar{j}+1}^{\bar{q}} \mu_G^\ell \kappa_G \bar{s}_{\bar{j}}^G) + \sum_{i \in \mathbf{S}(\bar{j})} (v_{i,\bar{j}}^\ell + \sum_{G=\bar{j}+1}^{\bar{q}} \mu_G^\ell \kappa_G \bar{v}_{i,\bar{j}}^G) \cdot z_i = 0 \quad (9)$$

Intuitively, event F states that no H_1 responses are meaningfully used in $\mathcal{A}$'s algebraic representation of $y_\ell^* = H_0(x_\ell^*)^{k+\alpha}$.

Given a t -OMDL input **challenge** $= (g_1, (g_1)^{\xi_1}, \dots, (g_1)^{\xi_t})$, we use $\mathcal{A}$ to solve for all $\{\xi_\iota\}$. We follow one of two reduction strategies at random. Strategy #1 succeeds if event F occurs, and strategy #2 succeeds if it doesn't.

Strategy #1

Run $\mathcal{B}''(\mathcal{A})$, but using the group elements of **challenge** to compute everything involving the secret keys $\{z_i\}$ and $\{\hat{z}_i\}$. In particular:

- For each $\iota \in [t]$, pick $\nu_\iota \leftarrow_{\S} \mathbb{Z}_m$ and define $\hat{\xi}_\iota := \xi_\iota + \nu_\iota$. The $\{\hat{\xi}_\iota\}$ values cannot be directly computed, but $\{(g_1)^{\hat{\xi}_\iota}\}$ can.
- In step I, define polynomials $z(i) := \sum_{\iota=1}^t \xi_\iota i^\iota$ and $\hat{z}(i) := \sum_{\iota=1}^t \hat{\xi}_\iota i^\iota$. Once again, these polynomials are only computable in the exponent of g_1 ; this is sufficient to compute the public keys.
- In response to an **SEval** query to server i with input $(\text{ctx}_{\bar{j}}, p_{\bar{j}})$, we can use the H_1 and H_2 trapdoors to compute the correct response in spite of not directly knowing z_i and $\hat{z}_i$: $(p_{\bar{j}})^{k_i} \cdot (\bar{h}_{\bar{j}})^{z_i} \cdot (\hat{\bar{h}}_{\bar{j}})^{\hat{z}_i} = (p_{\bar{j}})^{k_i} \cdot ((g_1)^{z_i})^{\bar{\tau}_{\bar{j}}} \cdot ((g_1)^{\hat{z}_i})^{\hat{\tau}_{\bar{j}}}$.
- When $\mathcal{A}$ wishes to **Corrupt** server i , use a single discrete logarithm oracle call to find the correct z_i and $\hat{z}_i$ to return. In particular, $z_i = \text{DL}_{g_1}((g_1)^{z_i})$ and $\hat{z}_i = z_i - \sum_{\iota=1}^t \nu_\iota i^\iota$.
- In step III, we treat z_i and $\hat{z}_i$ (for uncorrupted i) as unknown variables.

The first condition that could trigger (III.BADINTERP) is $|\mathbf{S}(\bar{j})| < t+1 - |\text{corrupt}|$. Equation (9) has only $|\mathbf{S}(\bar{j})|$ unknowns $\{z_i\}_{i \in \mathbf{S}(\bar{j})}$. Use $\text{DL}_{g_1}(\cdot) \mid \mathbf{S}(\bar{j})| - 1$ times to learn all of them but one, then use equation (9) to solve for the last. Use $\text{DL}_{g_1}(\cdot)$ $t - |\text{corrupt}| - |\mathbf{S}(\bar{j})|$ more times until a total of t z_i values are known. The underlying challenge solutions $\{\xi_\iota\}_{\iota \in [t]}$ can then be interpolated. Only $t-1$ $\text{DL}_{g_1}(\cdot)$ oracle queries have been made.

The second condition that could trigger (III.BADINTERP) is that there exists no $\mu_{\bar{j}}^\ell$ satisfying $(s_{\bar{j}}^\ell + \sum_{G=\bar{j}+1}^{\bar{q}} \mu_G^\ell \kappa_G \bar{s}_{\bar{j}}^G) = \mu_{\bar{j}}^\ell (\sum_{i \in \text{corrupt}} \lambda_i^{\bar{j}} z_i)$ and $(v_{i,\bar{j}}^\ell + \sum_{G=\bar{j}+1}^{\bar{q}} \mu_G^\ell \kappa_G \bar{v}_{i,\bar{j}}^G) = \mu_{\bar{j}}^\ell \lambda_i^{\bar{j}}$ for all $i \in \mathbf{S}(\bar{j})$. However, since

$|\mathbf{S}(\bar{j})| = t + 1 - |\text{corrupt}|$, we know that $\{z_i\}_{i \in \text{corrupt} \cup \mathbf{S}(\bar{j})}$ do in fact interpolate to 0:

$$\sum_{i \in \text{corrupt}} \lambda_i^{\bar{j}} z_i + \sum_{i \in \mathbf{S}(\bar{j})} \lambda_i^{\bar{j}} z_i = 0 \quad (10)$$

Equations (9) and (10) are independent linear equations with the same $t + 1 - |\text{corrupt}|$ unknowns $\{z_i\}_{i \in \mathbf{S}(\bar{j})}$. Use $\text{DL}_{g_1}(\cdot)$ $t - 1 - |\text{corrupt}|$ times to learn all of them but two, then use equations (9) and (10) to solve for the last two. A total of $t + 1$ z_i values are now known, so the underlying challenge solutions $\{\xi_\iota\}_{\iota \in [t]}$ can be interpolated. Only $t - 1$ $\text{DL}_{g_1}(\cdot)$ oracle queries have been made.

Strategy #2

There is an obvious reduction from t -OMDL to DL, and (per (Lemma 6)) there is a reduction from DL to RDL. Therefore, we present strategy #2 as an RDL reduction on input $((g_1)^{x_k}, (g_1)^{x_z}, (g_1)^{x_{\bar{z}}}, (g_1)^{x_1}, \dots, (g_1)^{x_{q+2\bar{q}}})$.

Run $\mathcal{B}''(\mathcal{A})$, but use $(g_k, g_z, g_{\bar{z}}) := ((g_1)^{x_k}, (g_1)^{x_z}, (g_1)^{x_{\bar{z}}})$. For all $j \in [q]$ use $h_j := (g_1)^{x_j}$ to respond to random oracle query $H_0(x_j)$. For all $\bar{j} \in [\bar{q}]$ use $\bar{h}_{\bar{j}} := (g_1)^{x_{q+\bar{j}}}$ and $\hat{\bar{h}}_{\bar{j}} := (g_1)^{x_{q+\bar{q}+\bar{j}}}$ to respond to random oracle queries $H_1(\text{ctx}_{\bar{j}}, p_{\bar{j}})$ and $H_2(\text{ctx}_{\bar{j}}, p_{\bar{j}})$, respectively. To confirm that this is possible, observe that the $\tau_k, \tau_z, \tau_{\bar{z}}, \{\tau_j\}_{j \in [q]}, \{\bar{\tau}_{\bar{j}}, \hat{\bar{\tau}}_{\bar{j}}\}_{\bar{j} \in [\bar{q}]}$ trapdoors are never used by $\mathcal{B}''$.

Since (II.ADVLOSES) did not occur, y_ℓ^* must correctly equal $(h_{j(\ell)})^k$. If F does not occur, then the exponent of $\bar{h}_{\bar{j}}$ is nonzero in equation (3). This is nearly an RDL solution, but we must eliminate the $\{p_J\}_{J \in [\bar{q}]^+}$ bases. Recursively expand each one using its representation, equation (2), until only $h_j, \bar{h}_G$, and $\hat{\bar{h}}_G$ bases remain. Since p_J can only include $\bar{h}_G$ and $\hat{\bar{h}}_G$ factors for $G < J$, this expansion does not alter the exponent of $\bar{h}_{\bar{j}}$, which therefore remains nonzero. Thus, we have an RDL solution.

The probability of correctly guessing whether or not F will occur is $1/2$, so this procedure wins t -OMDL with probability $1/2 \cdot \Pr[\mathcal{B}''(\mathcal{A}) \rightarrow (\text{III.BADINTERP})]$. Our hardness assumption therefore implies that $\Pr[\mathcal{B}''(\mathcal{A}) \rightarrow (\text{III.BADINTERP})] \leq 2 \cdot \epsilon_{\text{OMDL}}$. $\square$

Finally we are ready to combine our previous results and conclude the proof.

Theorem 1. *If $\bar{q}$ -co-SDL is $(\epsilon_{\text{SDL}}, T_{\text{SDL}})$ -hard in $(\mathbb{G}_1, \mathbb{G}_2)$, DDH is $(\epsilon_{\text{DDH}}, T_{\text{DDH}})$ -hard in $\mathbb{G}_1$, t -OMDL is $(\epsilon_{\text{OMDL}}, T_{\text{OMDL}})$ -hard in $\mathbb{G}_1$, $\mathcal{A}$ runs in time $\mathcal{O}(\min\{T_{\text{OMDL}}, T_{\text{DDH}}, T_{\text{SDL}}\} - q\bar{q}^2 - q \log m - \bar{q}^{1.815} \log m)$, $\mathcal{A}$ is algebraic with respect to $\mathbb{G}_1$, simulation of $\text{NIZK}_{\text{KeyGen}}$ and $\text{NIZK}_{\text{SEval}}$ incurs security loss ϵ_{NIZK} , $\mathcal{A}$ makes at most q queries to H_0 and $\bar{q}$ queries to H_1 and H_2 , and H_0, H_1 , and H_2 are modeled as random oracles, then $\text{Adv}_{\mathcal{A}, 2\text{B-HashTDH}}^{\text{om-unpredictable}} \leq 2 \cdot \epsilon_{\text{OMDL}} + \epsilon_{\text{DDH}} + 4 \cdot \epsilon_{\text{SDL}} + \epsilon_{\text{NIZK}}$. Thus, 2B-HashTDH is one-more unpredictable (Definition 7).*

Proof. Recall that the execution of $\mathcal{B}(\mathcal{A})$ ends with a contradiction, and it therefore aborts before that point with probability 1.

$$\begin{aligned}
1 &= \Pr[\mathcal{B}(\mathcal{A}) \rightarrow (\text{EARLYABORT})] + \Pr[\mathcal{B}(\mathcal{A}) \rightarrow (\text{IV.KEYKNOWN})] \\
&\leq \Pr[\mathcal{B}(\mathcal{A}) \rightarrow (\text{EARLYABORT})] + 4 \cdot \epsilon_{SDL} \\
&\leq \Pr[\mathcal{B}''(\mathcal{A}) \rightarrow (\text{EARLYABORT})] + \epsilon_{DDH} + 4 \cdot \epsilon_{SDL} \\
&\leq \Pr[\mathcal{B}''(\mathcal{A}) \rightarrow (\text{II.ADVLOSES})] + 2 \cdot \epsilon_{OMDL} + \epsilon_{DDH} + 4 \cdot \epsilon_{SDL}
\end{aligned}$$

$\mathcal{B}''$ differs from $\mathbf{Game}_{\mathcal{A}, 2B\text{-HashTDH}}^{\text{om-unpredictable}}$ only in the use of simulated NIZKs. Define one final hybrid $\mathcal{B}'''$ that uses real NIZKs (i.e. $\text{NIZK}_{\text{KeyGen.Prove}}$ and $\text{NIZK}_{\text{SEval.Prove}}$) instead. By the simulatability of the NIZKs (Appendix B), $|\Pr[\mathcal{B}''(\mathcal{A}) \rightarrow (\text{II.ADVLOSES})] - \Pr[\mathcal{B}'''(\mathcal{A}) \rightarrow (\text{II.ADVLOSES})]| \leq \epsilon_{NIZK}$.

$\mathcal{B}'''$ simulates $\mathbf{Game}_{\mathcal{A}, 2B\text{-HashTDH}}^{\text{om-unpredictable}}$ perfectly and outputs (II.ADVLOSES) iff $\mathcal{A}$ fails the game's win condition. Therefore, $\mathbf{Adv}_{\mathcal{A}, 2B\text{-HashTDH}}^{\text{om-unpredictable}} = 1 - \Pr[\mathcal{B}'''(\mathcal{A}) \rightarrow (\text{II.ADVLOSES})]$, and consequently:

$$\mathbf{Adv}_{\mathcal{A}, 2B\text{-HashTDH}}^{\text{om-unpredictable}} \leq 2 \cdot \epsilon_{OMDL} + \epsilon_{DDH} + 4 \cdot \epsilon_{SDL} + \epsilon_{NIZK} \quad \square$$

4.1 Threshold Blind Signature Security

Now that tOPRF security is proven, we provide a simple lemma saying that any publicly verifiable tOPRF is also a tBSig. Even though a tBSig doesn't need to be a tOPRF (specifically, it doesn't need to be deterministic and pseudorandom), our 2B-tBlindBLS scheme is a tOPRF, so this lemma enables us to quickly prove its security.

Lemma 5. *Suppose there exists a threshold oblivious pseudorandom function (Definition 1) tOPRF = (Setup, KeyGen, UEval, SEval, UAgg, Offline) and a PPT algorithm Verify such that for all integers $0 \leq t < n$, all $(\{\text{sk}_i\}_{i \in [n]}, \text{aux}) \leftarrow \text{KeyGen}(n, t)$, and all $(x, y) \in \mathcal{X} \times \mathcal{Y}$:*

$$\text{Verify}(\text{pk}, x, y) = 1 \iff y = \text{Offline}(\{\text{sk}_i\}_{i \in [n]}, \text{aux}, x) \quad (11)$$

where pk is part of aux .

It follows that tBSig = (Setup, KeyGen, UEval, SEval, UAgg, Verify) is a threshold blind signature scheme (where aux is parsed as (pk, aux) in the output of KeyGen and the input to UEval).

Proof. To prove Lemma 5, we must show that tBSig is verifiably correct (Definition 9), blind (Definition 10), and one-more unforgeable (Definition 11). These three properties are immediately implied by the deterministic correctness (Definition 2), blindness (Definition 5), and one-more unpredictability (Definition 7) of tOPRF, because equation (11) makes the respective definitions equivalent. $\square$

Per the definition of a bilinear pairing, 2B-tBlindBLS.Verify indeed satisfies equation (11). Therefore, by Lemma 5, 2B-tBlindBLS is a secure threshold blind signature scheme.

Note that our security proof for 2B-tBlindBLS depends upon the DDH assumption in $\mathbb{G}_1$. If the protocol is instantiated with a type 1 (i.e. symmetric) pairing setup, then the pairing function itself acts as a DDH tester. Therefore, the protocol should be instantiated only with a type 2 or 3 (i.e. asymmetric) pairing setup. We do not view this as a significant drawback; in practice, type 3 setups are more performant than types 1 or 2 anyways [47].

References

1. Michel Abdalla, Manuel Barbosa, Jonathan Katz, Julian Loss, and Jiayu Xu. Algebraic adversaries in the universal composability framework. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part III*, volume 13092 of *LNCS*, pages 311–341. Springer, Cham, December 2021.
2. Masayuki Abe. A secure three-move blind signature scheme for polynomially many signatures. In Birgit Pfitzmann, editor, *EUROCRYPT 2001*, volume 2045 of *LNCS*, pages 136–151. Springer, Berlin, Heidelberg, May 2001.
3. Masayuki Abe and Serge Fehr. Adaptively secure feldman VSS and applications to universally-composable threshold cryptography. In Matthew Franklin, editor, *CRYPTO 2004*, volume 3152 of *LNCS*, pages 317–334. Springer, Berlin, Heidelberg, August 2004.
4. Damiano Abram, Ariel Nof, Claudio Orlandi, Peter Scholl, and Omer Shlomovits. Low-bandwidth threshold ECDSA via pseudorandom correlation generators. In *2022 IEEE Symposium on Security and Privacy*, pages 2554–2572. IEEE Computer Society Press, May 2022.
5. Jesús F. Almansa, Ivan Damgård, and Jesper Buus Nielsen. Simplified threshold RSA with adaptive and proactive security. In Serge Vaudenay, editor, *EUROCRYPT 2006*, volume 4004 of *LNCS*, pages 593–611. Springer, Berlin, Heidelberg, May / June 2006.
6. Ghous Ali Amjad, Scott Hendrickson, Christopher A. Wood, and Kevin W. L. Yeo. Partially Blind RSA Signatures. Internet-Draft draft-irtf-cfrg-partially-blind-rsa-00, Internet Engineering Task Force, September 2024. Work in Progress.
7. iCloud Private Relay Overview, Dec 2021. https://www.apple.com/icloud/docs/iCloud_Private_Relay_Overview_Dec2021.pdf.
8. Renas Bacho, Sourav Das, Julian Loss, and Ling Ren. Adaptively secure three-round threshold schnorr signatures from DDH. In Yael Tauman Kalai and Seny F. Kamara, editors, *Advances in Cryptology – CRYPTO 2025*, pages 390–422, Cham, 2025. Springer Nature Switzerland.
9. Renas Bacho, Sourav Das, Julian Loss, and Ling Ren. Glacius: Threshold schnorr signatures from DDH with full adaptive security. In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology – EUROCRYPT 2025*, pages 304–334, Cham, 2025. Springer Nature Switzerland.
10. Renas Bacho and Julian Loss. On the adaptive security of the threshold bls signature scheme. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS ’22*, page 193–207, New York, NY, USA, 2022. Association for Computing Machinery.

11. Renas Bacho, Julian Loss, Stefano Tessaro, Benedikt Wagner, and Chenzhi Zhu. Twinkle: Threshold signatures from DDH with full adaptive security. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part I*, volume 14651 of *LNCS*, pages 429–459. Springer, Cham, May 2024.
12. Ali Bagherzandi, Jung Hee Cheon, and Stanislaw Jarecki. Multisignatures secure under the discrete logarithm assumption and a generalized forking lemma. In Peng Ning, Paul F. Syverson, and Somesh Jha, editors, *ACM CCS 2008*, pages 449–458. ACM Press, October 2008.
13. Ali Bagherzandi, Stanislaw Jarecki, Nitesh Saxena, and Yanbin Lu. Password-protected secret sharing. In Yan Chen, George Danezis, and Vitaly Shmatikov, editors, *ACM CCS 2011*, pages 433–444. ACM Press, October 2011.
14. Mihir Bellare, Elizabeth C. Crites, Chelsea Komlo, Mary Maller, Stefano Tessaro, and Chenzhi Zhu. Better than advertised security for non-interactive threshold signatures. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part IV*, volume 13510 of *LNCS*, pages 517–550. Springer, Cham, August 2022.
15. Mihir Bellare, Stefano Tessaro, and Chenzhi Zhu. Stronger security for non-interactive threshold signatures: BLS and FROST. Cryptology ePrint Archive, Paper 2022/833, 2022. <https://eprint.iacr.org/2022/833>.
16. Alexandra Boldyreva. Threshold signatures, multisignatures and blind signatures based on the gap-Diffie-Hellman-group signature scheme. In Yvo Desmedt, editor, *PKC 2003*, volume 2567 of *LNCS*, pages 31–46. Springer, Berlin, Heidelberg, January 2003.
17. Dan Boneh, Ben Lynn, and Hovav Shacham. Short signatures from the Weil pairing. *Journal of Cryptology*, 17(4):297–319, September 2004.
18. L. Brandao and Peralta R. NIST first call for multi-party threshold schemes, 2023. <https://csrc.nist.gov/publications/detail/nistir/8214c/draft>.
19. Jan Camenisch and Thomas Groß. Efficient attributes for anonymous credentials. In Peng Ning, Paul F. Syverson, and Somesh Jha, editors, *ACM CCS 2008*, pages 345–356. ACM Press, October 2008.
20. Jan Camenisch and Anna Lysyanskaya. An identity escrow scheme with appointed verifiers. In Joe Kilian, editor, *CRYPTO 2001*, volume 2139 of *LNCS*, pages 388–407. Springer, Berlin, Heidelberg, August 2001.
21. Jan Camenisch and Markus Stadler. Proof systems for general statements about discrete logarithms. Technical Report No. 260, March 1997. <ftp://ftp.inf.ethz.ch/pub/publications/tech-reports/>.
22. Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *42nd FOCS*, pages 136–145. IEEE Computer Society Press, October 2001.
23. Ran Canetti, Rosario Gennaro, Steven Goldfeder, Nikolaos Makriyannis, and Udi Peled. UC non-interactive, proactive, threshold ECDSA with identifiable aborts. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *ACM CCS 2020*, pages 1769–1787. ACM Press, November 2020.
24. Ran Canetti, Rosario Gennaro, Stanislaw Jarecki, Hugo Krawczyk, and Tal Rabin. Adaptive security for threshold cryptosystems. In Michael J. Wiener, editor, *CRYPTO’99*, volume 1666 of *LNCS*, pages 98–115. Springer, Berlin, Heidelberg, August 1999.
25. Sílvia Casacuberta, Julia Hesse, and Anja Lehmann. SoK: Oblivious pseudorandom functions. Cryptology ePrint Archive, Report 2022/302, 2022.
26. Guilhem Castagnos, Dario Catalano, Fabien Laguillaumie, Federico Savasta, and Ida Tucker. Bandwidth-efficient threshold EC-DSA. In Aggelos Kiayias, Markulf

- Kohlweiss, Petros Wallden, and Vassilis Zikas, editors, *PKC 2020, Part II*, volume 12111 of *LNCS*, pages 266–296. Springer, Cham, May 2020.
27. Sofia Celi, Alex Davidson, Steven Valdez, and Christopher A. Wood. Privacy Pass Issuance Protocols. RFC 9578, June 2024.
 28. Rutchathon Chairattana-Apirom, Lucjan Hanzlik, Julian Loss, Anna Lysyanskaya, and Benedikt Wagner. PI-cut-choo and friends: Compact blind signatures via parallel instance cut-and-choose and more. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part III*, volume 13509 of *LNCS*, pages 3–31. Springer, Cham, August 2022.
 29. Rutchathon Chairattana-Apirom, Stefano Tessaro, and Chenzhi Zhu. Pairing-free blind signatures from CDH assumptions. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part I*, volume 14920 of *LNCS*, pages 174–209. Springer, Cham, August 2024.
 30. David Chaum. Blind signatures for untraceable payments. In David Chaum, Ronald L. Rivest, and Alan T. Sherman, editors, *CRYPTO’82*, pages 199–203. Plenum Press, New York, USA, 1982.
 31. David Chaum, Amos Fiat, and Moni Naor. Untraceable electronic cash. In Shafi Goldwasser, editor, *CRYPTO’88*, volume 403 of *LNCS*, pages 319–327. Springer, New York, August 1990.
 32. Hien Chu, Paul Gerhart, Tim Ruffing, and Dominique Schröder. Practical Schnorr threshold signatures without the algebraic group model. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part I*, volume 14081 of *LNCS*, pages 743–773. Springer, Cham, August 2023.
 33. Aisling Connolly, Pascal Lafourcade, and Octavio Perez Kempner. Improved constructions of anonymous credentials from structure-preserving signatures on equivalence classes. In Goichiro Hanaoka, Junji Shikata, and Yohei Watanabe, editors, *Public-Key Cryptography – PKC 2022*, pages 409–438, Cham, 2022. Springer International Publishing.
 34. Elizabeth C. Crites, Chelsea Komlo, and Mary Maller. Fully adaptive Schnorr threshold signatures. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part I*, volume 14081 of *LNCS*, pages 678–709. Springer, Cham, August 2023.
 35. Elizabeth C. Crites, Chelsea Komlo, Mary Maller, Stefano Tessaro, and Chenzhi Zhu. Snowblind: A threshold blind signature in pairing-free groups. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part I*, volume 14081 of *LNCS*, pages 710–742. Springer, Cham, August 2023.
 36. Ivan Damgård, Thomas Pelle Jakobsen, Jesper Buus Nielsen, Jakob Illeborg Pagter, and Michael Bækvang Østergaard. Fast threshold ECDSA with honest majority. In Clemente Galdi and Vladimir Kolesnikov, editors, *SCN 20*, volume 12238 of *LNCS*, pages 382–400. Springer, Cham, September 2020.
 37. Poulami Das, Julia Hesse, and Anja Lehmann. DPaSE: Distributed password-authenticated symmetric-key encryption, or how to get many keys from one password. In Yuji Suga, Kouichi Sakurai, Xuhua Ding, and Kazue Sako, editors, *ASIACCS 22*, pages 682–696. ACM Press, May / June 2022.
 38. Sourav Das, Philippe Camacho, Zhuolun Xiang, Javier Nieto, Benedikt Bünz, and Ling Ren. Threshold signatures from inner product argument: Succinct, weighted, and multi-threshold. In Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda, editors, *ACM CCS 2023*, pages 356–370. ACM Press, November 2023.

39. Sourav Das and Ling Ren. Adaptively secure bls threshold signatures from ddh and co-cdh. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology – CRYPTO 2024*, pages 251–284, Cham, 2024. Springer Nature Switzerland.
40. Frank Denis, Frederic Jacobs, and Christopher A. Wood. RSA Blind Signatures. RFC 9474, October 2023.
41. Yvo Desmedt. Society and group oriented cryptography: A new concept. In Carl Pomerance, editor, *CRYPTO’87*, volume 293 of *LNCS*, pages 120–127. Springer, Berlin, Heidelberg, August 1988.
42. Yvo Desmedt and Yair Frankel. Threshold cryptosystems. In Gilles Brassard, editor, *CRYPTO’89*, volume 435 of *LNCS*, pages 307–315. Springer, New York, August 1990.
43. Jack Doerner, Yashvanth Kondi, Eysa Lee, abhi shelat, and LaKiah Tyner. Threshold BBS+ signatures for distributed anonymous credential issuance. In *2023 IEEE Symposium on Security and Privacy*, pages 773–789. IEEE Computer Society Press, May 2023.
44. Yair Frankel, Philip D. MacKenzie, and Moti Yung. Adaptively-secure optimal-resilience proactive RSA. In Kwok-Yan Lam, Eiji Okamoto, and Chaoping Xing, editors, *ASIACRYPT’99*, volume 1716 of *LNCS*, pages 180–194. Springer, Berlin, Heidelberg, November 1999.
45. Michael J. Freedman, Yuval Ishai, Benny Pinkas, and Omer Reingold. Keyword search and oblivious pseudorandom functions. In Joe Kilian, editor, *TCC 2005*, volume 3378 of *LNCS*, pages 303–324. Springer, Berlin, Heidelberg, February 2005.
46. Georg Fuchsbauer, Christian Hanser, and Daniel Slamanig. Structure-preserving signatures on equivalence classes and constant-size anonymous credentials. *Journal of Cryptology*, 1:1–49, 2018.
47. Steven D. Galbraith, Kenneth G. Paterson, and Nigel P. Smart. Pairings for cryptographers. *Discrete Applied Mathematics*, 156(16):3113–3121, 2008. Applications of Algebra to Cryptography.
48. Sanjam Garg, Abhishek Jain, Pratyay Mukherjee, Rohit Sinha, Mingyuan Wang, and Yinuo Zhang. hinTS: Threshold signatures with silent setup. Cryptology ePrint Archive, Report 2023/567, 2023.
49. Rosario Gennaro and Steven Goldfeder. Fast multiparty threshold ECDSA with fast trustless setup. In David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang, editors, *ACM CCS 2018*, pages 1179–1194. ACM Press, October 2018.
50. Rosario Gennaro, Steven Goldfeder, and Arvind Narayanan. Threshold-optimal DSA/ECDSA signatures and an application to bitcoin wallet security. In Mark Manulis, Ahmad-Reza Sadeghi, and Steve Schneider, editors, *ACNS 16 International Conference on Applied Cryptography and Network Security*, volume 9696 of *LNCS*, pages 156–174. Springer, Cham, June 2016.
51. Trust Tokens, Apr 2024. <https://developer.chrome.com/docs/privacy-sandbox/trust-tokens/>.
52. Panagiotis Grontas, Aris Pagourtzis, Alexandros Zacharakis, and Bingsheng Zhang. Towards everlasting privacy and efficient coercion resistance in remote electronic voting. In Aviv Zohar, Ittay Eyal, Vanessa Teague, Jeremy Clark, Andrea Bracciali, Federico Pintore, and Massimiliano Sala, editors, *FC 2018 Workshops*, volume 10958 of *LNCS*, pages 210–231. Springer, Berlin, Heidelberg, March 2019.
53. Y. Gu, S. Jarecki, P. Kedzior, P. Nazarian, and J. Xu. Threshold PAKE with security against compromise of all servers. In *Advances in Cryptology – ASIACRYPT 2024*, 2024.

54. Christian Hanser. *Signatures on Equivalence Classes: A New Tool for Privacy-Enhancing Cryptography*. PhD thesis, Graz University of Technology, Austria, February 2016.
55. Lucjan Hanzlik, Julian Loss, and Benedikt Wagner. Rai-choo! Evolving blind signatures to the next level. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part V*, volume 14008 of *LNCS*, pages 753–783. Springer, Cham, April 2023.
56. Carmit Hazay and Yehuda Lindell. Efficient protocols for set intersection and pattern matching with security against malicious and covert adversaries. In Ran Canetti, editor, *TCC 2008*, volume 4948 of *LNCS*, pages 155–175. Springer, Berlin, Heidelberg, March 2008.
57. Ethan Heilman, Foteini Baldimtsi, and Sharon Goldberg. Blindly signed contracts: Anonymous on-blockchain and off-blockchain bitcoin transactions. In Jeremy Clark, Sarah Meiklejohn, Peter Y. A. Ryan, Dan S. Wallach, Michael Brenner, and Kurt Rohloff, editors, *FC 2016 Workshops*, volume 9604 of *LNCS*, pages 43–60. Springer, Berlin, Heidelberg, February 2016.
58. Stanislaw Jarecki, Aggelos Kiayias, and Hugo Krawczyk. Round-optimal password-protected secret sharing and T-PAKE in the password-only model. In Palash Sarkar and Tetsu Iwata, editors, *ASIACRYPT 2014, Part II*, volume 8874 of *LNCS*, pages 233–253. Springer, Berlin, Heidelberg, December 2014.
59. Stanislaw Jarecki, Aggelos Kiayias, Hugo Krawczyk, and Jiayu Xu. TOPPSS: Cost-minimal password-protected secret sharing based on threshold OPRF. In Dieter Gollmann, Atsuko Miyaji, and Hiroaki Kikuchi, editors, *ACNS 17 International Conference on Applied Cryptography and Network Security*, volume 10355 of *LNCS*, pages 39–58. Springer, Cham, July 2017.
60. Stanislaw Jarecki, Hugo Krawczyk, and Jason K. Resch. Updatable oblivious key management for storage systems. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019*, pages 379–393. ACM Press, November 2019.
61. Stanislaw Jarecki, Hugo Krawczyk, and Jiayu Xu. OPAQUE: An asymmetric PAKE protocol secure against pre-computation attacks. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part III*, volume 10822 of *LNCS*, pages 456–486. Springer, Cham, April / May 2018.
62. Stanislaw Jarecki and Xiaomin Liu. Efficient oblivious pseudorandom function with applications to adaptive OT and secure computation of set intersection. In Omer Reingold, editor, *TCC 2009*, volume 5444 of *LNCS*, pages 577–594. Springer, Berlin, Heidelberg, March 2009.
63. Stanislaw Jarecki and Xiaomin Liu. Fast secure computation of set intersection. In Juan A. Garay and Roberto De Prisco, editors, *SCN 10*, volume 6280 of *LNCS*, pages 418–435. Springer, Berlin, Heidelberg, September 2010.
64. Erich Kaltofen and Victor Shoup. Subquadratic-time factoring of polynomials over finite fields. In *Proceedings of the Twenty-Seventh Annual ACM Symposium on Theory of Computing*, STOC '95, page 398–406, New York, NY, USA, 1995. Association for Computing Machinery.
65. Julia Kastner, Ky Nguyen, and Michael Reichle. Pairing-free blind signatures from standard assumptions in the ROM. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part I*, volume 14920 of *LNCS*, pages 210–245. Springer, Cham, August 2024.
66. Jinho Kim, Kwangjo Kim, and Chulsoo Lee. An efficient and provably secure threshold blind signature. In *4th International Conference on Information Security and Cryptology – ICISC'01*, Berlin, Heidelberg, 2001. Springer-Verlag.

67. Sungwook Kim, Hyeonbum Lee, and Jae Hong Seo. Efficient zero-knowledge arguments in discrete logarithm setting: Sublogarithmic proof or sublinear verifier. In Shweta Agrawal and Dongdai Lin, editors, *Advances in Cryptology – ASIACRYPT 2022*, pages 403–433, Cham, 2022. Springer Nature Switzerland.
68. Vladimir Kolesnikov, Ranjit Kumaresan, Mike Rosulek, and Ni Trieu. Efficient batched oblivious PRF with applications to private set intersection. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *ACM CCS 2016*, pages 818–829. ACM Press, October 2016.
69. Chelsea Komlo and Ian Goldberg. FROST: Flexible round-optimized Schnorr threshold signatures. In Orr Dunkelman, Michael J. Jacobson Jr., and Colin O’Flynn, editors, *SAC 2020*, volume 12804 of *LNCS*, pages 34–65. Springer, Cham, October 2020.
70. Veronika Kuchta and Mark Manulis. Rerandomizable threshold blind signatures. In *Trusted Systems- 6th International Conference – INTRUST 2014*, 2014.
71. Anja Lehmann, Phillip Nazarian, and Cavit Özbay. Stronger security for threshold blind signatures. In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology – EUROCRYPT 2025*, pages 335–364, Cham, 2025. Springer Nature Switzerland.
72. Benoît Libert, Marc Joye, and Moti Yung. Born and raised distributively: fully distributed non-interactive adaptively-secure threshold signatures with short shares. In Magnús M. Halldórsson and Shlomi Dolev, editors, *33rd ACM PODC*, pages 303–312. ACM, July 2014.
73. Yehuda Lindell. Fast secure two-party ECDSA signing. In Jonathan Katz and Hovav Shacham, editors, *CRYPTO 2017, Part II*, volume 10402 of *LNCS*, pages 613–644. Springer, Cham, August 2017.
74. Yehuda Lindell and Ariel Nof. Fast secure multiparty ECDSA with practical distributed key generation and applications to cryptocurrency custody. In David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang, editors, *ACM CCS 2018*, pages 1837–1854. ACM Press, October 2018.
75. Anna Lysyanskaya and Chris Peikert. Adaptive security in the threshold setting: From cryptosystems to signature schemes. In Colin Boyd, editor, *ASIACRYPT 2001*, volume 2248 of *LNCS*, pages 331–350. Springer, Berlin, Heidelberg, December 2001.
76. Moni Naor and Omer Reingold. Number-theoretic constructions of efficient pseudo-random functions. *J. ACM*, 51(2):231–262, mar 2004.
77. Tatsuaki Okamoto and Kazuo Ohta. Universal electronic cash. In Joan Feigenbaum, editor, *CRYPTO’91*, volume 576 of *LNCS*, pages 324–337. Springer, Berlin, Heidelberg, August 1992.
78. David Pointcheval and Jacques Stern. Security arguments for digital signatures and blind signatures. *Journal of Cryptology*, 13(3):361–396, June 2000.
79. Víctor Reyes-Macedo, Akinori Kawachi, Gina Gallegos-García, and Moisés Salinas-Rosales. A threshold-blind signature scheme and its application in blockchain-based systems. *IEEE Access*, 12:138239–138251, 2024.
80. Alfredo Rial and Ania M. Piotrowska. Security analysis of coconut, an attribute-based credential scheme with threshold issuance. Cryptology ePrint Archive, Report 2022/011, 2022.
81. Claus-Peter Schnorr. Security of blind discrete log signatures against interactive attacks. In Sihan Qing, Tatsuaki Okamoto, and Jianying Zhou, editors, *ICICS 01*, volume 2229 of *LNCS*, pages 1–12. Springer, Berlin, Heidelberg, November 2001.

82. Markus Stadler. Publicly verifiable secret sharing. In Ueli Maurer, editor, *Advances in Cryptology — EUROCRYPT '96*, pages 190–199, Berlin, Heidelberg, 1996. Springer Berlin Heidelberg.
83. Stefano Tessaro and Chenzhi Zhu. Short pairing-free blind signatures with exponential security. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part II*, volume 13276 of *LNCS*, pages 782–811. Springer, Cham, May / June 2022.
84. Masayuki Tezuka and Keisuke Tanaka. Redactable signature with compactness from set-commitment. *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, E104.A(9):1175–1187, September 2021.
85. Nirvan Tyagi, Sofia Celi, Thomas Ristenpart, Nick Sullivan, Stefano Tessaro, and Christopher A. Wood. A fast and simple partially oblivious PRF, with applications. In Orr Dunkelman and Stefan Dziembowski, editors, *Advances in Cryptology – EUROCRYPT 2022*, pages 674–705, Cham, 2022. Springer International Publishing.
86. Duc-Liem Vo, Fangguo Zhang, and Kwangjo Kim. A new threshold blind signature scheme from pairings. In *Symposium on Cryptography and Information Security – SCIS 2023*, 2003.
87. Joachim von zur Gathen and Daniel Panario. Factoring polynomials over finite fields: A survey. *Journal of Symbolic Computation*, 31(1):3–17, 2001.
88. Pengfei Wang, Xiangyu Su, Mario Larangeira, and Keisuke Tanaka. Auditable attribute-based credentials scheme and its application in contact tracing. In Christina Pöpper and Lejla Batina, editors, *Applied Cryptography and Network Security*, pages 88–118, Cham, 2024. Springer Nature Switzerland.

A Additional Preliminaries

In this section we include standard definitions deferred from Section 2 due to space constraints. In particular, we recall the setting of a bilinear pairing group needed for BLS signatures, and the assumptions used in our security proofs.

Definition 12 (Bilinear Pairing). *Let $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ be cyclic groups of prime order m with generators g_1, g_2, g_T , respectively. A bilinear pairing is an efficiently computable function $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ with the property that $e((g_1)^a, (g_2)^b) = (g_T)^{ab}$ for all $a, b \in \mathbb{Z}_m$.*

In a type 1 (or “symmetric”) pairing setup, there are efficiently computable homomorphisms from $\mathbb{G}_1$ to $\mathbb{G}_2$ and from $\mathbb{G}_2$ to $\mathbb{G}_1$ (it may be that $\mathbb{G}_1 = \mathbb{G}_2$). In a type 2 pairing setup, there is an efficiently computable homomorphism only from $\mathbb{G}_2$ to $\mathbb{G}_1$. In a type 3 pairing setup, there is no efficiently computable homomorphism in either direction.

Definition 13 (Discrete Logarithm Problem). *Let $\mathbb{G}$ be a cyclic group of prime order m with generator g . The Discrete Logarithm problem (DL) on $\mathbb{G}$ is, given (g, g^ξ) where $\xi \leftarrow_{\$} \mathbb{Z}_m$, to find ξ . We say that DL is (ϵ, T) -hard in $\mathbb{G}$ if all algorithms running in time T have no better than ϵ probability of solving the DL problem.*

Definition 14 (N-Relational Discrete Logarithm Problem). Let $\mathbb{G}$ be a cyclic group of prime order m with generator g . The N -Relational Discrete Logarithm problem (N -RDL) on $\mathbb{G}$ is, given $(g^{\xi_1}, \dots, g^{\xi_N})$ where $\xi_1, \dots, \xi_N \leftarrow_{\$} \mathbb{Z}_m$, to find $c_1, \dots, c_N \in \mathbb{Z}_m$ such that $\prod_{i \in [N]} (g^{\xi_i})^{c_i} = 1$ and $\exists_{i \in [N]} c_i \neq 0$. We say that N -RDL is (ϵ, T) -hard in $\mathbb{G}$ if all algorithms running in time T have no better than ϵ probability of solving the N -RDL problem.

Lemma 6. If DL is (ϵ, T) -hard in $\mathbb{G}$, then N -RDL is $(\frac{m}{m-1} \cdot \epsilon, \mathcal{O}(T - N \log m))$ -hard in $\mathbb{G}$.

The equivalence between the DL and N -RDL problems is well-known [67]. We include a brief proof.

Proof. Suppose there exists an adversary $\mathcal{A}$ that runs in time T and solves N -RDL with probability ϵ_{RDL} . We can use $\mathcal{A}$ to solve for ξ given a DL input (g, g^ξ) . For each $i \in [N]$, pick $a_i, b_i \leftarrow_{\$} \mathbb{Z}_m$ and define $h_i := g^{a_i} \cdot (g^\xi)^{b_i}$. Run $\mathcal{A}$ on RDL challenge $(h_1, \dots, h_N)$. Notice that, in effect, $\xi_i = a_i + b_i \xi$ for all i .

If $\mathcal{A}$ is successful, it outputs $(c_1, \dots, c_N)$ such that $\prod_{i \in [N]} (h_i)^{c_i} = 1$. In the exponent, we have the equation $\sum_{i \in [N]} (a_i + b_i \xi) c_i = 0$. We can isolate $\xi = (-\sum_{i \in [N]} a_i c_i) / (\sum_{i \in [N]} b_i c_i)$. As long as the denominator $\sum_{i \in [N]} b_i c_i \neq 0$, we solve DL.

Since $\mathcal{A}$ is successful, $\exists_{i \in [N]} c_i \neq 0$. Notice that b_i is information-theoretically hidden from $\mathcal{A}$, i.e. h_i is independent of b_i . It follows by a one-time pad argument that $\sum_{i \in [N]} b_i c_i$ is a uniformly random element of $\mathbb{Z}_m$ and $\Pr[\sum_{i \in [N]} b_i c_i = 0] = \frac{1}{m}$. Overall, then, our probability of solving DL is $(1 - \frac{1}{m}) \cdot \epsilon_{RDL}$. Our hardness assumption therefore implies that $\epsilon_{RDL} \leq \frac{m}{m-1} \cdot \epsilon$. $\square$

For brevity, we omit the factor $\frac{m}{m-1} \approx 1$ throughout this work.

Definition 15 (q -Strong Discrete Logarithm Problem). Let $\mathbb{G}$ be a cyclic group of prime order m with generator g . The q -Strong Discrete Logarithm problem (q -SDL) on $\mathbb{G}$ is, given $(g, g^\xi, g^{\xi^2}, \dots, g^{\xi^q})$ where $\xi \leftarrow_{\$} \mathbb{Z}_m$, to find ξ . We say that q -SDL is (ϵ, T) -hard in $\mathbb{G}$ if all algorithms running in time T have no better than ϵ probability of solving the q -SDL problem.

DL is a special case of q -SDL where $q = 1$.

Proving security of our pairing-based signature scheme requires a different version of q -SDL adapted to that setting.

Definition 16 (q -Strong co-Discrete Logarithm Problem). Let $\mathbb{G}_1, \mathbb{G}_2$ be cyclic groups of prime order m with generators g_1, g_2 , respectively. The q -Strong co-Discrete Logarithm problem (q -co-SDL) on $(\mathbb{G}_1, \mathbb{G}_2)$ is, given $(g_1, (g_1)^\xi, (g_1)^{\xi^2}, \dots, (g_1)^{\xi^q}, g_2, (g_2)^\xi)$ where $\xi \leftarrow_{\$} \mathbb{Z}_m$, to find ξ . We say that q -co-SDL is (ϵ, T) -hard in $(\mathbb{G}_1, \mathbb{G}_2)$ if all algorithms running in time T have no better than ϵ probability of solving the q -co-SDL problem.

We note that our variant of the q -co-SDL assumption is weaker than the one that has appeared in previous pairing-based works, e.g. [54, 46, 84, 33, 88], as our version provides only $(g_2)^\xi$ and not all q higher powers $(g_2)^{\xi^2}, \dots, (g_2)^{\xi^q}$.

Definition 17 (*t*-One-More Discrete Logarithm Problem). Let $\mathbb{G}$ be a cyclic group of prime order m with generator g . The *t*-One-More Discrete Logarithm problem (*t*-OMDL) on $\mathbb{G}$ is, given $(g, g^{\xi_1}, \dots, g^{\xi_t})$ where $\xi_1, \dots, \xi_t \leftarrow_{\$} \mathbb{Z}_m$ and given access to discrete logarithm oracle $\text{DL}_g(\cdot)$, to find all $\xi_1, \dots, \xi_t$ while making at most $t - 1$ calls to $\text{DL}_g(\cdot)$. We say that *t*-OMDL is (ϵ, T) -hard in $\mathbb{G}$ if all algorithms running in time T have no better than ϵ probability of solving the *t*-OMDL problem.

DL is a special case of *t*-OMDL where $t = 1$.

Definition 18 (Decisional Diffie-Hellman Problem). Let $\mathbb{G}$ be a cyclic group of prime order m with generator g . The Decisional Diffie-Hellman problem (DDH) on $\mathbb{G}$ is to distinguish Diffie-Hellman tuples from random tuples. In particular, the DDH advantage of an algorithm $\mathcal{A}$ is defined as:

$$\left| \Pr_{a,b \leftarrow_{\$} \mathbb{Z}_m} [\mathcal{A}(g, g^a, g^b, g^{ab}) = 1] - \Pr_{a,b,c \leftarrow_{\$} \mathbb{Z}_m} [\mathcal{A}(g, g^a, g^b, g^c) = 1] \right|$$

We say that DDH is (ϵ, T) -hard in $\mathbb{G}$ if all algorithms running in time T have no better than ϵ DDH advantage.

B Non-Interactive Zero Knowledge Proofs

The protocols in Figure 3 require two non-interactive zero knowledge proof of knowledge systems (NIZKs), denoted $\text{NIZK}_{\text{KeyGen}}$ and $\text{NIZK}_{\text{SEval}}$. Intuitively, the former ensures that public keys are well-formed, and the latter ensures that servers behave honestly. We represent a NIZK as a pair of algorithms ($\text{Prove}, \text{Verify}$) for a relation $\mathcal{R}$ of statement/witness pairs. $\text{Prove}(s, w) \rightarrow \pi$ produces a proof of knowledge π for $(s, w) \in \mathcal{R}$. $\text{Verify}(s, \pi) \rightarrow 0/1$ verifies whether π proves knowledge of a witness w such that $(s, w) \in \mathcal{R}$.

- $\text{NIZK}_{\text{KeyGen}}$ is a proof system for statements of the form $\text{pk} \in \mathbb{G}_1$ (for 2B-HashTDH) or $\text{pk} \in \mathbb{G}_2$ (for 2B-tBlindBLS) and witnesses of the form $k \in \mathbb{Z}_m$. Relation $\mathcal{R}_{\text{KeyGen}}$ is the set of (pk, k) pairs such that $\text{pk} = (g_k)^k$ (for 2B-HashTDH) or $\text{pk} = (g_2)^k$ (for 2B-tBlindBLS).
- $\text{NIZK}_{\text{SEval}}$ is a proof system for statements of the form $(p, h_1, h_2, \text{pk}_i, q_i) \in (\mathbb{G}_1)^5$ and witnesses of the form $(k_i, z_i, \hat{z}_i) \in (\mathbb{Z}_m)^3$. Relation $\mathcal{R}_{\text{SEval}}$ is the set of $((p, h_1, h_2, \text{pk}_i, q_i), (k_i, z_i, \hat{z}_i))$ pairs such that $\text{pk}_i = (g_k)^{k_i} \cdot (g_z)^{z_i} \cdot (g_z)^{\hat{z}_i}$ and $q_i = (p)^{k_i} \cdot (h_1)^{z_i} \cdot (h_2)^{\hat{z}_i}$.

These NIZKs are required to satisfy Completeness, Simulatability, and Strong Soundness. We consider NIZKs in the random oracle model, where the Prove and Verify algorithms have oracle access to a function H_{RO} randomly sampled from some distribution $\mathcal{H}$.

Definition 19 (Completeness). Non-interactive zero knowledge proof of knowledge system NIZK for relation $\mathcal{R}$ is complete iff, for every $(s, w) \in \mathcal{R}$, $\text{NIZK.Verify}(s, \text{NIZK.Prove}(s, w))$ always produces 1.

Definition 20 (Simulatability). The advantage of adversary $\mathcal{A}$ against protocol NIZK for relation $\mathcal{R}$ in the simulatability game is defined as:

$$\text{Adv}_{\mathcal{A}, \text{NIZK}, \text{SIM}}^{\text{simulatable}}(\kappa) = |\Pr[\mathcal{A}^{\text{NIZK.Prove}(\cdot, \cdot)}(1^\kappa) = 1] - \Pr[\mathcal{A}^{\text{SIM}(\cdot)}(1^\kappa) = 1]|$$

where $\mathcal{A}$ can only query its oracle on $(s, w) \in \mathcal{R}$ and SIM is an algorithm that takes only the s part as its input. In the first probability, H_{RO} is randomly sampled from $\mathcal{H}$. In the second probability, SIM programs the H_{RO} queries of $\mathcal{A}$.

Non-interactive zero knowledge proof of knowledge system NIZK is simulatable iff there exists an efficient simulator SIM such that, for all PPT adversaries $\mathcal{A}$, $\text{Adv}_{\mathcal{A}, \text{NIZK}, \text{SIM}}^{\text{simulatable}}$ is negligible.

Definition 21 (Strong Soundness). The acceptance event $\text{acc}_{\mathcal{A}}$ of prover $\mathcal{A}$ against protocol NIZK for relation $\mathcal{R}$ is defined as:

$$\mathcal{A}(1^\kappa; \rho) \rightarrow (\{(s_i, \pi_i)\}_{i \in [\omega]}, \phi) \text{ s.t. } \forall_{i \in [\omega]} \text{NIZK.Verify}(s_i, \pi_i) = 1$$

where $\mathcal{A}$ is a probabilistic algorithm that runs on randomness ρ sampled from some distribution $P_{\mathcal{A}}$. $\mathcal{A}$ outputs ω proofs $\{(s_i, \pi_i)\}_{i \in [\omega]}$ and a side output ϕ .

The acceptance event acc_{EXT} of extractor EXT against prover $\mathcal{A}$ is defined as:

$$\begin{aligned} & \mathcal{A}(1^\kappa; \rho) \rightarrow (\{(s_i, \pi_i)\}_{i \in [\omega]}, \phi) \text{ s.t. } \forall_{i \in [\omega]} \text{NIZK.Verify}(s_i, \pi_i) = 1 \\ & \wedge \text{EXT}^{\mathcal{A}(1^\kappa; \rho)}(1^\kappa) \rightarrow (\{w_i\}_{i \in [\omega]}) \text{ s.t. } \forall_{i \in [\omega]} (s_i, w_i) \in \mathcal{R} \end{aligned}$$

In this experiment, EXT and the “main” execution of $\mathcal{A}$ have oracle access to random function H_{RO} . For the executions of $\mathcal{A}$ run by EXT, EXT programs H_{RO} . Every execution of $\mathcal{A}$ runs on the same randomness ρ .

Non-interactive zero knowledge proof of knowledge system NIZK is sound iff, for every PPT prover $\mathcal{A}$, if $\Pr_{\rho \leftarrow \$_{P_{\mathcal{A}}}, H_{\text{RO}} \leftarrow \$_{\mathcal{H}}}[\text{acc}_{\mathcal{A}}]$ is non-negligible, then there exists a PPT extractor EXT such that $\Pr_{\rho \leftarrow \$_{P_{\mathcal{A}}}, H_{\text{RO}} \leftarrow \$_{\mathcal{H}}}[\text{acc}_{\text{EXT}}]$ is non-negligible.

The extraction loss $L_{\text{EXT}}(\cdot)$ is a lower-bound on the probability of acc_{EXT} as a function of the probability of $\text{acc}_{\mathcal{A}}$. The extraction time $T_{\text{EXT}}(\cdot)$ is an upper-bound on the runtime of $\text{EXT}^{\mathcal{A}}$ as a function of the runtime of $\mathcal{A}$.

Note that we are using a “multi-point” version of strong soundness that allows for (offline) extraction of multiple witnesses generated by a prover. This property is achieved by a NIZK created via the Fiat-Shamir transform applied to a Sigma protocol, per the generalized forking lemma of Bagherzandi et al. [12]. The Sigma protocols for relations $\mathcal{R}_{\text{KeyGen}}$ and $\mathcal{R}_{\text{SEval}}$ are known as proofs of discrete-logarithm representation: They are a straightforward extension of Schnorr’s proof of discrete-logarithm knowledge, and their cost is comparable to Schnorr’s signature, see e.g. the report by Camenisch and Stadler [21].

B.1 Performance Considerations

The security of 2B-HashTDH depends upon NIZK soundness only in its proof of blindness (Section 4). Even in that proof the NIZKs are only relevant to the

analysis of $\mathbf{UAgg}$ outputs, which are part of the blindness game but are excluded from the obliviousness game. Therefore, in applications where obliviousness is sufficient, NIZKs can be omitted from $\mathbf{2B}\text{-HashTDH}$ entirely. In that case, users can evaluate the $\mathbf{tOPRF}$ without needing any public data (i.e. $\mathbf{pk}, \mathbf{aux}$). As mentioned in Section 2.2, obliviousness is the more typical security expectation for $\mathbf{OPRF}$ s. Only those applications that demand a verifiable $\mathbf{tOPRF}$ need incur the overhead of public keys and NIZKs.

For $\mathbf{2B}\text{-tBlindBLS}$, the situation is even better. NIZK soundness does not appear at all in the proofs of security. The only feature that the NIZKs provide, then, is “partial verifiability,” i.e. the ability for a user to check whether each $\mathbf{SEval}$ response is correctly formed. This check is only important after an aggregated signature fails to verify. Therefore, we propose that NIZKs can and should be omitted from the normal operation of $\mathbf{2B}\text{-tBlindBLS}$. Instead, servers can provide an “Audit” interface by which a user can request a proof that a previous $\mathbf{SEval}$ response was correct. Since $\mathbf{SEval}$ is deterministic, servers can support this feature without even maintaining any state. In practice, the inevitability of being caught should strongly disincentivize dishonest server behavior, and NIZKs should be rarely needed.

C Proof of Blindness

We now prove that $\mathbf{2B}\text{-HashTDH}$ and $\mathbf{2B}\text{-tBlindBLS}$ are blind. We argue that, under expected conditions, the view of $\mathcal{A}$ in $\mathbf{Game}^{\text{blind}}$ is identically distributed regardless of the secret bit θ . In particular, the $\mathcal{O}^{\mathbf{UEval}}$ oracle always outputs two uniformly random $\mathbb{G}_1$ elements. If $\mathcal{A}$ sends honest server responses to $\mathcal{O}^{\mathbf{UAgg}}$, then that oracle outputs $(H_0(x_{0,sid})^k, H_0(x_{1,sid})^k)$, which does not depend on θ . If $\mathcal{A}$ sends dishonest responses, then $\mathbf{UAgg}$ is expected to abort and output $(\perp, \perp)$, which obviously reveals no information about θ .

Thus, blindness is perfect unless, for some sid , $\mathcal{O}^{\mathbf{UAgg}}$ produces a (non- $\perp$) output that is not of the form $(H_0(x_{0,sid})^k, H_0(x_{1,sid})^k)$ for some k . For $\mathbf{2B}\text{-tBlindBLS}$, this event is impossible due to the pairing-based verification check that $\mathbf{UAgg}$ performs; if y_0 and y_1 are not correctly formed (for some given public key $\mathbf{pk}$), $\mathbf{UAgg}$ is guaranteed to abort. For $\mathbf{2B}\text{-HashTDH}$, the event is not impossible, but it occurs with negligible probability.

Theorem 2. *If DL is (ϵ_{DL}, T_{DL}) -hard in $\mathbb{G}_1$, $\mathbf{NIZK}_{\text{KeyGen}}$ and $\mathbf{NIZK}_{\text{SEval}}$ are sound with extraction loss L_{EXT} and extraction time T_{EXT} (Definition 21), $\mathcal{A}$ runs in time $\mathcal{O}(T_{\text{EXT}}^{-1}(T_{\text{EXT}}^{-1}(T_{DL})))$, and $\mathcal{A}$ makes at most $q_{\mathbf{UAgg}}$ queries to $\mathcal{O}^{\mathbf{UAgg}}$, then $\mathbf{Adv}_{\mathcal{A}, \mathbf{2B}\text{-HashTDH}}^{\text{blind}} \leq 2q_{\mathbf{UAgg}} \cdot L_{\text{EXT}}^{-1}(L_{\text{EXT}}^{-1}(\epsilon_{DL}))$. Thus, $\mathbf{2B}\text{-HashTDH}$ is blind (Definition 5).*

Proof. Given an RDL input challenge $= (g_k, g_z, g_{\hat{z}})$, we can use $\mathcal{A}$ to find $c_k, c_z, c_{\hat{z}}$ such that $(g_k)^{c_k} \cdot (g_z)^{c_z} \cdot (g_{\hat{z}})^{c_{\hat{z}}} = 1$ and $c_k \neq 0 \vee c_z \neq 0 \vee c_{\hat{z}} \neq 0$.

Define $\mathcal{B}$ to be an algorithm that runs $\mathbf{Game}_{\mathcal{A}, \mathbf{2B}\text{-HashTDH}}^{\text{blind}}$ but uses challenge to set the public parameters. As argued above, $\mathbf{Adv}_{\mathcal{A}, \mathbf{2B}\text{-HashTDH}}^{\text{blind}}$ is

upper-bounded by the probability that, for some sid , $\mathcal{O}^{\text{UAgg}}$ produces an unexpected output. In particular, an output is unexpected if at least one of $y_{0,sid}$ and $y_{1,sid}$ is not correctly formed for the pk given in aux_{sid} .

$\mathcal{B}$ selects a sid used by $\mathcal{A}$ at random and $\theta \leftarrow_{\$} \{0,1\}$. $y_{\theta,sid}$ is not correctly formed with probability at least $\frac{1}{2q_{\text{UAgg}}} \cdot \text{Adv}_{\mathcal{A},2\text{B-HashTDH}}^{\text{blind}}$. This event implies that all NIZKs $\pi_{\text{KeyGen},sid}$ and $\{\pi_{i,\theta,sid}\}_{i \in \mathbf{S}_{\theta,sid}}$ successfully verify. $\mathcal{B}$ outputs these NIZKs.

Per the supposition that $\text{Adv}_{\mathcal{A},2\text{B-HashTDH}}^{\text{blind}}$ is non-negligible, the soundness of $\text{NIZK}_{\text{KeyGen}}$ implies the existence of an efficient extractor $\text{EXT}_{\text{KeyGen}}$ such that $\text{EXT}_{\text{KeyGen}}^{\mathcal{B}}$ outputs witness k for $\pi_{\text{KeyGen},sid}$ with probability at least $L_{\text{EXT}}(\frac{1}{2q_{\text{UAgg}}} \cdot \text{Adv}_{\mathcal{A},2\text{B-HashTDH}}^{\text{blind}})$. Similarly, the soundness of $\text{NIZK}_{\text{SEval}}$ implies the existence of an efficient extractor $\text{EXT}_{\text{SEval}}$ such that $\text{EXT}_{\text{SEval}}^{\text{EXT}_{\text{KeyGen}}^{\mathcal{B}}}$ outputs witnesses $(k_i, z_i, \hat{z}_i)$ for all $\{\pi_{i,\theta,sid}\}_{i \in \mathbf{S}_{\theta,sid}}$ with probability at least $L_{\text{EXT}}(L_{\text{EXT}}(\frac{1}{2q_{\text{UAgg}}} \cdot \text{Adv}_{\mathcal{A},2\text{B-HashTDH}}^{\text{blind}}))$.

UAgg aborts unless public keys correctly interpolate, i.e. $\text{pk} = (g_k)^k = \prod_{i \in \mathbf{S}_{\theta,sid}} \text{pk}_i^{\lambda_i}$. Representing pk_i using the extracted $(k_i, z_i, \hat{z}_i)$, we have $(g_k)^k = (g_k)^{\sum_{i \in \mathbf{S}_{\theta,sid}} \lambda_i k_i} \cdot (g_z)^{\sum_{i \in \mathbf{S}_{\theta,sid}} \lambda_i z_i} \cdot (g_{\hat{z}})^{\sum_{i \in \mathbf{S}_{\theta,sid}} \lambda_i \hat{z}_i}$. If $\sum_{i \in \mathbf{S}_{\theta,sid}} \lambda_i k_i = k$, $\sum_{i \in \mathbf{S}_{\theta,sid}} \lambda_i z_i = 0$, and $\sum_{i \in \mathbf{S}_{\theta,sid}} \lambda_i \hat{z}_i = 0$, then $y_{\theta,sid}$ would be correctly formed for public key $(g_k)^k$. It follows that one of these equalities does not hold, and we therefore have an RDL solution: $c_k = \sum_{i \in \mathbf{S}_{\theta,sid}} \lambda_i k_i - k$, $c_z = \sum_{i \in \mathbf{S}_{\theta,sid}} \lambda_i z_i$, $c_{\hat{z}} = \sum_{i \in \mathbf{S}_{\theta,sid}} \lambda_i \hat{z}_i$.

Putting it together, we have an $\mathcal{O}(T_{\text{EXT}}(T_{\text{EXT}}(T_{\mathcal{A}})))$ algorithm that wins RDL with probability $L_{\text{EXT}}(L_{\text{EXT}}(\frac{1}{2q_{\text{UAgg}}} \cdot \text{Adv}_{\mathcal{A},2\text{B-HashTDH}}^{\text{blind}}))$. Our hardness assumption therefore implies that, if $T_{\mathcal{A}} = \mathcal{O}(T_{\text{EXT}}^{-1}(T_{\text{EXT}}^{-1}(T_{DL})))$, then $\text{Adv}_{\mathcal{A},2\text{B-HashTDH}}^{\text{blind}} \leq 2q_{\text{UAgg}} \cdot L_{\text{EXT}}^{-1}(L_{\text{EXT}}^{-1}(\epsilon_{DL}))$. $\square$

D Universally Composable tOPRF

The UC *threshold* OPRF (tOPRF) functionality was first proposed by Jarecki et al. [59], based on the prior notion of UC OPRF [58]. Recently, Gu et al. [53] pointed out some ambiguities in the UC tOPRF model of [59] and proposed a fixed model, which we adopt here, as well as a protocol that securely realizes it in the static adversary setting under OMDH and DDH assumptions in ROM. We note that the UC (t)OPRF models of [58, 59, 53] are used as building blocks for applications such as password-protected secret sharing [59, 37], password-authenticated key exchange [58, 61], and a *threshold* password-authenticated key exchange (PAKE) [53], and that realizing the same functionality in the adaptive adversary setting, can lead to adaptive security of these applications. (This is the case e.g. in tPAKE of [53].)

In this section we present a variant UC-2B-HashTDH of our tOPRF protocol 2B-HashTDH, and we prove that it is adaptively secure under the UC model $\mathcal{F}_{\text{tOPRF}}$ of [53]. We thus extend our earlier game-based results and

provide an adaptively-secure tOPRF that can be used as a building block into any of the aforementioned higher-level constructions. Since our proofs are in the AGM, we use the adaptation of the UC model to AGM by Abdalla et al. [1]. Roughly speaking, Abdalla et al. [1] show that UC composition theorem applies to the AGM model, as long as the simulator used in the security proof is also *algebraic*, i.e. that it uses the group like the AGM algorithm. Technically [1] require the functionality itself to be algebraic, but that’s immediate in the case of $\mathcal{F}_{\text{tOPRF}}$ since this functionality is oblivious to the group used in the protocol that implements it. Finally (and naturally), the composition holds only for environments which are algebraic with respect to the protocol, i.e. they use the group elements output by the honest parties only as an AGM adversary would. In the theorem below we use the [1] terminology that protocol π *AGM-emulates* functionality F , but this is an AGM model counterpart to the standard notion that protocol π *UC-realizes* functionality F of Canetti [22].

D.1 Security Model

Figure 4 is the $\mathcal{F}_{\text{tOPRF}}$ functionality of [53] (with some minor syntactic changes).

$\mathcal{F}_{\text{tOPRF}}$ is in some ways stronger than the game-based definition of tOPRF given in Section 2.2. For example, $\mathcal{F}_{\text{tOPRF}}$ requires that a tOPRF be “one-more pseudorandom”, i.e. until $t + 1$ server responses are collected, the function output is not just unpredictable but completely pseudorandom. Our game-based definition separately requires one-more unpredictability and pseudorandomness, but those properties do not imply one-more pseudorandomness. Consider a protocol where every server can independently compute just the first bit of the tOPRF output for any input. Such a protocol could be one-more unpredictable (as long as the rest of the bits are unpredictable without $t + 1$ servers’ participation) and pseudorandom (as long as that first bit and the rest of the bits are pseudorandom), but it would not be one-more pseudorandom and it would not realize $\mathcal{F}_{\text{tOPRF}}$.

$\mathcal{F}_{\text{tOPRF}}$ also requires that, at the time of each evaluation, the ideal adversary commits to the instance *sid*, server set $\mathbf{S}$, and server context data *ctx* under which that evaluation is occurring. Somehow, a simulator must be able to “extract” that information from observing a real-world adversary’s behavior. This property is also not guaranteed by the game-based tOPRF definition of Section 2.2.

D.2 Protocol

Figure 6 is UC-2B-HashTDH, a tOPRF protocol realizing $\mathcal{F}_{\text{tOPRF}}$. UC-2B-HashTDH makes use of $\mathcal{F}_{\text{channel}}$, a functionality modeling a secure and authenticated communication channel, but only during initialization. $\mathcal{F}_{\text{channel}}$ is defined in Figure 5.

UC-2B-HashTDH is essentially our main tOPRF protocol 2B-HashTDH augmented with an “outer” hash function $\tilde{H}$ applied to the function’s final output y . This technique generically boosts one-more unpredictability to one-more pseudorandomness. By also including the function input x in the

Notation

The functionality interacts with a set of parties $\mathcal{P}$ and an adversary $\mathcal{A}^*$. $\mathcal{P}^* = \mathcal{P} \cup \{\mathcal{A}^*\}$. **Corr** is the initial set of corrupted parties. By convention, $\mathcal{A}^* \in \mathbf{Corr}$. We assume strings sid of form $sid = (\dots, \mathcal{S})$ where $\mathcal{S} = (S_1, \dots, S_n) \in (\mathcal{P}^*)^n$. $\mathcal{S}_{sid}$ denotes the list $\mathcal{S}$ specified by string sid .

Initially $\text{tx}[sid, \text{ctx}, i] := 0$ and $F_{sid}(x)$ are undefined for all sid, ctx, i, x . When $F_{sid}(x)$ is first referenced $\mathcal{F}_{\text{TOPRF}}$ assigns $F_{sid}(x) \leftarrow_{\$} \{0, 1\}^l$.

Initialization

1. On $(\text{toprf.init}, sid)$ from $P_0 \in \mathcal{P}^*$, if sid is new (abort otherwise):
 - send $(\text{toprf.init}, sid, P_0)$ to $\mathcal{A}^*$
 - save $(\text{toprf.init}, sid, P_0)$ and mark it **COMPR** if $P_0 \in \mathbf{Corr}$
2. On $(\text{toprf.sinit}, sid, i, P_0)$ from S where $S = \mathcal{S}_{sid}[i]$ or $(S = \mathcal{A}^* \text{ and } \mathcal{S}_{sid}[i] \in \mathbf{Corr})$, save record $(\text{toprf.sinit}, sid, i, P_0)$ marked **INACTIVE**
3. On $(\text{toprf.fininit}, sid, i)$ from $\mathcal{A}^*$ where $\exists$ record $\text{urec} = (\text{toprf.init}, sid, P_0)$ and record $\text{srec} = (\text{toprf.sinit}, sid, i, P_0)$ marked **INACTIVE**:
 - send $(\text{toprf.sinit}, sid, i)$ to $\mathcal{S}_{sid}[i]$
 - if urec is **COMPR** or $\mathcal{S}_{sid}[i] \in \mathbf{Corr}$ then mark **srec** **COMPR**
 - else (i.e. urec is not **COMPR** and $\mathcal{S}_{sid}[i] \notin \mathbf{Corr}$) mark **srec** **ACTIVE**

Corruption

4. On $(\text{toprf.corrupt}, P)$ from $\mathcal{A}^*$ (with permission from $\mathcal{Z}$):
 - set $\mathbf{Corr} := \mathbf{Corr} \cup \{P\}$
 - mark every **ACTIVE** record $(\text{toprf.sinit}, sid, i, P_0)$ **COMPR** where $P = \mathcal{S}_{sid}[i]$

Evaluation

5. On $(\text{toprf.eval}, sid, ssid_U, x)$ from $U \in \mathcal{P}^*$, if this is the first call from U for sid and $ssid_U$:
 - send $(\text{toprf.eval}, sid, ssid_U, U)$ to $\mathcal{A}^*$
 - save $(\text{toprf.eval}, sid, ssid_U, U, x)$ marked **FRESH**
6. On $(\text{toprf.sndrcomplete}, sid, i, \text{ctx})$ from S where $\exists$ record $\text{srec} = (\text{toprf.sinit}, sid, i, P_0)$ not marked **INACTIVE** and $(S = \mathcal{S}_{sid}[i] \text{ or } (S = \mathcal{A}^* \text{ and } \text{srec is marked } \mathbf{COMPR}))$:
 - send $(\text{toprf.sndrcomplete}, sid, i, \text{ctx})$ to $\mathcal{A}^*$
 - set $\text{tx}[sid, \text{ctx}, i]++$
7. On $(\text{toprf.rcvcomplete}, sid, ssid_U, sid^*, \text{ctx}^*, S)$ from $\mathcal{A}^*$ where $|S| = t + 1$ and $\exists$ record $(\text{toprf.eval}, sid, ssid_U, U, x)$ marked **FRESH**:
 - if $\exists j \in S$ such that $\text{tx}[sid^*, \text{ctx}^*, j] = 0$ then abort
 - otherwise mark the record **COMPLETED**, set $\text{tx}[sid^*, \text{ctx}^*, j]--$ for all $j \in S$, and send $(\text{toprf.eval}, sid, ssid_U, F_{sid^*}(x))$ to U

Fig. 4. $\mathcal{F}_{\text{TOPRF}}$: threshold OPRF functionality, parameterized by threshold t , number of servers n , and output length l .

Notation

The functionality interacts with a set of parties $\mathcal{P}$ and an adversary $\mathcal{A}^*$.

Secure Channel

1. On $(\text{channel.send}, \text{sid}, \text{R}, m)$ from $\text{S} \in \mathcal{P}$:
 - save $(\text{channel.message}, \text{sid}, \text{S}, \text{R}, m)$ marked PENDING
 - send $(\text{channel.send}, \text{sid}, \text{S}, \text{R}, |m|)$ to $\mathcal{A}^*$
2. On $(\text{channel.deliver}, \text{sid}, \text{S}, \text{R})$ from $\mathcal{A}^*$ where $\exists$ record $(\text{channel.message}, \text{sid}, \text{S}, \text{R}, m)$ marked PENDING:
 - mark the record COMPLETED
 - send $(\text{channel.deliver}, \text{sid}, \text{S}, m)$ to R

Fig. 5. $\mathcal{F}_{\text{channel}}$: secure and authenticated communication functionality

Setup

$\mathbb{G}_1$ is a group of prime order m with generator g_1 . $H_0 : \{0, 1\}^* \rightarrow \mathbb{G}_1$, $H_1 : \{0, 1\}^* \rightarrow \mathbb{G}_1$, $H_2 : \{0, 1\}^* \rightarrow \mathbb{G}_1$, and $\tilde{H} : \{0, 1\}^* \rightarrow \{0, 1\}^l$ are hash functions.

Initialization

1. On input $(\text{toprf.init}, \text{sid})$, initializer P_0 does:
 - pick $a_0, \dots, a_t, b_1, \dots, b_t, \hat{b}_1, \dots, \hat{b}_t \leftarrow_{\$} \mathbb{Z}_m$
 - define polynomials $k(i) := \sum_{\iota=0}^t a_{\iota} i^{\iota}$, $z(i) := \sum_{\iota=1}^t b_{\iota} i^{\iota}$, $\hat{z}(i) := \sum_{\iota=1}^t \hat{b}_{\iota} i^{\iota}$
 - $\forall i \in [n]$ $k_i := k(i)$, $z_i := z(i)$, $\hat{z}_i := \hat{z}(i)$
 - for each $i \in [n]$, send $(\text{channel.send}, (\text{sid}, i), \mathcal{S}_{\text{sid}}[i], (i, k_i, z_i, \hat{z}_i))$ to $\mathcal{F}_{\text{channel}}$
2. On input $(\text{toprf.sinit}, \text{sid}, i, \text{P}_0)$, server $\mathcal{S}_{\text{sid}}[i]$ does:
 - await $(\text{channel.deliver}, (\text{sid}, i), \text{P}_0, (i, k_i, z_i, \hat{z}_i))$ from $\mathcal{F}_{\text{channel}}$;
 - then save record $(\text{toprf.share}, \text{sid}, i, k_i, z_i, \hat{z}_i)$
 - output $(\text{toprf.finit}, \text{sid}, i)$

Evaluation

3. On input $(\text{toprf.eval}, \text{sid}, \text{ssid}_{\text{U}}, x)$, evaluator U does:
 - pick $r \leftarrow_{\$} \mathbb{Z}_m$ and compute $p := H_0(x)^r$
 - for each $i \in [n]$, send $(\text{sid}, i, \text{ssid}_{\text{U}}, p)$ to $\mathcal{S}_{\text{sid}}[i]$
 - await responses $(\text{sid}, i, \text{ssid}_{\text{U}}, q_i)$ from $\mathcal{S}_{\text{sid}}[i]$ for all $i \in \mathbf{S}$, for any set $\mathbf{S} \subseteq [n]$ of size $t + 1$;
 - then compute $q := \prod_{i \in \mathbf{S}} q_i^{\lambda_i}$ where λ_i is the Lagrange interpolation coefficient for index i and index set $\mathbf{S}$
 - output $(\text{toprf.eval}, \text{ssid}_{\text{U}}, \tilde{H}(x, q^{1/r}))$
4. On input $(\text{toprf.sndrcomplete}, \text{sid}, i, \text{ctx})$, server $\mathcal{S}_{\text{sid}}[i]$ does:
 - retrieve record $(\text{toprf.share}, \text{sid}, i, k_i, z_i, \hat{z}_i)$ (abort if not found)
 - await $(\text{sid}, i, \text{ssid}_{\text{U}}, p)$ from any U (if it hasn't already been received);
 - then compute $q_i := p^{k_i} \cdot H_1(\text{ctx}, p)^{z_i} \cdot H_2(\text{ctx}, p)^{\hat{z}_i}$
 - send response $(\text{sid}, i, \text{ssid}_{\text{U}}, q_i)$ to U

Fig. 6. Protocol UC-2B-HashTDH which realizes $\mathcal{F}_{\text{TOPRF}}$ in the $\mathcal{F}_{\text{channel}}$ -hybrid world.

input to $\tilde{H}$, we solve our protocol's *sid* extractability issue. ($\mathbf{S}$ and ctx are extractable due to the blinding factors used by the servers.)

D.3 Proof of Security

Theorem 3. *Protocol UC-2B-HashTDH $\mathbb{G}_1$ -AGM emulates functionality $\mathcal{F}_{\text{tOPRF}}$ with parameters t and n in the $(\mathcal{F}_{\text{channel}}, \mathcal{F}_{\text{RO}})$ -hybrid model, assuming hash functions $H_0, H_1, H_2, \tilde{H}$ are modeled as random oracles, and assuming the hardness of DL on $\mathbb{G}_1$ and one-more unpredictability of 2B-HashTDH.*

Specifically, we show a simulator SIM s.t. for any efficient adversary $\mathcal{A}$ against protocol $\Pi = \text{UC-2B-HashTDH}$ and any environment $\mathcal{Z}$ s.t. $(\mathcal{Z}, \mathcal{A})$ are $(\mathbb{G}_1, \Pi)$ -algebraic, then $(\mathcal{Z}, \text{SIM})$ are $(\mathbb{G}_1, \mathcal{F}_{\text{tOPRF}})$ -algebraic, and $\mathcal{Z}$'s advantage in distinguishing the view of $\mathcal{A}$ interacting with the real UC-2B-HashTDH protocol and the view of SIM interacting with the ideal functionality $\mathcal{F}_{\text{tOPRF}}$, is upper-bounded by $2q^2 \cdot \frac{m}{m-1} \cdot \epsilon_{DL} + \text{Adv}_{\mathcal{R} \leftrightarrow \mathcal{Z}, 2\text{B-HashTDH}}^{\text{om-unpredictable}}$, where q is the number of H_0 queries, $m = |\mathbb{G}_1|$, and ϵ_{DL} and $\text{Adv}_{\mathcal{R} \leftrightarrow \mathcal{Z}, 2\text{B-HashTDH}}^{\text{om-unpredictable}}$ are the advantages of algorithms (that run in time approximately equal to $\mathcal{Z}$) in the DL and 2B-HashTDH one-more unpredictability games, respectively.

Proof. For any adversary $\mathcal{A}^*$, we construct simulator SIM as shown in Figures 7, 8, and 9. Without loss of generality, we assume that $\mathcal{A}^*$ is a “dummy” adversary that merely passes messages to and from the environment $\mathcal{Z}$.

First, note that simulator SIM is $\mathbb{G}_1$ -algebraic. For simplicity of notation, we describe SIM's way of sampling random $\mathbb{G}_1$ elements in Fig. 9 using an unspecified sampling function, but this sampler can be trivially implemented in the algebraic way, by picking random exponent r and outputting $(g_1)^r$.

We now show that, for any efficient (i.e. PPT) $\mathcal{Z}$, the distinguishing advantage of $\mathcal{Z}$ between the real and simulated worlds is negligible. The argument proceeds by a series of game changes, starting from the real world $\mathcal{G}_0$ and ending at the simulated world $\mathcal{G}_4$. By $\text{Dist}_{\mathcal{Z}}^{\mathcal{G}, \mathcal{G}'}$ we denote distinguisher $\mathcal{Z}$'s distinguishing advantage between world $\mathcal{G}$ and world $\mathcal{G}'$. Specifically, $\text{Dist}_{\mathcal{Z}}^{\mathcal{G}, \mathcal{G}'} = |\Pr_{\mathcal{Z} \leftrightarrow \mathcal{G}}[\mathcal{Z} \rightarrow 1] - \Pr_{\mathcal{Z} \leftrightarrow \mathcal{G}'}[\mathcal{Z} \rightarrow 1]|$.

Game $\mathcal{G}_0$: The real world. The distinguisher $\mathcal{Z}$ interacts with UC-2B-HashTDH (Figure 6) in the role of the honest parties and in the role of the adversary. H_0, H_1, H_2 and $\tilde{H}$ are all true random oracles.

As a purely conceptual change from the true real world, one can imagine all the computational processes of the honest parties (i.e. UC-2B-HashTDH and $\mathcal{F}_{\text{channel}}$) abstracted into a single monolithic component, which we call the simulator. This component also simulates the code of $\mathcal{F}_{\text{tOPRF}}$ in parallel to its “real world” functions. By the end of the following sequence of game changes, all interactions with the honest parties will exclusively occur through the interface of $\mathcal{F}_{\text{tOPRF}}$.

Game $\mathcal{G}_1$: $\tilde{H}$ returns PRF outputs. $\mathcal{G}_1$ is $\mathcal{G}_0$ with the following changes:

Notation

Notation is as in $\mathcal{F}_{\text{TOPRF}}$. $\mathbb{G}_1$ is a group of prime order m with generator g_1 . Initially, $\text{evalset}_{sid}(\text{ctx}, p) := \emptyset$ for all sid , ctx , and p . Values t, n, l are parameters.

Honest Initialization

1. On $(\text{toprf.init}, sid, P_0)$ from $\mathcal{F}_{\text{TOPRF}}$ where $P_0 \neq \mathcal{A}^*$:
 - pick $a_0, \dots, a_t, b_1, \dots, b_t, \hat{b}_1, \dots, \hat{b}_t \leftarrow_{\$} \mathbb{Z}_m$ (such that a_0 has never been picked before)
 - define polynomials $k(i) := \sum_{\iota=0}^t a_{\iota} i^{\iota}$, $z(i) := \sum_{\iota=1}^t b_{\iota} i^{\iota}$, $\hat{z}(i) := \sum_{\iota=1}^t \hat{b}_{\iota} i^{\iota}$
 - save $(\text{toprf.init}, sid, P_0, (g_1)^{k(0)})$
 - for each $i \in [n]$, save $(\text{toprf.share}, sid, i, P_0, k(i), z(i), \hat{z}(i))$ marked INACTIVE
 - for each $i \in [n]$, send $(\text{channel.send}, (sid, i), P_0, \mathcal{S}_{sid}[i], |(i, k(i), z(i), \hat{z}(i))|)$ to $\mathcal{A}^*$
2. On $(\text{channel.deliver}, (sid, i), P_0, \mathcal{S}_{sid}[i])$ from $\mathcal{A}^*$ where $\exists$ record $(\text{toprf.share}, sid, i, P_0, k_i, z_i, \hat{z}_i)$ marked INACTIVE:
 - mark the record ACTIVE (or if $P_0 = \mathcal{A}^*$ mark it COMPR)
 - send $(\text{toprf.finit}, sid, i)$ to $\mathcal{F}_{\text{TOPRF}}$

Dishonest Initialization

3. On $(\text{channel.send}, (sid, i), \mathcal{S}_{sid}[i], (i, k_i, z_i, \hat{z}_i))$ from $\mathcal{A}^*$ on behalf of $P_0 \in \mathbf{Corr}$:
 - if this is the first such message for sid , then send $(\text{toprf.init}, sid)$ to $\mathcal{F}_{\text{TOPRF}}$ on behalf of P_0
 - save $(\text{toprf.share}, sid, i, \mathcal{A}^*, k_i, z_i, \hat{z}_i)$ marked INACTIVE

Corruption

4. On corruption by $\mathcal{A}^*$ of party $\mathcal{P}$ (with permission from $\mathcal{Z}$):
 - send $(\text{toprf.corrupt}, P)$ to $\mathcal{F}_{\text{TOPRF}}$
 - set $\mathbf{Corr} := \mathbf{Corr} \cup \{P\}$
5. If ever $\exists$ record $(\text{toprf.share}, sid, i, P_0, k_i, z_i, \hat{z}_i)$ marked ACTIVE where $\mathcal{S}_{sid}[i] \in \mathbf{Corr} - \{\mathcal{A}^*\}$, mark it COMPR and send $(\text{toprf.share}, sid, i, k_i, z_i, \hat{z}_i)$ to $\mathcal{A}^*$

Fig. 7. Simulator SIM for protocol UC-2B-HashTDH, part 1: Notation, Honest Initialization, Corrupt Initialization, and Corruption

Honest Evaluation

6. On $(\text{toprf.eval}, \text{sid}, \text{ssid}_U, U)$ from $\mathcal{F}_{\text{TOPRF}}$ where $U \neq \mathcal{A}^*$:
 - pick $r \leftarrow_{\mathcal{S}} \mathbb{Z}_m$ and $x' \leftarrow_{\mathcal{S}} \{0, 1\}^\lambda$ and define $p := H_0(x')^r$
 - for each $i \in [n]$, send $(\text{sid}, i, \text{ssid}_U, p)$ to $\mathcal{A}^*$ (addressed from U to $\mathcal{S}_{\text{sid}}[i]$)
 - and await responses $(\text{sid}, i, \text{ssid}_U, q_i)$ from $\mathcal{A}^*$ (addressed from $\mathcal{S}_{\text{sid}}[i]$ to U) for all $i \in \mathbf{S}$, for any set $\mathbf{S} \subseteq [n]$ of size $t + 1$;
 - then compute $q := \prod_{i \in \mathbf{S}} q_i^{\lambda_i}$ where λ_i is the Lagrange interpolation coefficient for index i and index set $\mathbf{S}$
 - if the algebraic representation of y is not of the form $y = H_0(x')^k$, then pick random $k \leftarrow_{\mathcal{S}} \mathbb{Z}_m$
 - run $\text{FindEvalset}((g_1)^k)$, which returns $(\text{sid}^*, \text{ctx}^*, \mathbf{S}')$ (see routine 9)
 - send $(\text{toprf.rcvcomplete}, \text{sid}, \text{ssid}_U, \text{sid}^*, \text{ctx}^*, \mathbf{S}')$ to $\mathcal{F}_{\text{TOPRF}}$
7. On $(\text{toprf.sndrcomplete}, \text{sid}, i, \text{ctx})$ from $\mathcal{F}_{\text{TOPRF}}$ and $(\text{sid}, i, \text{ssid}'_U, p)$ from $\mathcal{A}^*$ (addressed from U to $\mathcal{S}_{\text{sid}}[i]$) where $\mathcal{S}_{\text{sid}}[i] \neq \mathcal{A}^*$:
 - set $\text{evalset}_{\text{sid}}(\text{ctx}, p) := \text{evalset}_{\text{sid}}(\text{ctx}, p) \cup \{i\}$
 - retrieve $(\text{toprf.share}, \text{sid}, i, P_0, k_i, z_i, \hat{z}_i)$
 - compute $q_i := p^{k_i} \cdot H_1(\text{ctx}, p)^{z_i} \cdot H_2(\text{ctx}, p)^{\hat{z}_i}$
 - send $(\text{sid}, i, \text{ssid}'_U, \text{ctx}, q_i)$ to $\mathcal{A}^*$ (addressed from $\mathcal{S}_{\text{sid}}[i]$ to U)
8. Define subroutine $\text{FindEvalset}(k^*)$:
 - find record $(\text{toprf.init}, \text{sid}^*, P_0, k^*)$
 - if no such record exists, create it as follows:
 - choose an unused sid^* such that $\mathcal{S}_{\text{sid}^*} = (\mathcal{A}^*)^n$
 - send $(\text{toprf.init}, \text{sid}^*)$ to $\mathcal{F}_{\text{TOPRF}}$
 - receive $(\text{toprf.init}, \text{sid}^*, \mathcal{A}^*)$ in response
 - for each $i \in [n]$, send $(\text{toprf.sinit}, \text{sid}^*, i, \mathcal{A}^*)$ and $(\text{toprf.finit}, \text{sid}^*, i)$ to $\mathcal{F}_{\text{TOPRF}}$
 - save $(\text{toprf.init}, \text{sid}^*, \mathcal{A}^*, k^*)$
 - if $P_0 = \mathcal{A}^*$, then define $\mathbf{S}^* := [n]$
 - if $P_0 \neq \mathcal{A}^*$, then define $\mathbf{S}^* := \{i : \exists \text{ record } (\text{toprf.share}, \text{sid}^*, i, P_0, k_i, z_i, \hat{z}_i) \text{ marked COMP}\}$
 - for each $i \in \mathbf{S}^*$:
 - pick an unused ssid'
 - send $(\text{toprf.sndrcomplete}, \text{sid}^*, i, \text{ssid}')$ to $\mathcal{F}_{\text{TOPRF}}$
 - receive the same message in response (don't run routine 8)
 - pick any (ctx^*, p) such that $|\text{evalset}_{\text{sid}^*}(\text{ctx}^*, p)| \geq (t + 1) - |\mathbf{S}^*|$ (if no such (ctx^*, p) exists, then emit $(\text{FAIL}, \text{sid}^*)$ and halt the entire simulator)
 - pick any evaluation set $\mathbf{S}' \subseteq \mathbf{S}^* \cup \text{evalset}_{\text{sid}^*}(\text{ctx}^*, p)$ of size $|\mathbf{S}'| = t + 1$
 - set $\text{evalset}_{\text{sid}^*}(\text{ctx}^*, p) := \emptyset$
 - return $(\text{sid}^*, \text{ctx}^*, \mathbf{S}')$

Fig. 8. Simulator SIM for protocol UC-2B-HashTDH, part 2: Honest Evaluation

Dishonest Evaluation

9. On fresh query x to $H_0(\cdot)$, simply set $H_0(x) \leftarrow_{\$} \mathbb{G}_1$ and return it
10. On fresh query (ctx, p) to $H_1(\cdot)$, simply set $H_1(\text{ctx}, p) \leftarrow_{\$} \mathbb{G}_1$ and return it
11. On fresh query (ctx, p) to $H_2(\cdot)$, simply set $H_2(\text{ctx}, p) \leftarrow_{\$} \mathbb{G}_1$ and return it
12. On fresh query (x, y) to $\tilde{H}(\cdot)$:
 - if the algebraic representation of y is not of the form $y = H_0(x)^k$, then simply set $\tilde{H}(x, y) \leftarrow \{0, 1\}^*$ and return it
 - otherwise, run $\text{FindEvalset}((g_1)^k)$, which returns $(\text{sid}^*, \text{ctx}^*, \mathbf{S}')$ (see routine 9)
 - pick an unused ssid_U and send $(\text{toprf.eval}, \text{sid}^*, \text{ssid}_U, x)$ to $\mathcal{F}_{\text{TOPRF}}$
 - send $(\text{toprf.rcvcomplete}, \text{sid}^*, \text{ssid}_U, \text{sid}^*, \text{ctx}^*, \mathbf{S}')$ to $\mathcal{F}_{\text{TOPRF}}$
 - receive $(\text{toprf.eval}, \text{ssid}_U, \tilde{y})$ in response
 - set $H_3(x, y) := \tilde{y}$ and return it

Fig. 9. Simulator SIM for protocol UC-2B-HashTDH, part 3: Dishonest Evaluation

- Subroutine FindEvalset is introduced (Figure 8). Given parameter $k^* \in \mathbb{G}_1$, this subroutine finds a PRF instance sid^* that was initialized with key $k(0)$ such that $k^* = (g_1)^{k(0)}$. If no such instance exists, it uses the interface of $\mathcal{F}_{\text{TOPRF}}$ to create one, with $\mathcal{A}^*$ taking the nominal role of all n servers. In $\mathcal{G}_1$, FindEvalset does not yet search for a (ctx^*, p) pair under which a sid^* evaluation is legal. It therefore never fails and always outputs (only) sid^* .
- The simulator must answer $\tilde{H}$ queries that occur during honest user evaluation and that are made directly by the adversary. In both cases, $\tilde{H}$ uses FindEvalset to set their outputs (Figure 9). In particular, upon fresh query (x, y) , $\tilde{H}$ checks whether y is algebraically represented as $H_0(x)^k$ for some $k \in \mathbb{Z}_m$. If not, then $\tilde{H}$ simply samples a random output. If so, then $\tilde{H}$ calls $\text{FindEvalset}((g_1)^k)$, which returns sid^* . It then outputs $\tilde{y} := F_{\text{sid}^*}(x)$, where $F_{\text{sid}^*}(\cdot)$ is as defined in $\mathcal{F}_{\text{TOPRF}}$ (Figure 4).

In the previous game, $\tilde{H}(x, y)$ outputs a randomly sampled value corresponding to (x, y) . In this game, it outputs a randomly sampled value corresponding to (x, sid) , where sid is a PRF instance identifier that FindEvalset maps to (x, y) . In the case that y is not represented as $H_0(x)^k$ for some $k \in \mathbb{Z}_m$, sid is effectively a one-time value that is never used for any other (x, y) . If there exists a perfect bijection between pairs (x, y) and pairs (x, sid) , then $\mathcal{Z}$'s views of $\mathcal{G}_0$ and $\mathcal{G}_1$ are identical. Consider the two directions of this bijection:

1. Suppose there exist x, y, sid , and sid' such that $\tilde{H}(x, y)$ maps to both (x, sid) and (x, sid') .
 $\tilde{H}$ only consults FindEvalset for fresh queries (x, y) . Regardless of their representations, repeated (x, y) queries are guaranteed to produce the same output (i.e. map to the same sid). Therefore, $\text{sid} = \text{sid}'$.
2. Suppose there exist x, y, y' , and sid such that $\tilde{H}(x, y)$ and $\tilde{H}(x, y')$ both map to (x, sid) .

y and y' must be represented as $H_0(x)^k$ and $H_0(x)^{k'}$, respectively, for some $k, k' \in \mathbb{Z}_m$ (otherwise, they are guaranteed to map to different *sids*). Also, $\text{FindEvalset}((g_1)^k)$ and $\text{FindEvalset}((g_1)^{k'})$ must both map to *sid*. Per the procedure of FindEvalset , it follows that $k = k' = k(0)$, where $k(0)$ is the secret key for the PRF instance identified by *sid*. Therefore, $y = y'$.

Thus, this bijection between pairs (x, y) and pairs (x, sid) does indeed hold.

$$\mathbf{Dist}_{\mathcal{Z}}^{\mathcal{G}_0, \mathcal{G}_1} = 0$$

Game $\mathcal{G}_2$: Honest user evaluation is oblivious. $\mathcal{G}_2$ is $\mathcal{G}_1$ with the following changes:

- After picking random exponent $r \leftarrow_{\$} \mathbb{Z}_m$, the honest user evaluation procedure picks any new x' and sets $p := H_0(x')^r$ rather than $p := H_0(x)^r$ for the actual PRF input x . This ensures that the same x' is never used twice (and x' is never sent directly to $\tilde{H}$ by the adversary).
- Honest user evaluation proceeds as before, with the aggregation of q and computation of $y = q^{1/r}$. Then, however, the simulator checks whether y is algebraically represented as $H_0(x')^k$ for some $k \in \mathbb{Z}_m$. If not, it simply samples a random output $\tilde{y}$. If so, it calls $\text{FindEvalset}((g_1)^k)$, which returns sid^* . It then outputs $F_{\text{sid}^*}(x)$ for the actual input x (i.e. not x').

In both $\mathcal{G}_2$ and $\mathcal{G}_1$, p is a uniformly random group element. In both $\mathcal{G}_2$ and $\mathcal{G}_1$, honest user evaluation leads to subroutine call $\text{FindEvalset}((g_1)^k)$ where k is the discrete log $\text{DL}_p(q)$. There is only one difference between $\mathcal{G}_1$ and $\mathcal{G}_2$ that may be visible to $\mathcal{Z}$: in $\mathcal{G}_2$, honest user evaluations do not contribute to $\tilde{H}$'s “cache” of previous queries.

Even still, as long as all y values (i.e. those appearing during honest user evaluation and those sent to $\tilde{H}$ by the adversary) have acceptable algebraic representations, the games will be identical to $\mathcal{Z}$. Thus, $\mathcal{Z}$'s probability of distinguishing $\mathcal{G}_1$ from $\mathcal{G}_2$ is upper-bounded by the probability that $\mathcal{Z}$ causes two evaluations (x, y) and (x', y') such that $y = H_0(x)^k$ and $y' = H_0(x')^k$ for some $k \in \mathbb{Z}_m$, and yet either y or y' is not algebraically represented as such. At least one of these (x, y) pairs must occur during honest user evaluation; the other may come from honest user evaluation or from a direct adversarial query to $\tilde{H}$. In either case, this event corresponds to a failure by the simulator to recognize that the two evaluations are using the same key (and should therefore be mapped to the same *sid*). We will denote this event E and prove by reduction to DL that its probability is negligible.

Given a DL input **challenge** $= (g_1, (g_1)^\xi)$, we construct reduction $\mathcal{R}$, which uses $\mathcal{G}_2 \leftrightarrow \mathcal{Z}$ to solve for ξ . Denote by (x, y) and (x', y') the two evaluations that satisfy E . $\mathcal{R}$ begins by guessing $J, J' \leftarrow_{\$} [q]$ (where the H_0 queries by $\mathcal{G}_2 \leftrightarrow \mathcal{Z}$ are denoted $x_1, \dots, x_q$). $\mathcal{R}$ succeeds only if $x = x_J$ and $x' = x_{J'}$. For all $j \in [q]$, we denote $h_j := H_0(x_j)$.

Event E states that $y = (h_J)^k$ and $y' = (h_{J'})^k$ for some $k \in \mathbb{Z}_m$, yet either y or y' is not algebraically represented as such. Label the y and y' representations

as:

$$y = (g_1)^{c_0} \cdot \prod_{j \in [q]} (h_j)^{c_j}$$

$$y' = (g_1)^{c'_0} \cdot \prod_{j \in [q]} (h_j)^{c'_j}$$

H_1 and H_2 are also sources of $\mathbb{G}_1$ elements for $\mathcal{G}_2 \leftrightarrow \mathcal{Z}$. However, $\mathcal{R}$ simply programs those random oracles using group elements with known trapdoors (i.e. known discrete logarithms with respect to g_1); therefore any involvement of those group elements is included in c_0 and c'_0 in these representations.

Event E states that $c_0 \neq 0 \vee \exists_{j \in [q] - \{J\}} c_j \neq 0 \vee c'_0 \neq 0 \vee \exists_{j \in [q] - \{J'\}} c'_j \neq 0$.

Define $\gamma = \text{DL}_{h_J}(h_{J'})$, i.e. $(h_J)^\gamma = h_{J'}$. Using γ , we can combine the representations of $y = (h_J)^k$ and $y' = (h_{J'})^k$ into one equation:

$$\begin{aligned} 1 &= (g_1)^{\gamma c_0 - c'_0} \cdot \prod_{j \in [q]} (h_j)^{\gamma c_j - c'_j} \\ &= (h_J)^{\gamma c_J - c'_J} \cdot (h_{J'})^{\gamma c_{J'} - c'_{J'}} \cdot (g_1)^{\gamma c_0 - c'_0} \cdot \prod_{j \in [q] - \{J, J'\}} (h_j)^{\gamma c_j - c'_j} \\ &= (h_J)^{\gamma^2 c_{J'} + \gamma(c_J - c'_{J'}) - c'_J} \cdot (g_1)^{\gamma c_0 - c'_0} \cdot \prod_{j \in [q] - \{J, J'\}} (h_j)^{\gamma c_j - c'_j} \end{aligned}$$

Define F to be the event that some exponent in this equation is non-zero. Specifically, $\gamma^2 c_{J'} + \gamma(c_J - c'_{J'}) - c'_J \neq 0 \vee \gamma c_0 - c'_0 \neq 0 \vee \exists_{j \in [q] - \{J, J'\}} \gamma c_j - c'_j \neq 0$. $\mathcal{R}$ follows one of two reduction strategies uniformly at random. Strategy #1 succeeds if event F occurs, and strategy #2 succeeds if it doesn't. (Also, both strategies only succeed if event E occurs and J, J' are correct guesses.)

Strategy #1

This strategy follows the Relational DL approach (Definition 14). $\mathcal{R}$ picks $\gamma \leftarrow_{\$} \mathbb{Z}_m$ and programs $h_{J'} := (h_J)^\gamma$. For all other $j \in [q] - \{J'\}$ (including $j = J$), $\mathcal{R}$ picks $a_j, b_j \leftarrow_{\$} \mathbb{Z}_m$ and programs $h_j := (g_1)^{a_j} \cdot ((g_1)^\xi)^{b_j}$.

If E occurs, then $\mathcal{R}$ has values $d_0, \{d_j\}_{j \in [q] - \{J'\}}$ such that $(g_1)^{d_0} \cdot \prod_{j \in [q] - \{J'\}} (h_j)^{d_j} = 1$. In the exponent, we have the equation $d_0 + \sum_{j \in [q] - \{J'\}} (a_j + b_j \xi) d_j = 0$. We can isolate $\xi = (-d_0 - \sum_{j \in [q] - \{J'\}} a_j d_j) / (\sum_{j \in [q] - \{J'\}} b_j d_j)$. As long as the denominator $\sum_{j \in [q] - \{J'\}} b_j d_j \neq 0$, $\mathcal{R}$ solves DL.

If F occurs, $\exists_{j \in [q] - \{J'\}} d_j \neq 0$ (it is impossible that $d_0 \neq 0$ yet $\forall_{j \in [q] - \{J'\}} d_j = 0$). Notice that b_j is information-theoretically hidden from $\mathcal{G}_2 \leftrightarrow \mathcal{Z}$, i.e. h_j is independent of b_j . It follows by a one-time pad argument that $\sum_{j \in [q] - \{J'\}} b_j d_j$ is a uniformly random element of $\mathbb{Z}_m$ and $\Pr[\sum_{j \in [q] - \{J'\}} b_j d_j = 0] = \frac{1}{m}$. Therefore, strategy #1 solves DL with probability $1 - \frac{1}{m} = \frac{m-1}{m}$ if event E occurs, event F occurs, and J, J' are correct guesses.

Strategy #2

In this strategy, $\mathcal{R}$ picks $a \leftarrow_{\$} \mathbb{Z}_m$ and programs $h_J := (g_1)^a$ and $h_{J'} := ((g_1)^\xi)^a$. The goal of $\mathcal{R}$ is to compute $\xi = \gamma$. For all other $j \in [q] - \{J, J'\}$, $\mathcal{R}$ randomly programs $h_j \leftarrow_{\$} \mathbb{G}_1$.

If event F does not occur, then $\gamma^2 c_{J'} + \gamma(c_J - c'_{J'}) - c'_J = 0 \wedge \gamma c_0 - c'_0 = 0 \wedge \forall_{j \in [q] - \{J, J'\}} \gamma c_j - c'_j = 0$. If event E occurs, then either (a) $c_{J'} \neq 0 \vee c'_J \neq 0$, (b) $c_0 \neq 0 \vee c'_0 \neq 0$, or (c) $\exists_{j \in [q] - \{J, J'\}} c_j \neq 0 \vee c'_j \neq 0$. In case (a), the equation $\gamma^2 c_{J'} + \gamma(c_J - c'_{J'}) - c'_J = 0$ can be solved for γ . In case (b), the equation $\gamma c_0 - c'_0 = 0$ can be solved for γ . In case (c), the equation $\gamma c_j - c'_j = 0$ can be solved for γ . Therefore, strategy #2 solves DL with probability 1 if event E occurs, event F does not occur, and J, J' is a correct guess.

The probability of correctly guessing J, J' is $\geq \frac{1}{q^2}$ and the probability of correctly guessing whether or not F will occur is $\frac{1}{2}$. Overall, then, $\mathcal{R}$ solves DL with probability $\epsilon_{DL} \geq \Pr_{\mathcal{G}_2 \leftrightarrow \mathcal{Z}}[E] \cdot \frac{1}{q^2} \cdot \frac{1}{2} \cdot \frac{m-1}{m}$. As previously explained, $\mathcal{Z}$'s probability of distinguishing between $\mathcal{G}_1$ and $\mathcal{G}_2$ is upper-bounded by the probability of E .

$$\mathbf{Dist}_{\mathcal{Z}}^{\mathcal{G}_1, \mathcal{G}_2} \leq 2q^2 \cdot \frac{m}{m-1} \cdot \epsilon_{DL}$$

Game $\mathcal{G}_3$: Illegal evaluations cause failure. $\mathcal{G}_3$ is $\mathcal{G}_2$ with the following changes:

- As honest servers perform evaluations (i.e. `toprf.sndrcomplete`) the evalset of each (ctx, p) pair is tracked (Figure 8).
- Subroutine `FindEvalset` is expanded. After finding sid^* , it “prints blank tickets” for as many of this instance’s servers as it can (all n if the instance was just created); these blank tickets could correspond to any pair (ctx, p) . Then it finds a pair (ctx^*, p) that has been evaluated by a set $\mathbf{S}'$ of at least $t + 1$ servers and “uses it up” by resetting its evalset to empty. If no such (ctx^*, p) can be found, then it is illegal to perform an evaluation on this PRF instance; the simulator immediately emits the event $(\text{FAIL}, \text{sid}^*)$ and halts. Otherwise, the relevant instance identifier sid^* , server subsession identifier ctx^* , and evaluation set $\mathbf{S}'$ are returned. `FindEvalset` now fully matches its description in Figure 8.
- $\tilde{H}$ queries use $\mathcal{F}_{\text{TOPRF}}$'s `toprf.rcvcomplete` interface to query the output $\tilde{y}$ (using the sid^* , $\mathbf{S}'$, and ctx^* from `FindEvalset` as parameters). $\tilde{H}$ now fully matches its description in Figure 9.
- Honest user evaluation also uses $\mathcal{F}_{\text{TOPRF}}$'s `toprf.rcvcomplete` interface to query the output $\tilde{y}$ (using the sid^* , $\mathbf{S}'$, and ctx^* from `FindEvalset` as parameters). The honest user evaluation procedure now fully matches its description in Figure 8.

Observe that `FindEvalset`'s mechanism for determining the legality of an evaluation is always at least as restrictive as the “ticketing” mechanism used by $\mathcal{F}_{\text{TOPRF}}$ internally. $\mathcal{F}_{\text{TOPRF}}$ allows an evaluation as long as every server in the

evaluation set has at least one unused ticket corresponding to ctx . **FindEvalset** only allows an evaluation if every server in the evaluation set has at least one unused ticket corresponding to ctx^* **and** the same common query p . Therefore, the interactions with **toprf.rcvcomplete** introduced in this game change are guaranteed to succeed (since they are always preceded by successful calls to **FindEvalset**).

Observe furthermore that the eventual evaluation output values $\tilde{y}$ are unaltered by this game change. Therefore, the only potential change to $\mathcal{Z}$'s view is the newly added possibility of the FAIL event, which causes all execution to immediately halt. $\mathcal{Z}$'s probability of distinguishing between $\mathcal{G}_2$ and $\mathcal{G}_3$ is upper-bounded by the probability that $\mathcal{Z}$ triggers FAIL. We will prove by reduction to the one-more unpredictability of 2B-HashTDH that the probability of FAIL is negligible.

We construct reduction $\mathcal{R}$, which is an adversary against $\mathbf{Game}_{2\text{B-HashTDH}}^{\text{om-unpredictable}}$. $\mathcal{R}$ is $\mathcal{G}_3$ with the following changes:

- $\mathcal{R}$ begins by guessing the sid on which (FAIL, sid) will ultimately occur. Note that FAIL is only possible for honestly initialized sids .
- On (**toprf.init**, sid , P_0) from $\mathcal{F}_{\text{TOPRF}}$, $\mathcal{R}$ does not pick secret keys. Instead, it responds using the input from $\mathbf{Game}_{2\text{B-HashTDH}}^{\text{om-unpredictable}}$, including public key $(g_k)^k$.
- g_k is used in place of g_1 when saving public keys and when calling **FindEvalset**.
- $\mathcal{F}_{\text{TOPRF}}$ corruption events to parties in $\mathcal{S}_{\text{sid}}$ are forwarded to the $\mathcal{O}^{\text{Corrupt}}$ oracle of the $\mathbf{Game}_{2\text{B-HashTDH}}^{\text{om-unpredictable}}$ game.
- $\mathcal{R}$ uses the $\mathcal{O}^{\text{SEval}}$ oracle to respond to **toprf.sndrcomplete** messages targeting uncorrupted servers with sid .
- $\mathcal{R}$ saves a set $W = \{(x, y)\}$ of values corresponding to each **FindEvalset** execution that maps to sid . For **FindEvalset** calls originating from direct adversarial $\tilde{H}$ queries, these values are the input to $\tilde{H}$. For **FindEvalset** calls originating from honest user evaluation, the randomly chosen x' and the resulting $y = q^{1/r}$ are used.

The view of $\mathcal{Z}$ in this reduction is identical to its view in the game $\mathcal{G}_3$. The event FAIL exactly corresponds to the winning condition in the one-more unpredictability game. At the moment that FAIL occurs, the set W contains one more 2B-HashTDH evaluation pair than should be allowed. $\mathcal{R}$ outputs W and wins $\mathbf{Game}_{2\text{B-HashTDH}}^{\text{om-unpredictable}}$. $\mathcal{R} \leftrightarrow \mathcal{Z}$ is an algebraic algorithm that treats H_0 , H_1 , and H_2 as random oracles. Therefore, by Theorem 1, $\mathbf{Adv}_{\mathcal{R} \leftrightarrow \mathcal{Z}, 2\text{B-HashTDH}}^{\text{om-unpredictable}}$ is negligible.

As previously explained, $\mathcal{Z}$'s probability of distinguishing between $\mathcal{G}_2$ and $\mathcal{G}_3$ is upper-bounded by the probability of FAIL.

$$\mathbf{Dist}_{\mathcal{Z}}^{\mathcal{G}_2, \mathcal{G}_3} \leq \mathbf{Adv}_{\mathcal{R} \leftrightarrow \mathcal{Z}, 2\text{B-HashTDH}}^{\text{om-unpredictable}}$$

Game $\mathcal{G}_4$: The simulated world. The change from $\mathcal{G}_3$ to $\mathcal{G}_4$ is purely conceptual. All interactions between the simulator and the honest parties now

occur through the interface of $\mathcal{F}_{\text{tOPRF}}$, so one can imagine the monolithic simulator now cleanly split into the two components $\mathcal{F}_{\text{tOPRF}}$ and **SIM**.

$$\mathbf{Dist}_{\mathcal{Z}}^{\mathcal{G}_3, \mathcal{G}_4} = 0$$

Summing up the distinguishing advantage bounds for each incremental game change yields an overall bound on $\mathcal{Z}$'s distinguishing advantage between the real and simulated worlds.

$$\mathbf{Dist}_{\mathcal{Z}}^{\mathcal{G}_0, \mathcal{G}_4} \leq 2q^2 \cdot \frac{m}{m-1} \cdot \epsilon_{DL} + \mathbf{Adv}_{\mathcal{R} \leftrightarrow \mathcal{Z}, \text{2B-HashTDH}}^{\text{om-unpredictable}}$$

The distinguishing advantage of any efficient, algebraic $\mathcal{Z}$ between the real and simulated worlds is negligible. Thus, UC-2B-HashTDH realizes $\mathcal{F}_{\text{tOPRF}}$. $\square$